

Python, Datenanalyse und Machine Learning

Von der ersten Zeile Python bis zum eigenen ML-Workflow

Dr.-Ing. Grigory Devadze

Was Sie mitnehmen

- **Python** lesen und schreiben: Datentypen, Kontrollstrukturen, Funktionen
- Daten mit **pandas** einlesen, bereinigen, gruppieren und mit **matplotlib** und **seaborn** darstellen
- **Klassifikation, Regression und Clustering** mit scikit-learn trainieren und mit passenden Kennzahlen bewerten
- Einen vollständigen **ML-Workflow als Pipeline**, den Sie auf eigene Daten übertragen können

Ablauf

Teil	Thema
1	Python-Grundlagen
2	NumPy und pandas
3	Daten einlesen, aufbereiten, explorieren und visualisieren
4	Grundlagen von Machine Learning und KI
5	Überwachtes Lernen: Regression und Klassifikation
6	Modelltraining, Feature Engineering, Hyperparameter und Evaluation
7	Unüberwachtes Lernen: Clustering und Segmentierung
8	Einführung in Deep Learning
9	Der ML-Workflow mit scikit-learn
10	Praxisbeispiele und Interpretation von Modellen
11	Transfer in die Praxis

Kursmaterial

- Ein öffentliches Repository enthält Folien, Notebooks, Daten und Umgebungsdateien
- <https://github.com/grigory-consulting/python-ml-grundlagen>
- Das Repository bleibt nach dem Kurs erreichbar

Teil 1: Python-Grundlagen

Die Sprachbausteine, die Sie für Datenanalyse und Machine Learning brauchen: Werte, Entscheidungen, Schleifen, Sammlungen und Funktionen.

Was Sie in diesem Teil lernen

- Werte in Variablen ablegen und die Datentypen `int` , `float` , `str` , `bool` , `None` unterscheiden
- Mit Operatoren rechnen und Texte mit f-Strings formatieren
- Abläufe mit `if` , `for` und `while` steuern
- Daten in Listen, Tupeln, Sets und Dictionaries ablegen und wieder herausholen
- Eigene Funktionen schreiben, Fehlermeldungen lesen und Module importieren

Python im Überblick

- Python ist eine interpretierte, universell einsetzbare Programmiersprache
- Ihr Code wird Anweisung für Anweisung ausgeführt, ein eigener Übersetzungsschritt (Kompilieren) entfällt
- Python ist dynamisch typisiert: Sie deklarieren keinen Typ, der Wert bringt ihn mit
- Blöcke entstehen durch Einrückung, nicht durch geschweifte Klammern
- Einsatzgebiete: Datenanalyse, künstliche Intelligenz, wissenschaftliches Rechnen, Automatisierung, Webentwicklung
- Die große Standardbibliothek und die aktive Community liefern Werkzeuge für fast jede Aufgabe

Syntax und Einrückung

```
alter = 17

if alter >= 18:
    print("volljährig")           # gehört zum if-Block
    print("eigener Vertrag")     # gehört auch zum if-Block
print("Prüfung beendet")        # steht außerhalb, läuft immer

# -> Prüfung beendet
```

- Eine Anweisung pro Zeile, kein Semikolon am Zeilenende
- Ein Doppelpunkt eröffnet einen Block, die Einrückung legt fest, was dazugehört
- Üblich sind vier Leerzeichen pro Ebene. VS Code setzt sie nach dem Doppelpunkt selbst.
- `#` leitet einen Kommentar ein: Python ignoriert den Rest der Zeile
- Groß- und Kleinschreibung zählt: `alter` und `Alter` sind zwei verschiedene Namen

Die Einrückung gehört zur Syntax. Ist sie falsch, meldet Python `IndentationError` oder führt den falschen Block aus.

Variablen und Zuweisung

```
alter = 45
monatsbeitrag = 412.50
name = "Erika Muster"

alter = alter + 1           # rechts rechnen, dann links ablegen
monatsbeitrag = monatsbeitrag * 2
print(alter, monatsbeitrag) # -> 46 825.0

a = b = c = 0              # derselbe Wert für mehrere Namen
x, y, z = 1, 2, 3          # mehrere Werte auf einmal
```

- Eine Variable ist ein Name für einen Wert. `=` heißt „weise zu“, nicht „ist gleich“.
- Der Typ muss nicht deklariert werden und darf sich ändern: `alter = "unbekannt"` ist erlaubt
- Namen beginnen mit Buchstabe oder Unterstrich, danach Buchstaben, Ziffern, Unterstriche
- Konvention in Python: Kleinbuchstaben mit Unterstrich, zum Beispiel `monatsbeitrag`

Datentypen

Typ	Bedeutung	Beispiel
<code>int</code>	ganze Zahl	<code>alter = 45</code>
<code>float</code>	Gleitkommazahl	<code>monatsbeitrag = 412.50</code>
<code>str</code>	Text (Zeichenkette)	<code>plz = "01067"</code>
<code>bool</code>	Wahrheitswert	<code>raucher = False</code>
<code>None</code>	„kein Wert vorhanden“	<code>austrittsdatum = None</code>

```
print(type(45), type(412.50), type("01067"))
# -> <class 'int'> <class 'float'> <class 'str'>
print(type(False), type(None))
# -> <class 'bool'> <class 'NoneType'>
```

Eine Postleitzahl ist Text, keine Zahl: Als `int` verliert `01067` die führende Null.

Typen umwandeln

```
print("6" + "7")           # -> 67      Text wird aneinandergehängt
print(6 + 7)               # -> 13      Zahlen werden addiert
print(float("6") + float("7")) # -> 13.0

print(int("01067"))       # -> 1067   führende Null ist weg
print(str(45) + " Jahre")  # -> 45 Jahre
print(int(412.99))        # -> 412    schneidet ab, rundet nicht
print(6.0 + 7)            # -> 13.0   int und float gemischt: float
```

- `int()` , `float()` , `str()` und `bool()` wandeln einen Wert in den jeweiligen Typ um
- `"45" + 1` führt zu einem `TypeError` : Python rät nicht, was gemeint ist
- `int("abc")` führt zu einem `ValueError` : der Text enthält keine Zahl

Arithmetische Operatoren

```
print(1 + 4 * 3)      # -> 13   Punkt vor Strich
print((1 + 4) * 3)    # -> 15   Klammern legen die Reihenfolge fest
print(2 ** 10)        # -> 1024 Potenz

jahresbeitrag = 12 * 412.50
print(jahresbeitrag)  # -> 4950.0
print(round(412.456, 2)) # -> 412.46
print(abs(-5), max(5, 9), min(5, 9)) # -> 5 9 5
```

- Ein Ausdruck ist ein Wert oder eine Rechnung, die einen Wert ergibt
- Operatoren: `+`, `-`, `*`, `/`, `//`, `%`, `**`
- `**` bindet am stärksten, dann `*`, `/`, `//`, `%`, danach `+` und `-`
- Kurzformen: `summe += 10` bedeutet `summe = summe + 10`, ebenso `-=`, `*=`, `/=`

Division: /, // und %

```
print(35 / 5)      # -> 7.0      / liefert immer float
print(84 / 10)     # -> 8.4
print(84 // 10)    # -> 8        ganzzahlige Division
print(84 % 10)     # -> 4        Rest der Division (Modulo)
print(43 % 5)      # -> 3

tage = 45
print(tage // 7, "Wochen und", tage % 7, "Tage")
# -> 6 Wochen und 3 Tage
```

- `/` liefert in Python 3 immer ein `float`, auch wenn die Division aufgeht
- `//` schneidet auf die ganze Zahl ab, `%` liefert den Rest
- Typische Anwendung von `%`: gerade oder ungerade prüfen mit `n % 2 == 0`

Gleitkommazahlen sind nicht exakt: `0.1 + 0.2` ergibt `0.30000000000000004`. Geldbeträge für die Ausgabe runden.

Strings

```
stadt = "  Dresden  "
name = "Erika Muster"

print(len(name))           # -> 12
print(name[0], name[-1])   # -> E r    Index ab 0, -1: letztes
print(name[:5])            # -> Erika   Slicing: bis vor Index 5
print(name.upper(), name.lower()) # -> ERIKA MUSTER erika muster
print(stadt.strip())        # -> Dresden  Leerzeichen am Rand weg
print(name.startswith("Er")) # -> True
print(name.split(" "))      # -> ['Erika', 'Muster']
print("Erika" + " " + "Muster") # -> Erika Muster
```

- Strings stehen in einfachen oder doppelten Anführungszeichen
- Methoden lassen sich verketten: `stadt.strip().lower().startswith("dr")`
- Strings sind unveränderlich: `upper` und `strip` liefern einen **neuen** String

f-Strings

```
alter = 45
beitrag = 412.5
name = "Erika Muster"

print(f"Sie haben {65 - alter} Jahre bis zur Rente")
# -> Sie haben 20 Jahre bis zur Rente

print(f"{name}: {beitrag:.2f} EUR pro Monat")
# -> Erika Muster: 412.50 EUR pro Monat

print(f"Jahresbeitrag: {12 * beitrags:,.2f} EUR")
# -> Jahresbeitrag: 4,950.00 EUR
print(f"Anteil: {0.256:.1%}")      # -> Anteil: 25.6%
```

- Ein `f` vor dem Anführungszeichen, Ausdrücke in geschweiften Klammern
- Nach dem Doppelpunkt steht das Format: `.2f` zwei Nachkommastellen, `,` Tausendertrenner, `.1%` Prozent
- Ältere Schreibweisen, die Sie in fremdem Code sehen: `"{} Jahre".format(20)` und `"%d Jahre" % 20`

Ausgabe und Eingabe: `print` und `input`

```
print("Alter:", 45, "Jahre")           # -> Alter: 45 Jahre
print("01067", "Dresden", sep=";")     # -> 01067;Dresden
print("ohne Zeilenumbruch", end=" ")
print("weiter")                        # -> ohne Zeilenumbruch weiter

name = input("Wie heißen Sie? ")
alter = int(input("Wie alt sind Sie? "))
print(f"Hallo {name}, in {65 - alter} Jahren sind Sie 65.")
```

- `print` gibt mehrere Werte aus und setzt Leerzeichen dazwischen, `sep` und `end` ändern das
- `input` zeigt den Text an, wartet auf eine Eingabe und liefert sie **immer als String**
- Für Rechnungen die Eingabe mit `int()` oder `float()` umwandeln
- In VS Code erscheint das Eingabefeld von `input` oben am Fensterrand

Vergleiche und logische Operatoren

```
alter, arztbesuche = 45, 8

print(alter == 45)      # -> True      gleich (zwei Gleichheitszeichen)
print(alter != 45)      # -> False     ungleich
print(alter < 65)       # -> True
print(arztbesuche >= 10) # -> False

print(alter > 40 and arztbesuche > 5) # -> True   beide Seiten wahr
print(alter > 60 or arztbesuche > 5)  # -> True   mindestens eine wahr
print(not alter > 60)                 # -> True   kehrt um
print(18 <= alter < 65)                # -> True   Bereichsprüfung
```

- Vergleichsoperatoren: `==`, `!=`, `<`, `>`, `<=`, `>=`. Das Ergebnis ist immer `True` oder `False`.
- Logische Operatoren: `and`, `or`, `not`. Klammern machen lange Bedingungen lesbar.
- `=` weist zu, `==` vergleicht

if und else

```
alter = 17

if alter >= 18:
    print("volljährig")
else:
    print("minderjährig")
# -> minderjährig

if alter < 18:
    print("Mitversicherung über die Eltern prüfen")
# -> Mitversicherung über die Eltern prüfen
```

- `if` führt den eingerückten Block nur aus, wenn die Bedingung `True` ergibt
- `else` ist optional und fängt alle übrigen Fälle ab
- Bedingung, Doppelpunkt, eingerückter Block: dieselbe Form wie später bei `for`, `while` und `def`

if, elif, else

```
alter = 45

if alter < 18:
    gruppe = "unter 18"
elif alter < 40:
    gruppe = "18 bis 39"
elif alter < 65:
    gruppe = "40 bis 64"
else:
    gruppe = "65 und älter"

print(gruppe)      # -> 40 bis 64
```

- Python prüft die Bedingungen von oben nach unten und nimmt den **ersten** Treffer
- Alle weiteren Zweige werden übersprungen, deshalb genügt `alter < 40` ohne Untergrenze
- Beliebige viele `elif`, höchstens ein `else` am Ende

Truthiness: Wahrheitswert beliebiger Objekte

```
name = ""
if name:
    print("Name vorhanden")
else:
    print("Name fehlt")          # -> Name fehlt

versicherte = []
if not versicherte:
    print("Liste ist leer")      # -> Liste ist leer

print(bool(0), bool(""), bool([]), bool(None))
# -> False False False False
print(bool(7), bool("x"), bool([0]))
# -> True True True
```

- Als falsch gelten: `False`, `None`, `0`, `0.0`, leerer String, leere Liste, leeres Dict, leeres Set, leeres Tupel
- Alles andere gilt als wahr
- `if versicherte:` liest sich als „wenn die Liste etwas enthält“

for-Schleife und range

```
for jahr in range(1, 4):
    print(jahr, "Jahr(e):", jahr * 12, "Monatsbeiträge")
# -> 1 Jahr(e): 12 Monatsbeiträge
# -> 2 Jahr(e): 24 Monatsbeiträge
# -> 3 Jahr(e): 36 Monatsbeiträge

print(list(range(5)))          # -> [0, 1, 2, 3, 4]
print(list(range(2, 6)))       # -> [2, 3, 4, 5]
print(list(range(10, 0, -2)))  # -> [10, 8, 6, 4, 2]
```

- `for variable in folge:` führt den Block einmal pro Element aus
- `range(start, stop, step)` : `start` zählt mit, `stop` zählt **nicht** mit
- `range(5)` beginnt bei 0 und liefert fünf Werte

for über Listen und Strings, enumerate

```
staedte = ["Berlin", "Dresden", "Köln"]

for stadt in staedte:
    print(stadt.upper())
# -> BERLIN DRESDEN KÖLN    (je eine Zeile)

for zeichen in "PLZ":
    print(zeichen, end=" ")      # -> P L Z

for nr, stadt in enumerate(staedte, start=1):
    print(nr, stadt)
# -> 1 Berlin
# -> 2 Dresden
# -> 3 Köln
```

- Eine `for`-Schleife läuft über alles, was Elemente hat: Listen, Strings, Tupel, Dicts, Dateien
- Ein Zähler von Hand (`i = i + 1`) ist fast nie nötig
- `enumerate` liefert Position und Wert zusammen. Ohne `start` beginnt die Zählung bei 0.

Kumulative Schleifen: summieren und zählen

```
leistungen = [120.0, 0.0, 845.5, 60.0, 0.0, 310.0]

summe = 0
anzahl_ohne = 0
for betrag in leistungen:
    summe += betrag
    if betrag == 0:
        anzahl_ohne += 1

print("Summe:", summe)                # -> Summe: 1335.5
print("ohne Leistung:", anzahl_ohne)  # -> ohne Leistung: 2
print("Mittel:", round(summe / len(leistungen), 2))  # -> Mittel: 222.58
```

- Das Muster: Startwert **vor** der Schleife setzen, in der Schleife fortschreiben, danach ausgeben
- Für Summen ist der Startwert 0, für Produkte 1
- Abkürzungen für Standardfälle: `sum(leistungen)`, `len(leistungen)`, `max(leistungen)`

while-Schleife

```
guthaben = 1000.0
monate = 0

while guthaben >= 412.5:
    guthaben -= 412.5
    monate += 1

print(monate, "Monate gedeckt, Rest:", guthaben)
# -> 2 Monate gedeckt, Rest: 175.0
```

- `while` wiederholt den Block, solange die Bedingung wahr ist
- Geeignet, wenn vorher nicht feststeht, wie oft wiederholt wird
- Ist die Bedingung schon zu Beginn falsch, läuft der Block kein einziges Mal
- Im Block muss sich etwas ändern, das die Bedingung irgendwann falsch macht

Endlosschleife im Notebook: Die Zelle zeigt dauerhaft den Laufindikator. Mit **Unterbrechen** in der Werkzeugleiste stoppen.

break und continue

```
plz_liste = ["10115", "01067", "", "5066", "50667", "STOP", "80331"]

for plz in plz_liste:
    if plz == "STOP":
        break                # Schleife sofort verlassen
    if len(plz) != 5:
        continue            # diesen Wert überspringen
    print(plz, end=" ")
# -> 10115 01067 50667
```

- `break` beendet die Schleife vollständig
- `continue` springt zum nächsten Durchlauf, der Rest des Blocks entfällt
- Beide gelten für `for` und `while` und wirken nur auf die innerste Schleife

Listen

```
alter = [34, 51, 27, 68, 45]
gemischt = [1, "Dresden", 412.5, True]      # Typen dürfen gemischt sein
leer = []

print(len(alter))          # -> 5
print(51 in alter)         # -> True
print(min(alter), max(alter), sum(alter))    # -> 27 68 225

matrix = [[1, 2, 3],
           [4, 5, 6]]      # Liste von Listen
print(matrix[1][2])        # -> 6
```

- Eine Liste ist eine geordnete, veränderbare Sammlung in eckigen Klammern
- Die Reihenfolge bleibt erhalten, Duplikate sind erlaubt
- Listen gehören zu den meistverwendeten Datenstrukturen in Python
- Eine Spalte einer Tabelle können Sie sich vorerst als Liste vorstellen

Listen: Indexing und Slicing

```
beitraege = [310, 325, 340, 355, 370]

print(beitraege[0])      # -> 310           erstes Element
print(beitraege[-1])     # -> 370           letztes Element
print(beitraege[1:4])    # -> [325, 340, 355] Index 1 bis vor 4
print(beitraege[:3])     # -> [310, 325, 340] die ersten drei
print(beitraege[3:])     # -> [355, 370]     ab Index 3
print(beitraege[::2])    # -> [310, 340, 370] jedes zweite
print(beitraege[::-1])   # -> [370, 355, 340, 325, 310] umgekehrt
```

Index	0	1	2	3	4
Wert	310	325	340	355	370
negativer Index	-5	-4	-3	-2	-1

- Der Index beginnt bei 0. Negative Indizes zählen vom Ende.
- `liste[start:stop:step]` : `stop` zählt nicht mit, das Ergebnis ist eine neue Liste

Listen verändern

```
staedte = ["Berlin", "Dresden", "Köln"]

staedte[1] = "Leipzig"           # Element ersetzen
staedte.append("Hamburg")        # am Ende anfügen
staedte.insert(1, "Bonn")        # an Index 1 einfügen
print(staedte)
# -> ['Berlin', 'Bonn', 'Leipzig', 'Köln', 'Hamburg']

staedte.remove("Bonn")           # nach Wert entfernen
letzte = staedte.pop()           # letztes entfernen und zurückgeben
print(letzte)                   # -> Hamburg
print(staedte)                  # -> ['Berlin', 'Leipzig', 'Köln']

print(staedte + ["Kiel", "Ulm"])
# -> ['Berlin', 'Leipzig', 'Köln', 'Kiel', 'Ulm']
```

- Listen sind veränderbar (mutable): Methoden wie `append` ändern die Liste selbst und geben `None` zurück

`staedte = staedte.append("Kiel")` zerstört die Liste: Danach ist `staedte` gleich `None`.

Listen: wichtige Methoden

Aufruf	Wirkung
<code>len(liste)</code>	Anzahl der Elemente
<code>liste.append(x)</code>	fügt <code>x</code> am Ende an
<code>liste.insert(i, x)</code>	fügt <code>x</code> an Index <code>i</code> ein
<code>liste.remove(x)</code>	entfernt das erste Vorkommen von <code>x</code>
<code>liste.pop(i)</code>	entfernt das Element an Index <code>i</code> und gibt es zurück (ohne <code>i</code> : das letzte)
<code>liste.index(x)</code>	Index des ersten Vorkommens von <code>x</code>
<code>liste.count(x)</code>	zählt, wie oft <code>x</code> vorkommt
<code>liste.sort()</code>	sortiert die Liste selbst
<code>liste.reverse()</code>	kehrt die Reihenfolge um
<code>liste.copy()</code>	flache Kopie
<code>liste.clear()</code>	entfernt alle Elemente

```
alter = [34, 51, 27, 68, 27]
print(alter.count(27))    # -> 2
alter.sort()
print(alter)              # -> [27, 27, 34, 51, 68]
```

Listen kopieren

```
import copy

a = [[1, 2], [3, 4]]
b = a                # keine Kopie: zweiter Name für dieselbe Liste
c = a.copy()         # flache Kopie: neue äußere Liste, innere geteilt
d = copy.deepcopy(a) # tiefe Kopie: alles wird dupliziert

a.append([5, 6])      # Änderung an der äußeren Liste
print(b)              # -> [[1, 2], [3, 4], [5, 6]]    b ist a
print(c)              # -> [[1, 2], [3, 4]]            c bleibt

a[0][0] = 99          # Änderung in einer inneren Liste
print(c)              # -> [[99, 2], [3, 4]]            schlägt durch
print(d)              # -> [[1, 2], [3, 4]]            unabhängig
```

- `b = a` kopiert nichts: beide Namen zeigen auf dieselbe Liste
- `copy()` legt eine neue äußere Liste an, die inneren Objekte bleiben gemeinsam
- `copy.deepcopy()` dupliziert auch alle inneren Objekte

Tupel

```
versicherter = ("Erika Muster", 45, "01067")

print(versicherter[0])      # -> Erika Muster
print(versicherter[:2])     # -> ('Erika Muster', 45)
print(len(versicherter))    # -> 3

einzeln = (45,)             # ein Element: Komma nicht vergessen

versicherter[1] = 46
# -> TypeError: 'tuple' object does not support item assignment
```

- Ein Tupel ist eine unveränderliche Liste: runde Klammern, Elemente durch Kommas getrennt
- Indexing und Slicing funktionieren wie bei Listen
- Ändern, Anfügen und Entfernen sind nicht möglich
- Einsatz: feste Wertegruppen wie Koordinaten, eine Tabellenzeile, mehrere Rückgabewerte

Tupel entpacken

```
versicherter = ("Erika Muster", 45, "01067")
name, alter, plz = versicherter          # Unpacking
print(name, plz)                         # -> Erika Muster 01067

standorte = {(52.5, 13.4): "Berlin", (48.8, 2.3): "Paris"}
print(standorte[(52.5, 13.4)])           # -> Berlin

def min_max(werte):
    return min(werte), max(werte)         # gibt ein Tupel zurück

juengste, aelteste = min_max([34, 51, 27, 68])
print(juengste, aelteste)                 # -> 27 68

print(tuple([1, 2, 3]), list((1, 2, 3))) # -> (1, 2, 3) [1, 2, 3]
```

- Links so viele Namen wie rechts Werte: Python verteilt sie der Reihe nach
- Tupel sind als Schlüssel in einem Dict erlaubt, Listen nicht
- `tuple()` und `list()` wandeln ineinander um

Sets

```
plz_liste = ["10115", "01067", "10115", "50667", "01067"]

eindeutig = set(plz_liste)
print(eindeutig)          # -> {'10115', '01067', '50667'}
print(len(eindeutig))     # -> 3
print("50667" in eindeutig) # -> True

eindeutig.add("80331")
eindeutig.discard("99999") # kein Fehler, wenn der Wert fehlt
leeres_set = set()        # {} wäre ein leeres Dict
```

- Ein Set ist eine ungeordnete Sammlung **eindeutiger** Elemente: keine Duplikate, kein Index, die Reihenfolge der Ausgabe ist beliebig
- Typische Aufgaben: Duplikate entfernen, Zugehörigkeit prüfen, Mengen vergleichen
- Elemente müssen unveränderlich sein: Zahlen, Strings, Tupel, aber keine Listen

Mengenoperationen

```
kurs_a = {"Anna", "Marc", "Lisa", "Ben"}
kurs_b = {"Lisa", "Marc", "Eva"}

print(kurs_a & kurs_b)      # -> {'Lisa', 'Marc'}           in beiden
print(kurs_a | kurs_b)     # -> alle fünf Namen           in A oder B
print(kurs_a - kurs_b)     # -> {'Anna', 'Ben'}           nur in A
print(kurs_a ^ kurs_b)     # -> {'Anna', 'Ben', 'Eva'}     in genau einem
```

Operator	Bedeutung
&	Schnittmenge
	Vereinigung
-	Differenz
^	symmetrische Differenz

Anwendung: Welche Versichertennummern stehen in Datei A, fehlen aber in Datei B?

Dictionaries

```
versicherter = {  
    "name": "Erika Muster",  
    "alter": 45,  
    "plz": "01067",  
    "raucher": False,  
}  
  
print(versicherter["alter"])           # -> 45  
versicherter["alter"] = 46             # Wert ändern  
versicherter["stadt"] = "Dresden"      # neues Paar anlegen  
print(len(versicherter))              # -> 5  
print(versicherter)  
# -> {'name': 'Erika Muster', 'alter': 46, 'plz': '01067',  
#     'raucher': False, 'stadt': 'Dresden'}
```

- Ein Dictionary (Dict) speichert Schlüssel-Wert-Paare: Zugriff über den Schlüssel, nicht über eine Position
- Schlüssel sind eindeutig und unveränderlich (meist Strings), Werte dürfen alles sein
- Einsatz: Nachschlagen, Konfigurationen, Zählungen, eine Tabellenzeile mit benannten Feldern

Dictionaries: sicher zugreifen mit `get`

```
beitrag = {"10115": 398.0, "01067": 412.5}

# beitrag["50667"]          -> KeyError: '50667'

print(betrag.get("50667"))   # -> None
print(betrag.get("50667", 0.0)) # -> 0.0   eigener Ersatzwert
print("01067" in beitrag)    # -> True   prüft die Schlüssel

del beitrag["10115"]         # Paar löschen
wert = beitrag.pop("01067")  # löschen und Wert zurückgeben
print(wert, beitrag)         # -> 412.5 {}
```

- `d[schluesseel]` wirft einen `KeyError`, wenn der Schlüssel fehlt
- `d.get(schluesseel, ersatz)` liefert stattdessen den Ersatzwert, ohne Angabe `None`
- `in` prüft Schlüssel, nicht Werte

Dictionaries durchlaufen

```
anzahl = {"Berlin": 1200, "Dresden": 640, "Köln": 910}

for stadt in anzahl:                # läuft über die Schlüssel
    print(stadt, anzahl[stadt])

for stadt, n in anzahl.items():      # Schlüssel und Wert zusammen
    print(f"{stadt:<8}{n:>6}")
# -> Berlin      1200
# -> Dresden      640
# -> Köln         910

print(list(anzahl.keys()))           # -> ['Berlin', 'Dresden', 'Köln']
print(list(anzahl.values()))          # -> [1200, 640, 910]
print(sum(anzahl.values()))           # -> 2750
```

- `items()` liefert Tupel aus Schlüssel und Wert, das Unpacking im Schleifenkopf verteilt sie
- `keys()` und `values()` liefern nur die eine Seite
- Die Reihenfolge entspricht der Reihenfolge des Einfügens

List Comprehensions

```
alter = [34, 51, 27, 68, 45]

# klassisch mit Schleife
ab_40 = []
for a in alter:
    if a >= 40:
        ab_40.append(a)

# dasselbe als Comprehension
ab_40 = [a for a in alter if a >= 40]
print(ab_40)                                # -> [51, 68, 45]

in_5_jahren = [a + 5 for a in alter]
print(in_5_jahren)                          # -> [39, 56, 32, 73, 50]
print([x * x for x in range(1, 6)])         # -> [1, 4, 9, 16, 25]
```

- Form: `[ausdruck for element in folge if bedingung]`, der `if`-Teil ist optional
- Lesen Sie von der Mitte aus: erst `for`, dann `if`, zuletzt der Ausdruck vorn

Dict Comprehensions

```
print({x: x * x for x in range(5)})  
# -> {0: 0, 1: 1, 2: 4, 3: 9, 4: 16}  
  
staedte = ["Berlin", "Dresden", "Köln"]  
laenge = {s: len(s) for s in staedte}  
print(laenge)      # -> {'Berlin': 6, 'Dresden': 7, 'Köln': 4}  
  
kuerzel = {"deu": "Deutsch", "eng": "Englisch"}  
umgedreht = {wert: key for key, wert in kuerzel.items()}  
print(umgedreht)   # -> {'Deutsch': 'deu', 'Englisch': 'eng'}
```

Schreibweise	Ergebnis
<code>[x * x for x in werte]</code>	Liste
<code>{x: x * x for x in werte}</code>	Dict
<code>{x * x for x in werte}</code>	Set

Funktionen: def und return

```
def jahresbeitrag(monatsbeitrag):  
    """Rechnet einen Monatsbeitrag auf zwölf Monate hoch."""  
    ergebnis = 12 * monatsbeitrag  
    return ergebnis  
  
print(jahresbeitrag(412.5))          # -> 4950.0  
summe = jahresbeitrag(398.0) + jahresbeitrag(412.5)  
print(summe)                        # -> 9726.0
```

- `def name(parameter):` definiert die Funktion, der eingerückte Block ist ihr Rumpf
- `return` gibt einen Wert zurück und beendet die Funktion. Ohne `return` liefert sie `None`.
- Parameter stehen in der Definition, Argumente sind die Werte beim Aufruf
- Der Docstring in dreifachen Anführungszeichen erscheint bei `help(jahresbeitrag)`
- Variablen aus dem Rumpf (`ergebnis`) existieren außerhalb der Funktion nicht

Defaultwerte und Keyword-Argumente

```
def beitrags(einkommen, satz=0.15, zuschlag=0.0):  
    return einkommen * satz + zuschlag  
  
print(beitrags(3000))                # -> 450.0 beide Defaults  
print(beitrags(3000, 0.16))          # -> 480.0 nach Position  
print(beitrags(3000, zuschlag=25.0))  # -> 475.0 per Name  
print(beitrags(zuschlag=25.0, einkommen=3000, satz=0.16))  # -> 505.0
```

- Ein Parameter mit Defaultwert darf beim Aufruf fehlen
- Parameter ohne Default stehen in der Definition vor denen mit Default
- Keyword-Argumente (`name=wert`) sind unabhängig von der Reihenfolge und machen Aufrufe lesbar

So lesen Sie später Aufrufe wie `pd.read_csv("data/titanic.csv", sep=",")`: ein Pflichtargument nach Position, der Rest per Name mit Defaults.

Mehrere Rückgabewerte, `*args` und `**kwargs`

```
def min_max(werte):  
    return min(werte), max(werte)           # zwei Werte: ein Tupel  
  
lo, hi = min_max([34, 51, 27, 68])  
print(lo, hi)                             # -> 27 68  
  
def addiere_alle(*args):                   # beliebig viele Argumente  
    return sum(args)                       # args ist ein Tupel  
  
def info_ausgeben(**kwargs):               # beliebig viele Keyword-Argumente  
    for key, value in kwargs.items():      # kwargs ist ein Dict  
        print(f"{key}: {value}")  
  
print(addiere_alle(1, 2, 3, 4))            # -> 10  
info_ausgeben(alter=50, stadt="Wuppertal")  
# -> alter: 50   stadt: Wuppertal   (je eine Zeile)
```

- Mehrere Rückgabewerte sind ein Tupel, das Sie beim Aufruf entpacken
- `*args` und `**kwargs` müssen Sie im Kurs nicht selbst schreiben, aber in Dokumentationen erkennen

Type Hints

```
def jahresbeitrag(monatsbeitrag: float, monate: int = 12) -> float:
    return monatsbeitrag * monate

def verdoppeln(werte: list[int]) -> list[int]:
    return [2 * x for x in werte]

print(jahresbeitrag(412.5))           # -> 4950.0
print(jahresbeitrag("ab", 3))         # -> ababab   kein Fehler!

print(isinstance(412.5, float))       # -> True     Typprüfung zur Laufzeit
```

- Type Hints geben an, welche Typen eine Funktion erwartet und zurückgibt
- Python prüft sie zur Laufzeit **nicht**: es sind Hinweise für Menschen, Editoren und Prüfwerkzeuge wie `mypy`
- VS Code nutzt sie für Autovervollständigung und Warnungen
- Mit `isinstance(obj, Typ)` prüfen Sie einen Typ zur Laufzeit selbst

Fehlermeldungen lesen

```
def jahresbeitrag(monatsbeitraege):  
    summe = 0.0  
    for b in monatsbeitraege:  
        summe = summe + b  
    return summe  
  
print(jahresbeitrag([412.5, 412.5, "412,5"]))
```

```
Traceback (most recent call last):  
  File "beitrag.py", line 7, in <module>  
    print(jahresbeitrag([412.5, 412.5, "412,5"]))  
  File "beitrag.py", line 4, in jahresbeitrag  
    summe = summe + b  
TypeError: unsupported operand type(s) for +: 'float' and 'str'
```

- Von **unten nach oben** lesen: Die letzte Zeile nennt Fehlertyp und Ursache
- Darüber steht die Zeile mit dem Fehler: Zeile 4, in `jahresbeitrag`
- Weiter oben folgt die Aufrufkette bis zu Ihrer eigenen Zelle

Häufige Fehlertypen

Fehlertyp	Typische Ursache	Beispiel
<code>SyntaxError</code>	Doppelpunkt, Klammer oder Anführungszeichen fehlt	<code>if alter > 18</code>
<code>IndentationError</code>	Einrückung fehlt oder passt nicht	Block nach <code>if</code> nicht eingerückt
<code>NameError</code>	Name unbekannt: Tippfehler oder Zelle nicht ausgeführt	<code>print(altr)</code>
<code>TypeError</code>	Operation passt nicht zum Typ	<code>"45" + 1</code>
<code>ValueError</code>	Typ stimmt, Wert nicht	<code>int("abc")</code>
<code>IndexError</code>	Index außerhalb der Liste	<code>liste[10]</code>
<code>KeyError</code>	Schlüssel fehlt im Dict	<code>d["stadt"]</code>
<code>ZeroDivisionError</code>	Division durch 0	<code>summe / 0</code>
<code>ModuleNotFoundError</code>	Paket fehlt oder falscher Kernel	<code>import pandas</code>

Die letzte Zeile der Meldung vollständig lesen, bevor Sie etwas ändern. Sie nennt fast immer die Ursache.

Fehler abfangen mit `try` und `except`

```
eingaben = ["412.5", "398", "k.A.", "405.0"]
beträge = []

for text in eingaben:
    try:
        beträge.append(float(text))
    except ValueError as e:
        print("Übersprungen:", e)

print(beträge)
# -> Übersprungen: could not convert string to float: 'k.A.'
# -> [412.5, 398.0, 405.0]
```

- Python versucht den `try`-Block. Tritt der genannte Fehler auf, läuft stattdessen der `except`-Block.
- Das Programm läuft danach weiter, statt abzuberechnen
- Fangen Sie den **konkreten** Fehlertyp ab, nicht pauschal alles
- Optional: `else` läuft nur ohne Fehler, `finally` läuft immer

Module importieren

```
import math
print(math.sqrt(16), math.floor(2.9), math.ceil(2.3))    # -> 4.0 2 3
print(math.pi)                                           # -> 3.141592653589793

from math import sqrt, log10
print(sqrt(16), log10(100))                              # -> 4.0 2.0

import random as rd
rd.seed(42)
print(rd.randint(0, 9))                                  # ganze Zufallszahl von 0 bis 9
```

Schreibweise	Aufruf danach
<code>import math</code>	<code>math.sqrt(16)</code>
<code>from math import sqrt</code>	<code>sqrt(16)</code>
<code>import pandas as pd</code>	<code>pd.read_csv(...)</code>

- Ein Modul ist eine Datei mit fertigen Funktionen. Die Standardbibliothek ist installiert, Pakete wie pandas kommen über `pip` dazu.

Objekte und Methoden: was `df.head()` bedeutet

```
name = "erika muster"
print(name.title())           # -> Erika Muster   Methode eines str

alter = [34, 51, 27]
alter.append(68)              # Methode eines list-Objekts
print(type(alter))            # -> <class 'list'>

import pandas as pd
df = pd.read_csv("data/versicherte.csv")    # df ist ein DataFrame-Objekt
df.head()                                  # Methode: die ersten fünf Zeilen
df.shape                                   # Attribut ohne Klammern: (Zeilen, Spalten)
```

- In Python ist jeder Wert ein Objekt mit einem Typ (einer Klasse)
- Eine **Methode** ist eine Funktion, die zum Objekt gehört: `objekt.methode(argumente)`
- Ein **Attribut** ist ein Datenwert des Objekts: `objekt.attribut`, ohne Klammern
- Der Typ bestimmt, welche Methoden es gibt: `str` hat `title`, `list` hat `append`, `DataFrame` hat `head`

`df.head()` heißt: „Objekt `df`, führe deine Methode `head` aus.“ Welche Methoden ein Objekt hat, zeigt Tab nach dem Punkt.

Zusammenfassung

- Fünf Grundtypen: `int` , `float` , `str` , `bool` , `None` . `/` liefert immer `float` , `//` und `%` rechnen ganzzahlig.
- Einrückung bildet Blöcke: `if` / `elif` / `else` entscheidet, `for` läuft über Elemente, `while` läuft bis zu einer Bedingung
- Vier Sammlungen: Liste (geordnet, veränderbar), Tupel (unveränderlich), Set (eindeutig), Dict (Schlüssel und Wert, sicherer Zugriff mit `get`)
- Funktionen mit `def` , `return` , Defaultwerten und Keyword-Argumenten: so sind auch pandas und scikit-learn aufgebaut
- Fehlermeldungen von unten nach oben lesen, Module mit `import` einbinden, Methoden mit `objekt.methode()` aufrufen

Teil 2: NumPy und pandas

Mit Arrays rechnen und Tabellen auswerten: die beiden Bibliotheken, auf denen Datenanalyse in Python meist aufbaut.

Was Sie in diesem Teil lernen

- Sie legen NumPy-Arrays an und rechnen ohne Schleife mit ganzen Spalten
- Sie wählen Werte mit Indizes, Slices und Boolean-Masken aus
- Sie lesen eine CSV-Datei als DataFrame ein und verschaffen sich einen Überblick
- Sie filtern, sortieren und ergänzen Spalten (BMI, Altersgruppe)
- Sie werten Tabellen mit `groupby` aus und verbinden zwei Tabellen mit `merge`

NumPy, pandas, matplotlib: wer macht was

Bibliothek	Kernobjekt	Wofür
NumPy	<code>ndarray</code>	schnelles Rechnen mit Zahlenfeldern (Vektoren, Matrizen)
pandas	<code>Series</code> , <code>DataFrame</code>	Tabellen mit Spaltennamen, gemischten Typen und fehlenden Werten
matplotlib / seaborn	<code>Figure</code> , <code>Axes</code>	Diagramme

```
import numpy as np
import pandas as pd
```

Die Kürzel `np` und `pd` sind Konvention: Die Dokumentation und fast alle Beispiele verwenden sie.

ndarray anlegen, shape und dtype

```
a = np.array([1, 2, 3, 4])          # aus einer Liste
m = np.array([[1, 2], [3, 4]])      # verschachtelte Liste: Matrix

a.shape      # -> (4,)      ein Eintrag: Vektor mit 4 Werten
m.shape      # -> (2, 2)    (Zeilen, Spalten)
a.dtype      # -> int64     ein Typ für alle Elemente

np.zeros((2, 4))      # 2x4 Nullen
np.ones((4, 2))       # 4x2 Einsen
np.arange(0, 1, 0.25) # -> [0.  0.25 0.5  0.75]
np.linspace(0, 1, 5)  # -> [0.  0.25 0.5  0.75 1.  ]
np.eye(3)            # 3x3 Einheitsmatrix
```

Eine Liste darf Typen mischen, ein Array nicht: `np.array([4., 5, 6])` wird komplett zu `float64`.

Rechnen mit ganzen Arrays

```
a = np.array([1, 2, 3])
b = np.array([4, 5, 6])

a + b      # -> [5 7 9]
a * 2      # -> [2 4 6]
a ** 2     # -> [1 4 9]
a * b      # -> [ 4 10 18]   elementweise, kein Matrixprodukt
np.sqrt(b) # -> [2.   2.236 2.449]
```

- Operatoren wirken **elementweise** auf das ganze Array
- NumPy-Funktionen wie `np.sqrt`, `np.log`, `np.round` ebenfalls
- Mit Listen geht das nicht: `[1, 2, 3] * 2` ergibt `[1, 2, 3, 1, 2, 3]`

Vektorisierung statt Schleife

Schleife über jedes Element

```
import time
rng = np.random.default_rng(42)
arr = rng.random(1_000_000)

start = time.time()
out = np.zeros_like(arr)
for i in range(arr.size):
    out[i] = arr[i] ** 2
print(time.time() - start, "s")
```

Vektorisiert

```
start = time.time()
out = arr ** 2
print(time.time() - start, "s")
```

Gleiches Ergebnis, eine Zeile, und die Rechnung läuft in kompiliertem Code statt im Python-Interpreter.

Wenn Sie über ein Array oder eine DataFrame-Spalte eine for-Schleife schreiben wollen: erst prüfen, ob es einen Array-Ausdruck dafür gibt.

Broadcasting

NumPy rechnet auch mit Arrays **unterschiedlicher Form**, wenn sich die kleinere Form auf die größere „strecken“ lässt.

```
m = np.array([[1, 2, 3],
               [4, 5, 6]])          # shape (2, 3)
v = np.array([10, 20, 30])         # shape (3,)

m + 100      # Skalar auf jedes Element
# -> [[101 102 103]
#      [104 105 106]]

m + v        # v wird auf jede Zeile angewendet
# -> [[11 22 33]
#      [14 25 36]]
```

Regel: Die Formen werden von rechts verglichen. Jede Dimension muss gleich sein oder 1. (2, 3) und (3,) passt, (2, 3) und (2,) ergibt einen ValueError.

Indizierung und Slicing

```
data = np.array([[1, 2, 3],
                 [4, 5, 6],
                 [7, 8, 9]])

data[0, 1]      # -> 2           erste Zeile, zweite Spalte
data[:, 1]      # -> [2 5 8]     zweite Spalte
data[2, :]      # -> [7 8 9]     dritte Zeile
data[1:3, :2]   # -> [[4 5]
                 #       [7 8]]   Zeilen 1 bis 2, Spalten 0 bis 1
```

- Schreibweise bei zwei Dimensionen: `[Zeile, Spalte]`, Zählung ab 0
- `:` allein heißt „alles in dieser Richtung“
- Wie bei Listen: Der Endindex eines Slice gehört nicht mehr dazu

Boolean-Masken

```
alter = np.array([34, 71, 19, 66, 45, 82])

maske = alter >= 65
maske          # -> [False  True False  True False  True]
alter[maske]   # -> [71 66 82]

maske.sum()    # -> 3      True zählt als 1
maske.mean()   # -> 0.5    Anteil der Treffer

alter[(alter >= 18) & (alter < 65)]  # -> [34 19 45]
```

Bedingungen verknüpfen Sie mit & (und), | (oder), ~ (nicht). Jede Bedingung steht in Klammern. and und or funktionieren mit Arrays nicht.

Aggregationen und axis

```
data = np.array([[1, 2, 3],
                 [4, 5, 6]])

data.sum()           # -> 21           über alles
data.mean()          # -> 3.5
data.std()           # -> 1.7078...
data.max()           # -> 6

data.sum(axis=0)      # -> [5 7 9]      je Spalte
data.sum(axis=1)      # -> [ 6 15]      je Zeile
data.mean(axis=0)     # -> [2.5 3.5 4.5]
```

axis nennt die Richtung, die **verschwindet**: axis=0 fasst die Zeilen zusammen (ein Wert je Spalte), axis=1 fasst die Spalten zusammen (ein Wert je Zeile).

Zufallszahlen mit `default_rng(42)`

```
rng = np.random.default_rng(42)           # Generator mit festem Startwert

rng.random(3)                             # 3 Werte gleichverteilt in [0, 1)
rng.integers(1, 7, size=5)                 # 5 Würfelwürfe (7 ausgeschlossen)
rng.normal(loc=170, scale=10, size=5)      # normalverteilt um 170

# Simulation: zwei Würfel, 10 000 Würfe. Wie oft ist die Summe 7?
wuerfe = rng.integers(1, 7, size=(10_000, 2))
summen = wuerfe.sum(axis=1)
(summen == 7).mean()                      # -> ca. 0.167 (Theorie: 1/6)
```

- Derselbe Startwert liefert bei jedem Lauf dieselben Zahlen: Ergebnisse bleiben **reproduzierbar**
- In diesem Kurs ist der Startwert bei NumPy immer `42`, bei scikit-learn heißt der Parameter `random_state` und steht immer auf `1`

Matrixprodukt mit @

```
a = np.array([1, 2, 3])
b = np.array([4, 5, 6])

a * b      # -> [ 4 10 18]   elementweise
a @ b      # -> 32          Skalarprodukt: 1*4 + 2*5 + 3*6

A = np.array([[5, 10], [15, 20], [25, 30]])    # shape (3, 2)
B = np.array([[1, 2], [3, 4]])                 # shape (2, 2)

A @ B      # shape (3, 2)
# -> [[ 35  50]
#      [ 75 110]
#      [115 170]]
```

Die inneren Dimensionen müssen übereinstimmen: $(3, 2) @ (2, 2)$ ergibt $(3, 2)$. `np.dot(A, B)` rechnet dasselbe.

pandas: Series und DataFrame

Series: eine Spalte mit Index

```
s = pd.Series([72, 85, 64],  
              name="gewicht_kg")  
s.mean()      # -> 73.666...
```

DataFrame: Tabelle aus mehreren Series mit gemeinsamem Index

```
df = pd.DataFrame({  
    "vorname": ["Anna", "Ben", "Cem"],  
    "alter": [25, 41, 30],  
    "gewicht_kg": [72, 85, 64],  
})
```

	vorname	alter	gewicht_kg
0	Anna	25	72
1	Ben	41	85
2	Cem	30	64

- Links steht der **Index** (Zeilenbeschriftung)
- Jede Spalte hat **einen** Datentyp
- `df["alter"]` ist eine Series, `df["alter"].to_numpy()` ein NumPy-Array
- Von den NumPy-Folien gilt weiter: Rechnen, Masken, `mean`, `sum`

CSV-Datei einlesen

```
import pandas as pd

df = pd.read_csv("data/versicherte.csv")

df.shape          # -> (5025, 18)
df.head()         # die ersten 5 Zeilen
df.tail(10)       # die letzten 10 Zeilen
df.sample(5, random_state=1) # 5 zufällige Zeilen
```

- Der Pfad ist relativ zum aktuellen Arbeitsverzeichnis
- `head()` und `tail()` nach jedem Einlesen: Stimmen Spaltennamen und Werte?
- Der Datensatz ist **synthetisch**: rund 5000 erfundene Versicherte, keine echten Personen

Überblick mit `info` und `describe`

```
df.info()
```

```
RangeIndex: 5025 entries, 0 to 5024  
Data columns (total 18 columns):  
#   Column                Non-Null Count  Dtype  
0   versicherten_nr        5025 non-null   ...  
4   alter                  5025 non-null   int64  
...
```

```
df.describe()                # Kennzahlen aller Zahlenspalten  
df["leistungsausgaben_eur"].describe()  
df.describe(include="all").T  # auch Textspalten, gedreht
```

- `info()` : Spaltennamen, Datentyp, Anzahl vorhandener Werte
- `describe()` : Anzahl, Mittelwert, Standardabweichung, Minimum, Quartile (25 %, 50 % = Median, 75 %), Maximum

Spalten auswählen

```
df["bmi"]                                # eine Spalte: Series
df[ ["vorname", "nachname", "bundesland"] ]  # mehrere Spalten: DataFrame

df["bmi"].mean()
df["bmi"].max()

klein = df[ ["vorname", "nachname", "geburtsdatum"] ].copy()
```

- Einfache Klammer mit Name: **Series**. Doppelte Klammer mit Liste: **DataFrame**.
- `df.bmi` funktioniert auch, aber nicht bei Leerzeichen im Namen und nicht beim Anlegen neuer Spalten. Bleiben Sie bei `df["bmi"]`.
- `.copy()` erzeugt eine unabhängige Kopie, die Sie gefahrlos verändern können

Zeilen und Spalten mit `loc` und `iloc`

`loc` : nach Beschriftung

```
df.loc[5, "groesse_cm"]  
df.loc[:, "gewicht_kg"]  
df.loc[100:200, "beruf":"raucher"]  
df.loc[[5, 10], ["alter", "bmi"]]  
  
# Wert setzen  
df.loc[5, "groesse_cm"] = 166
```

Slices mit `loc` schließen das Ende **ein**: `100:200` sind 101 Zeilen.

`iloc` : nach Position

```
df.iloc[5, 12]  
df.iloc[:, 13]  
df.iloc[100:200, 9:12]  
df.iloc[:5, :3]  
  
# erste und letzte Zeile  
df.iloc[ [0, -1] ]
```

Slices mit `iloc` schließen das Ende **aus**, wie bei Listen und NumPy.

Im Alltag fast immer `loc`: Spaltennamen bleiben gültig, auch wenn sich die Reihenfolge der Spalten ändert.

Filtern mit Masken

```
maske = df["alter"] >= 65
df[maske]                                # nur Zeilen mit True in der Maske

# mehrere Bedingungen: Frauen von 18 bis unter 30, Nichtraucherinnen
auswahl = df[
    (df["alter"] >= 18)
    & (df["alter"] < 30)
    & (df["geschlecht"] == "weiblich")
    & (~df["raucher"])
]
len(auswahl)                            # Anzahl Treffer

df[df["bundesland"].isin(["Berlin", "Brandenburg"])]
df[df["leistungsausgaben_eur"].between(1000, 2000)]
```

Dieselben Regeln wie bei NumPy: `&`, `|`, `~` und jede Bedingung in Klammern.

Filtern mit query

```
df.query("geschlecht == 'männlich' and bmi > 30")

df.query("alter >= 65 and raucher == True")

grenze = 5000
df.query("leistungsausgaben_eur > @grenze")

df.query("bundesland in ['Berlin', 'Hamburg', 'Bremen'])")
```

- Die Bedingung steht als Text da, Spaltennamen ohne `df[...]`
- Hier sind `and`, `or`, `not` erlaubt
- Python-Variablen sprechen Sie mit `@name` an

Maske und query liefern dasselbe Ergebnis. Nehmen Sie, was Sie in drei Wochen noch lesen können.

Neue Spalten berechnen

```
# BMI = Gewicht in kg / (Größe in m)^2
groesse_m = df["groesse_cm"] / 100
df["bmi_neu"] = (df["gewicht_kg"] / groesse_m ** 2).round(1)

# Kontrolle gegen die vorhandene Spalte
(df["bmi_neu"] - df["bmi"]).abs().max()

# Ausgaben je Arztbesuch, Flag-Spalte
df["ausgaben_je_besuch"] = (
    df["leistungsausgaben_eur"] / df["arztbesuche_jahr"]
)
df["senior"] = df["alter"] >= 65
```

- Die Rechnung läuft vektorisiert über alle 5025 Zeilen, ohne Schleife
- Zuweisung an einen neuen Spaltennamen legt die Spalte an, an einen vorhandenen überschreibt sie

Altersgruppen mit `pd.cut`

```
df["altersgruppe"] = pd.cut(
    df["alter"],
    bins=[0, 18, 65, 120],
    labels=["Kind", "Erwachsen", "Senior"],
    right=False,                # [0, 18), [18, 65), [65, 120)
)

df["altersgruppe"].value_counts()
```

- `bins` sind die Grenzen, `labels` die Namen der Klassen dazwischen (eine weniger als Grenzen)
- `right=False` : Die linke Grenze gehört dazu, die rechte nicht. Ein Alter von 18 fällt in „Erwachsen“, 65 in „Senior“.
- Das Ergebnis hat den Typ `category` mit fester Reihenfolge

apply: eigene Funktion je Wert, sparsam einsetzen

```
def bmi_klasse(bmi):  
    if pd.isna(bmi):                # fehlender Wert bleibt fehlend  
        return None  
    if bmi < 18.5:  
        return "Untergewicht"  
    elif bmi < 25:  
        return "Normalgewicht"  
    elif bmi < 30:  
        return "Übergewicht"  
    else:  
        return "Adipositas"  
  
df["bmi_klasse"] = df["bmi"].apply(bmi_klasse)
```

- `apply` ruft die Funktion für **jeden Wert einzeln** auf: bequem, aber eine versteckte Python-Schleife
- Dasselbe vektorisiert: `pd.cut(df["bmi"], bins=[0, 18.5, 25, 30, 100], labels=[...], right=False)`

Reihenfolge der Wahl: Spaltenarithmetik, dann pandas-Funktionen (`pd.cut`, `.str`, `.dt`, `map`), erst zuletzt `apply`.

Sortieren und Zählen

```
# die fünf höchsten Leistungsausgaben
df.sort_values("leistungsausgaben_eur", ascending=False).head()

# erst nach Bundesland, darin nach Alter absteigend
df.sort_values(["bundesland", "alter"], ascending=[True, False])

df.nlargest(5, "arztbesuche_jahr")           # Kurzform für die Top 5

# Häufigkeiten
df["bundesland"].value_counts()              # absolut
df["blutgruppe"].value_counts(normalize=True).round(3)  # Anteile
df["beruf"].nunique()                        # Anzahl verschiedener Werte
```

- `sort_values` gibt eine **neue**, sortierte Tabelle zurück. `df` selbst bleibt unverändert.
- `value_counts` ist die schnellste Antwort auf „Welche Werte kommen wie oft vor?“

Gruppieren mit `groupby` und `agg`

```
# mittlere Leistungsausgaben je Bundesland
df.groupby("bundesland")["leistungsausgaben_eur"].mean()

# mehrere Kennzahlen auf einmal
df.groupby("bundesland")["leistungsausgaben_eur"].agg(
    ["count", "mean", "median", "max"]
).round(0)

# je Spalte eine eigene Kennzahl
df.groupby(["geschlecht", "raucher"]).agg({
    "groesse_cm": "mean",
    "gewicht_kg": "mean",
    "bmi": "max",
})
```

Das Muster hat drei Schritte: **aufteilen** (`groupby`), **Spalte wählen**, **zusammenfassen** (`mean` , `sum` , `agg`).

Zwei Gruppierungsschlüssel

```
mittel = (  
    df.groupby(["bundesland", "altersgruppe"], observed=True)  
        ["leistungsausgaben_eur"]  
        .mean()  
        .round(0)  
)  
mittel.loc["Berlin"]           # alle Altersgruppen in Berlin  
mittel.loc[("Berlin", "Senior")] # ein einzelner Wert  
  
# als Kreuztabelle: Bundesländer in Zeilen, Altersgruppen in Spalten  
tabelle = mittel.unstack()  
tabelle.sort_values("Senior", ascending=False).head()  
  
# Gruppen filtern: Bundesländer mit Mittel über 3000 Euro  
je_land = df.groupby("bundesland")["leistungsausgaben_eur"].mean()  
je_land[je_land > 3000]
```

Zwei Tabellen verbinden mit `merge`

```
# zweite Tabelle: 20 % der Versicherten nehmen an einem Programm teil
teilnahme = (
    df[ ["versicherten_nr"] ].drop_duplicates()
    .sample(frac=0.2, random_state=1)
    .assign(programm=True)
)

gesamt = pd.merge(df, teilnahme, on="versicherten_nr", how="left")
gesamt["programm"].isna().sum()      # -> 4021 ohne Teilnahme
```

how=	Ergebnis
"inner"	nur Schlüssel, die in beiden Tabellen vorkommen
"left"	alle Zeilen der linken Tabelle, rechts fehlende Werte werden <code>NaN</code>
"outer"	alle Zeilen aus beiden Tabellen

`indicator=True` ergänzt die Spalte `_merge` mit `left_only`, `right_only` oder `both`.

Datumsspalten

```
df["geburtsdatum"] = pd.to_datetime(df["geburtsdatum"])
df["geburtsdatum"].dtype           # -> datetime64[...]: Datum statt Text

df["geburtsjahr"] = df["geburtsdatum"].dt.year
df["geburtsmonat"] = df["geburtsdatum"].dt.month
df["wochentag"] = df["geburtsdatum"].dt.day_name()

# Alter in Jahren aus dem Geburtsjahr
heute = pd.Timestamp("today")
df["alter_neu"] = heute.year - df["geburtsjahr"]

# Tage seit einem Datum
(heute - df["geburtsdatum"]).dt.days
```

- Nach `read_csv` ist ein Datum zunächst Text. Erst `pd.to_datetime` macht daraus ein Datum.
- Über `.dt` erreichen Sie Jahr, Monat, Tag, Wochentag

Ergebnisse speichern

```
auswertung = (  
    df.groupby(["bundesland", "altersgruppe"], observed=True)  
        ["leistungsausgaben_eur"]  
        .agg(["count", "mean"])  
        .reset_index()  
)  
  
auswertung.to_csv("auswertung.csv", index=False)  
  
# für Excel mit deutscher Ländereinstellung  
auswertung.to_csv("auswertung_excel.csv", index=False,  
                  sep=";", decimal=".", encoding="utf-8-sig")
```

- `index=False` lässt die Zeilennummern weg
- `reset_index()` macht aus den Gruppierungsschlüsseln wieder normale Spalten
- `to_excel("auswertung.xlsx")` schreibt direkt eine Excel-Datei, braucht aber das Zusatzpaket `openpyxl`

Zusammenfassung

- NumPy rechnet elementweise mit ganzen Arrays. Schleifen über Daten brauchen Sie fast nie.
- Boolean-Masken mit `&` , `|` , `~` filtern Arrays und DataFrames auf dieselbe Weise
- `axis=0` fasst je Spalte zusammen, `axis=1` je Zeile
- pandas-Grundrezept: `read_csv` , `head` / `info` / `describe` , filtern, Spalten ableiten, `groupby` + `agg`
- `merge` verbindet Tabellen über einen Schlüssel, `pd.to_datetime` und `.dt` erschließen Datumsangaben

Teil 3: Daten einlesen, aufbereiten, explorieren und visualisieren

Von der Rohdatei zum geprüften, bereinigten Datensatz und zu Diagrammen, die Muster sichtbar machen.

Was Sie in diesem Teil lernen

- Sie lesen CSV-Dateien mit den passenden Parametern ein, erkennen typische Einlesefehler und holen Daten per SQL aus einer Datenbank
- Sie prüfen einen neuen Datensatz systematisch: Größe, Typen, Kennzahlen, fehlende Werte
- Sie behandeln fehlende Werte, Duplikate, falsche Datentypen und Ausreißer nachvollziehbar
- Sie bauen Diagramme mit matplotlib und seaborn: Histogramm, Countplot, Boxplot, Scatterplot, Heatmap
- Sie lesen Korrelationen richtig und verwechseln sie nicht mit Ursachen

Von der Rohdatei zur Analyse



- **Einlesen:** Datei, Trennzeichen, Kodierung, Datentypen
- **Überblick:** Größe, Spalten, Kennzahlen, fehlende Werte
- **Bereinigen:** fehlende Werte, Duplikate, Typen, Ausreißer
- **Explorieren und Visualisieren:** Verteilungen, Gruppen, Zusammenhänge

Viele Probleme zeigen sich schon vor dem ersten Modell: Schiefe, Ausreißer, fehlende Werte. Wer sie hier findet, spart später viele Durchläufe in der Modellphase.

Typische Probleme in Rohdaten

Problem	Beispiel
Fehlende Werte	leere Zelle, NULL , N/A , "undefined" , -
Uneinheitliche Datumsformate	01/02/2023 , 2023-02-01 , 02-01-23
Groß- und Kleinschreibung	"Berlin" und "berlin"
Tippfehler in Kategorien	"Frau" , "fraU" , "Fr"
Verschiedene Einheiten	3.5 km , 3500 m
Falsche Zeichenkodierung	ÄÖÜ erscheint als Ã„,Ã-Ãœ
Doppelte Einträge mit Abweichung	"Müller GmbH" und "Müller GmbH."
Werte außerhalb des plausiblen Bereichs	Alter 200, negative Beträge

Die fünf Fragen an jede Datei: Ist sie **vollständig**, **gültig**, **konsistent**, **aktuell** und **eindeutig**?

read_csv: die wichtigsten Parameter

```
import pandas as pd

titanic = pd.read_csv("data/titanic.csv")           # Komma, UTF-8

wein = pd.read_csv("data/winequality-red.csv", sep=";") # Semikolon

versicherte = pd.read_csv(
    "data/versicherte.csv",
    parse_dates=["geburtsdatum"],          # Text direkt als Datum einlesen
    dtype={"plz": str},                    # führende Nullen behalten
)
```

Parameter	Wofür	Typischer Wert
<code>sep</code>	Trennzeichen zwischen den Spalten	<code>",", ";", "\t"</code>
<code>encoding</code>	Zeichenkodierung	<code>"utf-8", "latin-1", "cp1252"</code>
<code>decimal</code>	Dezimalzeichen	<code>",, "</code> bei deutschen Exporten
<code>parse_dates</code>	Spalten, die ein Datum enthalten	<code>["geburtsdatum"]</code>
<code>dtype</code>	Typ je Spalte erzwingen	<code>{"plz": str}</code>

Typische Fehler beim Einlesen

Symptom	Ursache	Abhilfe
<code>FileNotFoundError</code>	falscher Pfad oder falsches Arbeitsverzeichnis	Pfad prüfen, <code>import os; os.getcwd()</code>
Alles steht in einer Spalte	falsches Trennzeichen	<code>sep=";"</code>
<code>UnicodeDecodeError</code> oder <code>Ã</code> statt <code>ä</code>	falsche Kodierung	<code>encoding="latin-1"</code> oder <code>"cp1252"</code>
Zahlenspalte hat Typ Text	Dezimalkomma, Tausenderpunkt, Text wie <code>"k. A."</code>	<code>decimal=","</code> , <code>thousands="."</code> , <code>na_values=[...]</code>
Datum ist Text	pandas rät Datumsspalten nicht	<code>parse_dates=[...]</code> oder <code>pd.to_datetime</code>
PLZ <code>01067</code> wird zu <code>1067</code>	Spalte als Zahl gelesen	<code>dtype={"plz": str}</code>

```
wein_falsch = pd.read_csv("data/winequality-red.csv")
wein_falsch.shape      # -> (1599, 1)    alles in einer Spalte
wein.shape              # -> (1599, 12)   mit sep=";"
```

Daten aus einer Datenbank: `read_sql`

```
import sqlite3

con = sqlite3.connect("data/versicherte.db")      # Verbindung öffnen
sql = """
    SELECT bundesland, COUNT(*) AS anzahl,
           AVG(leistungsausgaben_eur) AS ausgaben_mittel
    FROM versicherte
    WHERE arztbesuche_jahr >= 10
    GROUP BY bundesland
    ORDER BY anzahl DESC
"""
df = pd.read_sql(sql, con)                       # Ergebnis ist ein DataFrame
con.close()
print(df.head(2).round(2))
```

	bundesland	anzahl	ausgaben_mittel
0	Berlin	221	5526.41
1	Nordrhein-Westfalen	151	6054.46

- Die Datenbank filtert und gruppiert. In pandas kommt nur das Ergebnis an, hier 16 Zeilen statt 5 025
- Bei einer anderen Datenbank ändert sich nur die Verbindung, `read_sql` und alles danach bleiben gleich

Der Titanic-Datensatz

Spalte	Bedeutung	Typ	Auffälligkeit
PassengerId	laufende Nummer	ganzzahlig	reine Kennung
Survived	überlebt (0 = nein, 1 = ja)	binär, Zielgröße	ungleich verteilt
Pclass	Passagierklasse (1, 2, 3)	ordinal	hängt mit dem sozialen Status zusammen
Name	vollständiger Name	Text	Anrede steckt im Text
Sex	Geschlecht	Kategorie	
Age	Alter in Jahren	Zahl	fehlende Werte
SibSp	Geschwister und Ehepartner an Bord	Anzahl	sehr viele Nullen
Parch	Eltern und Kinder an Bord	Anzahl	sehr viele Nullen
Ticket	Ticketnummer	Text	uneinheitliches Format
Fare	Ticketpreis	Zahl	schief verteilt, Ausreißer
Cabin	Kabinennummer	Text	über 75 % fehlen
Embarked	Zustiegshafen (C, Q, S)	Kategorie	wenige fehlende Werte

Erster Überblick: Größe und Typen

```
df = pd.read_csv("data/titanic.csv")

df.shape          # -> (891, 12)   (Zeilen, Spalten)
list(df.columns)  # Spaltennamen als Liste
df.head()
df.info()
```

```
RangeIndex: 891 entries, 0 to 890
Data columns (total 12 columns):
#   Column          Non-Null Count  Dtype
0   PassengerId      891 non-null   int64
5   Age              714 non-null   float64
10  Cabin            204 non-null   str
11  Embarked         889 non-null   str
```

Non-Null Count kleiner als 891 heißt: In dieser Spalte fehlen Werte.

Kennzahlen mit describe

```
df[ ["Age", "Fare"] ].describe().round(2)
```

	Age	Fare
count	714.00	891.00
mean	29.70	32.20
std	14.53	49.69
min	0.42	0.00
25%	20.12	7.91
50%	28.00	14.45
75%	38.00	31.00
max	80.00	512.33

- **Mittelwert** (`mean`) reagiert empfindlich auf Ausreißer, der **Median** (`50%`) ist robust
- Bei `Age` liegen beide nah beieinander
- Bei `Fare` ist der Mittelwert mehr als doppelt so hoch wie der Median: Die Verteilung ist **schief**, wenige sehr teure Tickets ziehen den Mittelwert nach oben
- **Streuung**: `std` und der Abstand zwischen `25%` und `75%` (Interquartilsabstand)

Fehlende Werte finden

```
df.isna().sum().sort_values(ascending=False).head(4)
```

Cabin	687
Age	177
Embarked	2
PassengerId	0

```
(df.isna().mean() * 100).round(2).sort_values(ascending=False).head(3)
```

Cabin	77.10
Age	19.87
Embarked	0.22

`isna()` liefert eine Tabelle aus `True` / `False` . `sum()` zählt die `True` -Werte je Spalte, `mean()` ergibt den Anteil.

Fehlende Werte: drei Strategien

Strategie	Befehl	Geeignet, wenn
Löschen	<code>df.dropna(subset=[...])</code>	nur wenige Zeilen betroffen sind (<code>Embarked</code> : 2 von 891)
Auffüllen (Imputation)	<code>df["Age"].fillna(median)</code>	die Spalte wichtig ist und ein plausibler Ersatzwert existiert
Markieren (Flag)	<code>df["Age"].isna()</code> als neue Spalte	das Fehlen selbst eine Information sein kann

- Eine Spalte wie `Cabin` mit 77 % Lücken lässt sich nicht sinnvoll auffüllen: weglassen oder nur „Kabine bekannt ja/nein“ behalten
- Auffüllen und Markieren lassen sich kombinieren

Löschen Sie nie blind. `df.dropna()` ohne Parameter lässt bei Titanic von 891 Zeilen nur 183 übrig, weil `Cabin` fast überall fehlt.

Fehlende Werte behandeln

```
spalten = ["Survived", "Pclass", "Sex", "Age",  
           "SibSp", "Parch", "Fare", "Embarked"]  
feat_df = df[spalten].copy()           # ohne Cabin, Name, Ticket  
  
# 1. Löschen: die 2 Zeilen ohne Zustiegshafen  
feat_df = feat_df.dropna(subset=["Embarked"])  
  
# 2. Markieren: fehlte das Alter?  
feat_df["Age_missing"] = feat_df["Age"].isna().astype(int)  
  
# 3. Auffüllen: Median des Alters  
median_alter = feat_df["Age"].median() # -> 28.0  
feat_df["Age"] = feat_df["Age"].fillna(median_alter)  
  
feat_df.isna().sum().sum()             # -> 0
```

Ergebnis immer wieder **zuweisen** (`feat_df = feat_df.dropna(...)`). `dropna` und `fillna` geben eine neue Tabelle zurück, das Original bleibt unverändert.

Duplikate

```
demo = pd.DataFrame({
    "Name": ["Anna", "Ben", "Carlos", "Anna", "Anna"],
    "Alter": [25, 30, 22, 25, 25],
    "Einkommen": [50000, 48000, 52000, 50000, 52000],
})

demo.duplicated()                # True für jede wiederholte Zeile
demo.duplicated().sum()          # -> 1   (Zeile 3 gleicht Zeile 0)

demo.drop_duplicates()           # exakt gleiche Zeilen raus
demo.drop_duplicates(subset=["Name"]) # je Name nur die erste Zeile

df.duplicated().sum()            # Titanic: -> 0
df["PassengerId"].is_unique      # -> True
```

- Exakte Duplikate können Sie entfernen. Bei `subset` entscheiden Sie, welche Spalten eine Zeile eindeutig machen
- Duplikate verzerren Kennzahlen und Modelle

Datentypen korrigieren

```
feat_df.dtypes

feat_df["Sex"] = feat_df["Sex"].astype("category")
feat_df["Embarked"] = feat_df["Embarked"].astype("category")
feat_df["Age_missing"] = feat_df["Age_missing"].astype(bool)

feat_df["Sex"].cat.categories      # -> ['female', 'male']

# Zahl, die als Text ankam: nicht lesbare Einträge werden NaN
pd.to_numeric(pd.Series(["12.5", "7", "k. A."]), errors="coerce")
# -> 12.5, 7.0, NaN
```

- `category` für Spalten mit wenigen, wiederkehrenden Werten: spart Speicher und dokumentiert die erlaubten Werte
- Kennungen (`PassengerId` , Versichertennummer, PLZ) sind **keine** Rechengrößen, auch wenn sie aus Ziffern bestehen
- Datum: `pd.to_datetime` , bekannt aus dem vorigen Teil

Kategorien zählen

```
df["Survived"].value_counts()
```

```
# -> 0: 549, 1: 342
```

```
df["Survived"].value_counts(normalize=True).round(3)
```

```
# -> 0: 0.616, 1: 0.384
```

```
df["Embarked"].value_counts(dropna=False)
```

```
# -> S: 644, C: 168, Q: 77, NaN: 2
```

```
# Überlebensrate je Gruppe: Mittelwert einer 0/1-Spalte
```

```
df.groupby("Sex")["Survived"].mean().round(3)
```

```
# -> female: 0.742, male: 0.189
```

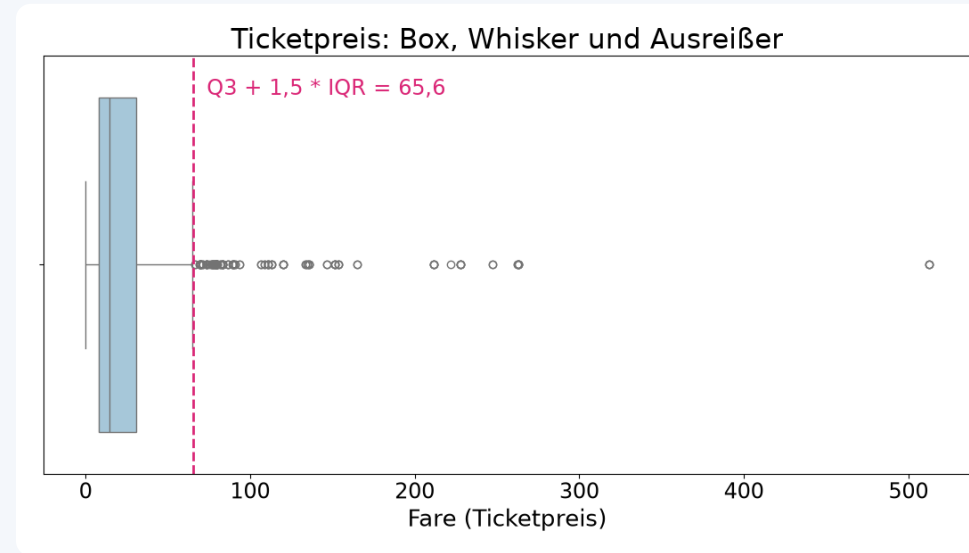
```
df.groupby("Pclass")["Survived"].mean().round(2)
```

```
# -> 1: 0.63, 2: 0.47, 3: 0.24
```

Ausreißer erkennen: der Boxplot

- **Box:** vom 25%-Quantil (Q1) bis zum 75%-Quantil (Q3), darin liegt die mittlere Hälfte der Werte
- **Strich in der Box:** Median
- **Whisker:** bis zum letzten Wert innerhalb von 1,5 Interquartilsabständen ab der Box
- **Einzelne Punkte:** Werte jenseits der Whisker, Kandidaten für Ausreißer

```
import seaborn as sns
sns.boxplot(x=df["Fare"])
```



Ausreißer mit der IQR-Regel

```
q1 = df["Fare"].quantile(0.25)      # -> 7.91
q3 = df["Fare"].quantile(0.75)      # -> 31.0
iqr = q3 - q1                       # -> 23.09   Interquartilsabstand

untergrenze = q1 - 1.5 * iqr        # -> -26.72
obergrenze = q3 + 1.5 * iqr        # -> 65.63

maske = (df["Fare"] < untergrenze) | (df["Fare"] > obergrenze)
maske.sum()                        # -> 116 Tickets
df[maske].sort_values("Fare", ascending=False).head()
```

- Die Regel markiert 116 von 891 Tickets, also 13 %
- 104 davon sind teure Tickets der ersten Klasse, keine Messfehler

Die IQR-Regel liefert **Kandidaten**, keine Urteile. Erst ansehen, dann die Ursache verstehen, dann entscheiden.

Ausreißer behandeln

```
# 1. Entfernen: nur bei nachweislich falschen Werten
ohne = df[~maske]

# 2. Begrenzen (Clipping) auf das 5%- und 95%-Quantil
low, high = df["Fare"].quantile([0.05, 0.95])      # -> 7.23, 112.08
df["Fare_clip"] = df["Fare"].clip(lower=low, upper=high)

# 3. Plausibilitätsregel aus dem Fachwissen
df = df[df["Age"].between(0, 110) | df["Age"].isna()]

# 4. Markieren und behalten
df["Fare_hoch"] = df["Fare"] > obergrenze
```

- `clip` setzt Werte außerhalb der Grenzen **auf** die Grenze. Die Zeile bleibt erhalten
- Schreiben Sie das Ergebnis in eine **neue Spalte**, dann bleibt das Original zum Vergleich

Korrelation

```
df.corr(numeric_only=True)["Survived"].round(2)
```

PassengerId	-0.01
Survived	1.00
Pclass	-0.34
Age	-0.08
SibSp	-0.04
Parch	0.08
Fare	0.26

- Der Korrelationskoeffizient liegt zwischen -1 und +1. Nahe 0: kein **linearer** Zusammenhang
- `Pclass` -0,34: höhere Klassennummer (dritte Klasse), geringere Überlebensrate
- `Fare` +0,26: teurere Tickets, höhere Überlebensrate
- `numeric_only=True` lässt Textspalten aus. `Sex` fehlt deshalb hier, obwohl es das stärkste Merkmal ist

Korrelation ist keine Kausalität

- Korrelation misst einen Zusammenhang, keine Ursache
- Zwei Größen können gemeinsam schwanken, weil beide von einer **dritten** Größe abhängen
- Titanic: `Fare` und `Survived` korrelieren mit +0,26. Der Ticketpreis hat aber niemanden gerettet. Dahinter steht die Passagierklasse: `Fare` und `Pclass` korrelieren mit -0,55
- Gedankenbeispiel: Gehen im Winter mehr Handschuhe verloren und steigt zugleich der Dieselpreis, laufen beide Kurven parallel. Die gemeinsame Ursache ist die Jahreszeit

Korrelationen sind gute Ausgangspunkte für Hypothesen, aber kein Beweis. Für „A verursacht B“ brauchen Sie Fachwissen oder ein Experiment.

Warum visualisieren?

- Ein Diagramm macht sichtbar, was eine Tabelle mit 891 Zeilen versteckt: Trends, Ausreißer, Gruppenunterschiede
- Kennzahlen allein können täuschen: Derselbe Mittelwert passt zu völlig verschiedenen Verteilungen
- Ein gutes Diagramm verhindert falsche Schlüsse, ein schlechtes erzeugt sie

Frage	Diagramm
Wie ist eine Zahlenspalte verteilt, ist sie schief?	Histogramm
Gibt es Ausreißer, wie unterscheiden sich Gruppen?	Boxplot
Wie oft kommt jede Kategorie vor?	Countplot (Balken)
Wie hängen zwei Zahlenspalten zusammen?	Scatterplot
Wie hängen viele Spalten zusammen?	Heatmap der Korrelationsmatrix
Wie entwickelt sich ein Wert über die Zeit?	Linienplot

matplotlib: das Grundgerüst

```
import matplotlib.pyplot as plt

fig, ax = plt.subplots(figsize=(8, 5))  # Figure = Blatt, Axes = Diagramm

ax.hist(df["Age"].dropna(), bins=30)

ax.set_title("Altersverteilung der Passagiere")
ax.set_xlabel("Alter in Jahren")
ax.set_ylabel("Anzahl")

fig.tight_layout()
fig.savefig("alter_histogramm.png", dpi=150)
plt.show()
```

- `fig` ist das ganze Bild, `ax` ein einzelnes Koordinatensystem darin
- Alles, was Sie beschriften oder zeichnen, läuft über `ax`
- `savefig` vor `plt.show()` aufrufen

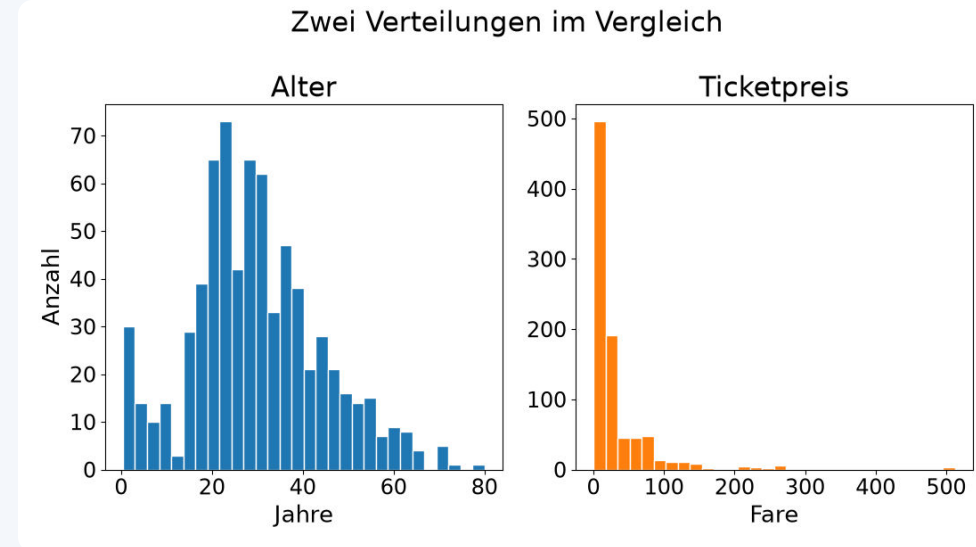
Mehrere Achsen in einer Figure

```
fig, axes = plt.subplots(
    1, 2, figsize=(11, 6.2))

axes[0].hist(df["Age"].dropna(),
             bins=30)
axes[0].set_title("Alter")
axes[0].set_xlabel("Jahre")
axes[0].set_ylabel("Anzahl")

axes[1].hist(df["Fare"], bins=30,
             color="#ff7f0e")
axes[1].set_title("Ticketpreis")
axes[1].set_xlabel("Fare")

fig.suptitle("Zwei Verteilungen")
fig.tight_layout()
```



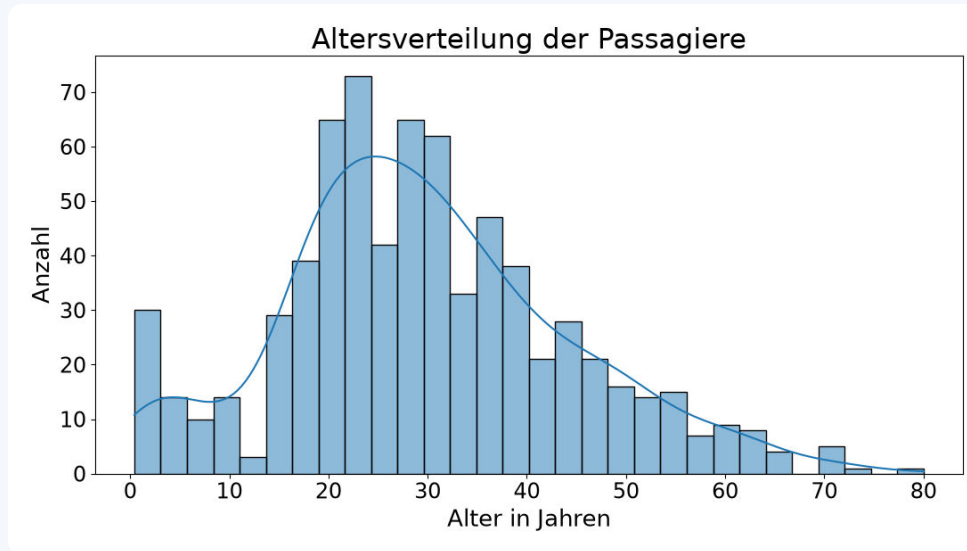
`plt.subplots(zeilen, spalten)` liefert ein Array von Achsen. `axes[0]` ist das linke, `axes[1]` das rechte Diagramm.

seaborn: Histogramm mit `histplot`

```
import seaborn as sns

fig, ax = plt.subplots()
sns.histplot(data=df, x="Age",
             bins=30, kde=True,
             ax=ax)
ax.set_title("Altersverteilung")
ax.set_xlabel("Alter in Jahren")
ax.set_ylabel("Anzahl")
```

- `data=` nimmt den DataFrame, `x=` den Spaltennamen
- `kde=True` legt eine geglättete Dichtekurve über die Balken
- `ax=ax` zeichnet in unser matplotlib-Gerüst

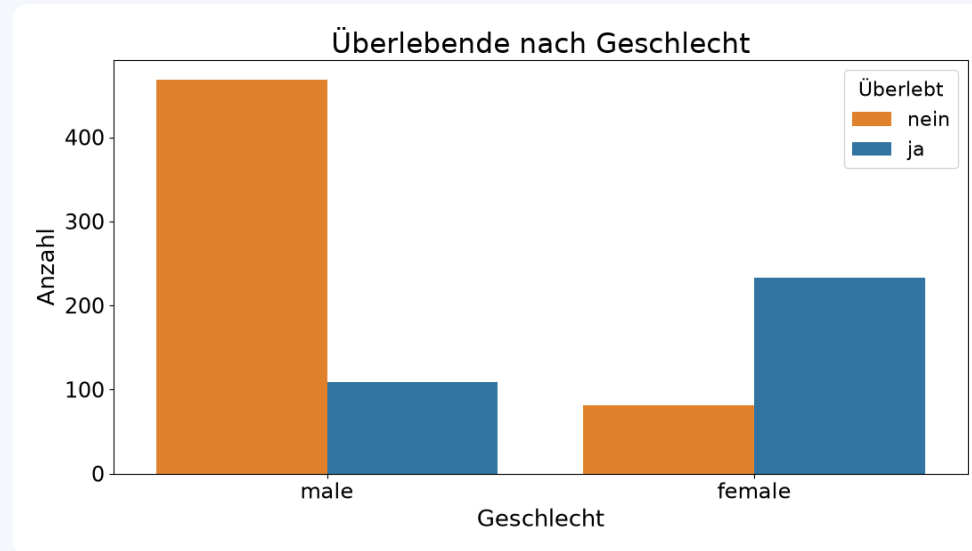


Kategorien zählen mit `countplot` und `hue`

```
df["Überlebt"] = df["Survived"].map(
    {0: "nein", 1: "ja"})

fig, ax = plt.subplots()
sns.countplot(data=df, x="Sex",
              hue="Überlebt", ax=ax)
ax.set_title(
    "Überlebende nach Geschlecht")
ax.set_xlabel("Geschlecht")
ax.set_ylabel("Anzahl")
```

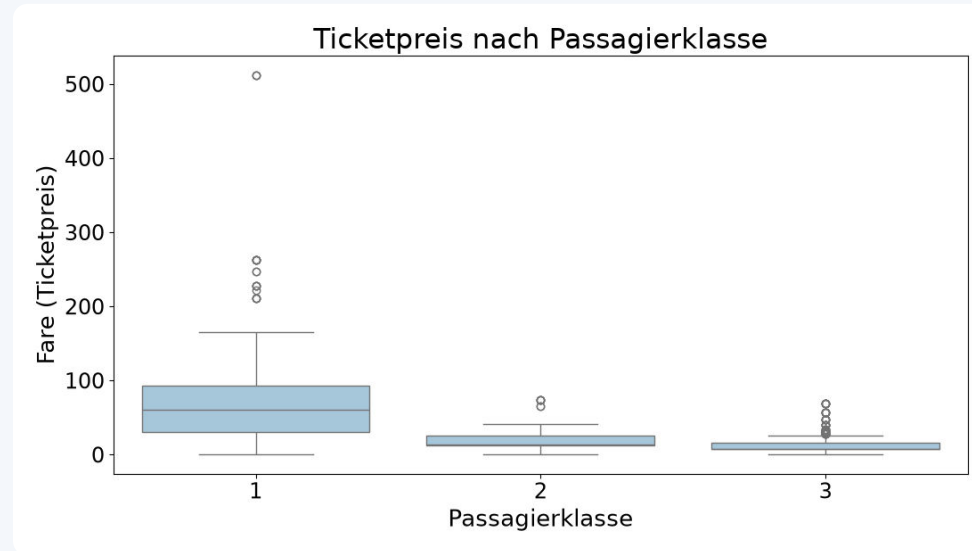
- `countplot` zählt selbst, ein `value_counts` vorab ist nicht nötig
- `hue=` teilt jeden Balken nach einer zweiten Spalte auf und erzeugt die Legende



Gruppen vergleichen mit `boxplot`

```
fig, ax = plt.subplots()
sns.boxplot(data=df, x="Pclass",
            y="Fare", ax=ax)
ax.set_title(
    "Ticketpreis nach Klasse")
ax.set_xlabel("Passagierklasse")
ax.set_ylabel("Fare")
```

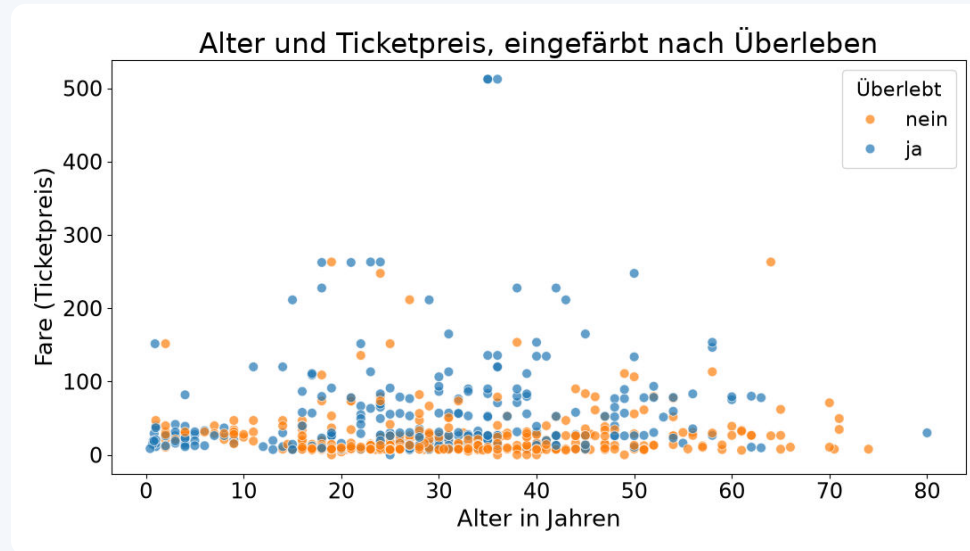
- `x=` Kategorie, `y=` Zahlenspalte: ein Boxplot je Gruppe
- Median, Streuung und Ausreißer aller Gruppen stehen direkt nebeneinander
- 104 der 116 IQR-Ausreißer gehören zur ersten Klasse



Zwei Zahlenspalten: scatterplot mit hue

```
fig, ax = plt.subplots()
sns.scatterplot(data=df, x="Age",
               y="Fare",
               hue="Überlebt",
               alpha=0.7, ax=ax)
ax.set_title("Alter und Ticketpreis")
ax.set_xlabel("Alter in Jahren")
ax.set_ylabel("Fare")
```

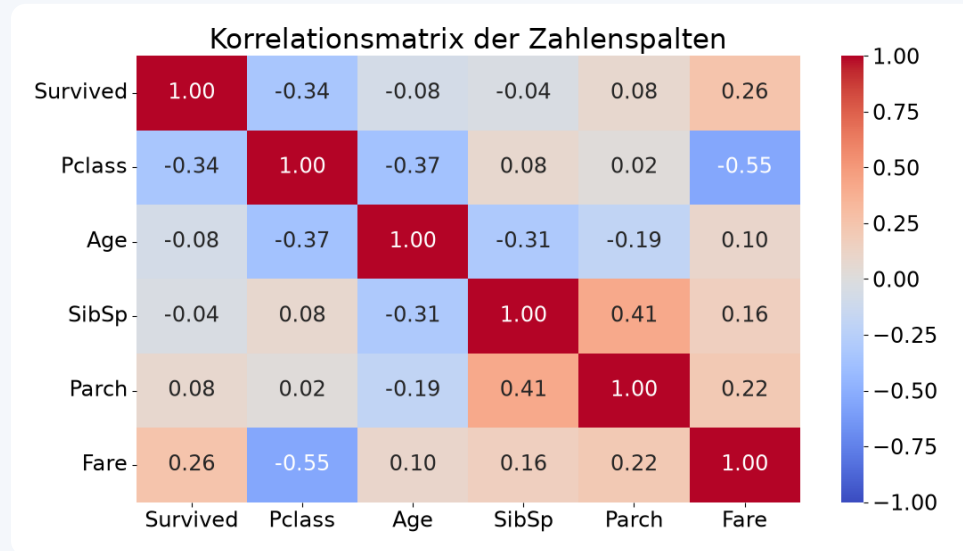
- Ein Punkt je Passagier
- `hue=` färbt nach einer dritten Spalte
- `alpha=0.7` macht Punkte durchscheinend, damit Häufungen sichtbar bleiben



Heatmap der Korrelationsmatrix

```
spalten = ["Survived", "Pclass", "Age",  
           "SibSp", "Parch", "Fare"]  
corr = df[spalten].corr()  
  
fig, ax = plt.subplots()  
sns.heatmap(corr, annot=True,  
            fmt=".2f",  
            cmap="coolwarm",  
            vmin=-1, vmax=1, ax=ax)  
ax.set_title("Korrelationsmatrix")
```

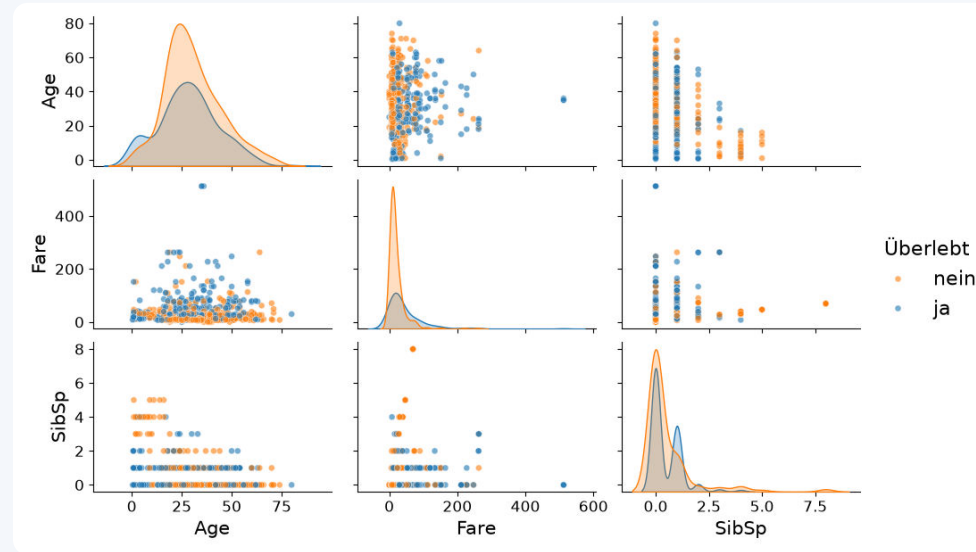
- `annot=True` schreibt die Werte in die Felder,
`fmt=".2f"` rundet auf zwei Stellen
- `vmin=-1, vmax=1` verankert die Farbskala:
Weiß bedeutet 0



Überblick mit pairplot

```
sns.pairplot(  
    df,  
    vars=["Age", "Fare", "SibSp"],  
    hue="Überlebt",  
)
```

- Jede Zahlenspalte gegen jede: Scatterplots außerhalb, Verteilungen auf der Diagonale
- Guter erster Blick auf einen neuen Datensatz
- Bei mehr als fünf, sechs Spalten wird das Raster unlesbar und langsam: Spalten mit `vars=` auswählen



Regeln für gute Diagramme

- **Eine Aussage je Diagramm.** Formulieren Sie die Aussage vorher als Satz und schreiben Sie sie in den Titel
- **Diagrammtyp nach Frage wählen:** Verteilung, Vergleich, Zusammenhang oder Verlauf
- **Achsen beschriften,** mit Einheit. Spaltennamen wie `Fare` sind keine Beschriftung für Dritte
- **Farbe nur mit Bedeutung:** Farbe kodiert eine Gruppe oder einen Wert, keine Dekoration
- **Achsen ehrlich wählen:** Balkendiagramme beginnen bei 0
- **Lesbarkeit prüfen:** Schriftgröße, gedrehte Achsenbeschriftungen bei langen Kategorien
(`ax.tick_params(axis="x", rotation=45)`)

Test vor dem Weitergeben: Versteht eine Kollegin das Diagramm auf den ersten Blick, ohne Ihre Erklärung?

Zusammenfassung

- Nach jedem Einlesen prüfen: `shape` , `head()` , `info()` . Falsches Trennzeichen, falsche Kodierung und Dezimalkomma fallen dort sofort auf
- Fehlende Werte erst zählen (`isna().sum()` , Anteil mit `isna().mean()`), dann je Spalte entscheiden: löschen, mit dem Median füllen, markieren
- Ausreißer liefert die IQR-Regel als Kandidaten. Entfernen nur bei nachweislichen Fehlern, sonst begrenzen (`clip`) oder behalten
- Diagramme bauen Sie auf dem Gerüst `fig, ax = plt.subplots()` , seaborn zeichnet mit `data=` , `x=` , `y=` , `hue=` , `ax=` hinein
- Korrelation zeigt Zusammenhänge, keine Ursachen

Teil 4: Grundlagen von Machine Learning und KI

Was KI und Machine Learning sind, wie ein Modell aus Daten lernt und wie Sie Ihr erstes Modell trainieren.

Was Sie in diesem Teil lernen

- KI, Machine Learning und Deep Learning voneinander abgrenzen
- Die drei Lernarten mit je einem Beispiel benennen
- Entscheiden, wann sich ML gegenüber festen Regeln lohnt
- Die Begriffe Merkmal, Zielgröße, Trainings- und Testdaten sicher verwenden
- Ein erstes Modell mit scikit-learn trainieren: `fit` , `predict` , `score`

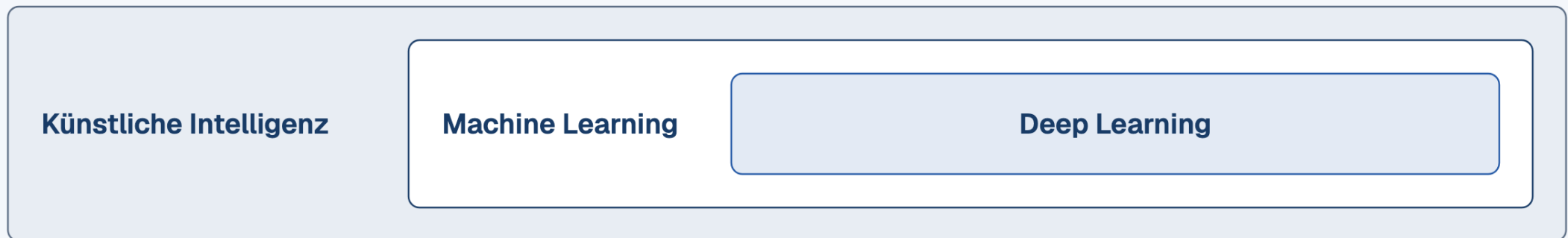
Was ist Künstliche Intelligenz?

Künstliche Intelligenz (KI) bezeichnet Systeme oder Maschinen, die Aufgaben ausführen, für die normalerweise menschliche Intelligenz erforderlich ist.

- **Lernen:** aus Beispielen besser werden
- **Problemlösen:** einen Weg zu einem Ziel finden
- **Wahrnehmung:** Bilder und Signale auswerten
- **Sprachverstehen:** Texte lesen und erzeugen

KI, Machine Learning und Deep Learning

Begriff	Bedeutung
Künstliche Intelligenz (KI)	Oberbegriff für Maschinen, die „intelligent“ handeln
Machine Learning (ML)	Teilgebiet der KI: Systeme lernen aus Daten, ohne dass jede Regel von Hand programmiert wird
Deep Learning	Teilgebiet des ML, das mit künstlichen neuronalen Netzen arbeitet



Kurze Geschichte der KI

Zeit	Was passiert
1956	Dartmouth Conference (John McCarthy u. a.) prägt den Begriff „Artificial Intelligence“
1950er bis 1970er	Erste KI-Programme: Schachprogramme, Theorembeweiser. Viel Optimismus, dann Rückschläge („KI-Winter“)
1980er	Expertensysteme (Vorbild: MYCIN, 1970er, medizinische Diagnosen). Grenze: Wissen muss von Hand eingepflegt werden
2000er	Durchbrüche im Machine Learning: mehr Daten, bessere Algorithmen, leistungsfähige Hardware
2010er bis heute	KI im Alltag: Sprachassistenten, Bildanalyse, Produktempfehlungen. Deep Learning spielt die zentrale Rolle
heute	Große Sprachmodelle (LLMs, z. B. GPT, BERT) verarbeiten und erzeugen natürliche Sprache

Wie lernt ein Modell aus Daten?

- Ein Algorithmus sucht **Muster** in vielen Beispielen
- Das Modell lernt Zusammenhänge aus Beispieldaten, statt nur fest programmierte Regeln auszuführen
- Ziel ist **Generalisierung**: Das Modell soll auf neuen Daten funktionieren, bekannte Fälle nachzurechnen genügt nicht
- In der Praxis: aus Vergangenheitsdaten Vorhersagen für neue Fälle ableiten

Ein Modell ist nur dann gut, wenn es auf Daten funktioniert, die es beim Lernen nicht gesehen hat.

Lernarten im Überblick

Lernart	Daten	Beispiel
Überwachtes Lernen	Beispiele mit bekannter Lösung (gelabelt)	Spam-Erkennung
Unüberwachtes Lernen	Beispiele ohne Lösung, das Verfahren sucht Strukturen	Kundensegmentierung
Bestärkendes Lernen	Belohnung und Bestrafung für Aktionen	AlphaGo

- Überwacht mit Zahl als Ziel: **Regression**. Mit Klasse als Ziel: **Klassifikation**
- Unüberwacht: **Clustering** gruppiert ähnliche Datenpunkte

Beispiel Spam-Erkennung: Daten und Merkmale

Ein ML-Modell sortiert unerwünschte E-Mails aus, indem es Muster in den Nachrichten auswertet.

Schritt 1: Daten sammeln und vorbereiten

- Datenquelle: E-Mails mit dem Label „Spam“ oder „Kein Spam“
- Merkmale, die das Modell zu sehen bekommt:
 - bestimmte Wörter („Won“, „Free“, „Quick money“)
 - Anzahl der Links, Anteil an Großbuchstaben
 - Absenderadresse, Länge und Struktur der E-Mail

Schritt 2: Modell trainieren (überwachtes Lernen)

- Das System wird mit vielen gelabelten E-Mails trainiert
- Es gewichtet die Merkmale: Welche Eigenschaften deuten am stärksten auf Spam hin?

Beispiel Spam-Erkennung: Anwendung und Fehler

Schritt 3: neue E-Mails klassifizieren

- Eine neue E-Mail trifft ein, das Modell wertet die Merkmale aus und entscheidet: Spam oder nicht
- Viele spam-typische Wörter: hohe Spam-Wahrscheinlichkeit
- Legitime Muster erkannt: die E-Mail bleibt im Posteingang

Schritt 4: Modell verbessern

- **False Positive:** eine legitime E-Mail wird fälschlich als Spam markiert
- **False Negative:** Spam wird nicht erkannt und landet im Posteingang
- Rückmeldungen der Nutzer („Kein Spam“, „Spam melden“) fließen als neue Trainingsdaten ein

ML gegenüber festen Regeln

Regelbasiert

- „wenn Wert > Schwelle, dann Alarm“
- gut bei einfachen, stabilen Situationen
- transparent, aber oft unflexibel
- Menschen müssen die Regeln laufend nachpflegen

Machine Learning

- lernt Muster aus mehreren Merkmalen gleichzeitig
- erkennt komplexere Wechselwirkungen
- passt sich mit neuen Daten an
- braucht Daten, Evaluation und Monitoring

ML ersetzt Fachlogik nicht. Es ergänzt sie dort, wo Muster für feste Regeln zu komplex werden.

Wo feste Regeln an Grenzen stoßen

Problem	Regelbasiert	ML-Ansatz
Veränderung	„Free" war ein typisches Spam-Wort, heute steht dort „Fr33" oder „F.r.e.e". Jede Variante braucht eine neue Regel	lernt neue Schreibweisen aus neuen Beispielen
Versteckte Muster	prüft nur offensichtliche Kriterien („enthält das Wort casino?")	nutzt auch Satzstruktur, Absenderverhalten, Ähnlichkeit zu bekanntem Spam
Kontext	Hotelbestätigung mit „Free WiFi" landet im Spam (False Positive)	lernt, den Kontext mit auszuwerten
Menge	Experten schreiben und testen laufend neue Regeln	skaliert mit den Daten: Nachtrainieren statt neuer Regeln von Hand

Wann sich ML lohnt und wann nicht

ML lohnt sich, wenn

- ausreichend historische Daten vorliegen
- eine wiederkehrende Vorhersage oder Klassifikation Nutzen stiftet
- einfache Heuristiken die Zusammenhänge nicht mehr sauber beschreiben
- Fehlentscheidungen messbar bewertet werden können

ML lohnt sich eher nicht, wenn

- es kaum Daten, aber viele Sonderfälle gibt
- die Entscheidung fachlich noch nicht klar formuliert ist
- eine bekannte Regel das Problem bereits zuverlässig löst
- weder Nutzen noch Fehlerkosten bewertet werden können

Grenzen von ML

- **Die Vergangenheit sagt die Zukunft nicht immer voraus.** Historische Daten setzen stabile Bedingungen voraus. Sind Menschen Teil des Systems, gilt das oft nicht. Beispiel: Finanzkrisen lassen sich nicht allein aus historischen Daten vorhersagen.
- **Unbekannte Merkmale.** Wer Daten erhebt, legt vorher fest, welche Variablen gesammelt werden. Kritische Größen können fehlen. Beispiel Medizin: unbekannte genetische Faktoren oder Umweltbedingungen beeinflussen den Behandlungserfolg, stehen aber nicht in den Daten.
- **Ein bekanntes Verfahren wird ausgenutzt.** Wird ein Algorithmus zum Standard, können Beteiligte die Eingaben gezielt manipulieren. Beispiel: CDO-Ratings vor der Finanzkrise 2008.

Merkmale und Zielgröße

- **Merkmal (engl. Feature):** eine Eingabespalte, aus der das Modell lernt. Alle Merkmale zusammen bilden X
- **Zielgröße (Target, Label):** die Spalte, die das Modell vorhersagen soll. Sie heißt y
- Jede Zeile ist ein Beispiel: Merkmale plus bekannte Lösung

	Pclass	Sex	Age	Fare	Survived
Rolle	Merkmal	Merkmal	Merkmal	Merkmal	Zielgröße

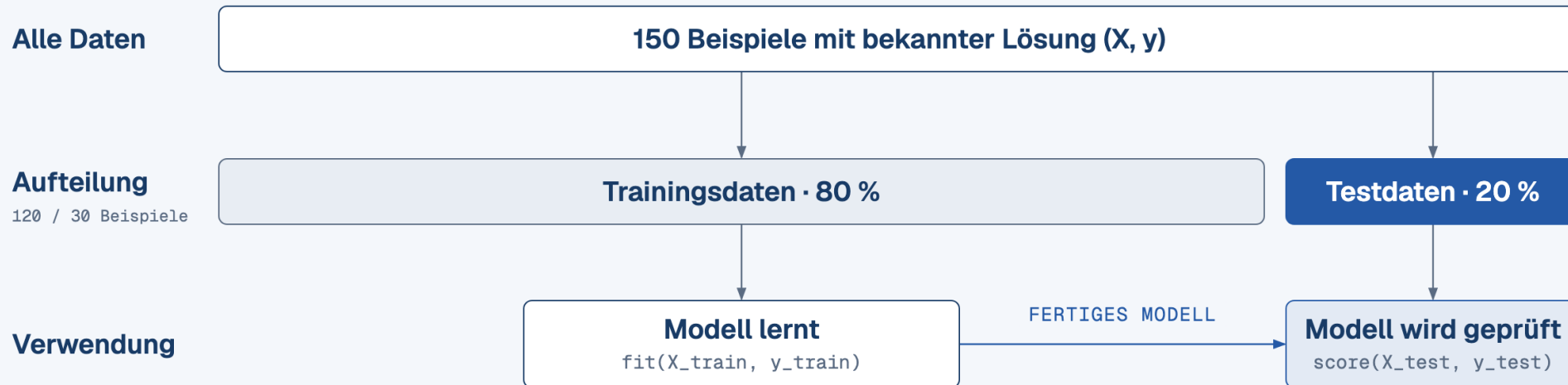
Ohne klar definierte Zielgröße gibt es kein sauberes ML-Problem. Eine unscharfe Zielgröße ergibt meist ein unscharfes Modell.

Was ein gutes Merkmal ausmacht

- fachlich plausibel
- zum Zeitpunkt der Vorhersage **rechtzeitig verfügbar**
- stabil und reproduzierbar berechenbar
- nicht aus der Zielgröße abgeleitet
- verständlich genug, um es mit dem Fachbereich zu besprechen

Trainings- und Testdaten

- **Trainingsdaten:** daraus lernt das Modell seine Parameter
- **Testdaten:** zurückgehaltene Beispiele für die neutrale Endbewertung
- Das Modell sieht die Testdaten beim Lernen nie



Aufteilen mit `train_test_split`

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split

X, y = load_iris(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=1
)
```

80 % der Zeilen zum Lernen, 20 % zum Prüfen

Wer auf den Trainingsdaten bewertet, misst Auswendiglernen. Nur die Testdaten zeigen, ob das Modell generalisiert.

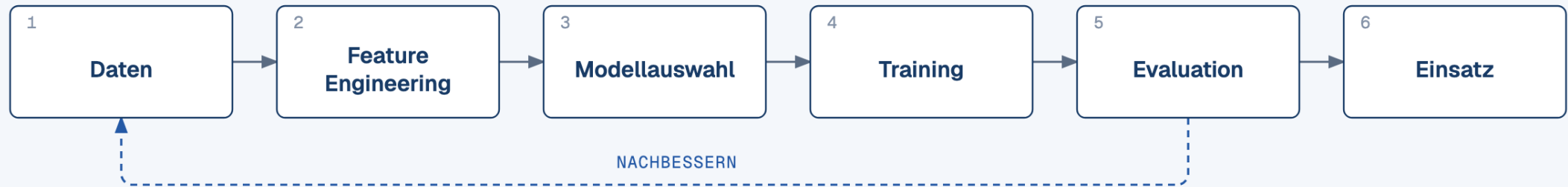
Datenleck (Leakage)

Ein Datenleck liegt vor, wenn das Modell Informationen sieht, die im echten Einsatz zum Zeitpunkt der Vorhersage nicht verfügbar sind.

- Beispiel: Ein Modell soll einen Geräteausfall vorhersagen und bekommt als Merkmal den Reparaturcode, der erst nach dem Ausfall vergeben wird
- Zweites Beispiel: Die Skalierung wird auf allen Daten berechnet, bevor Trainings- und Testdaten getrennt werden
- Folge: unrealistisch gute Ergebnisse im Test, schlechtes Modell im Einsatz

Bewerten Sie jedes Merkmal aus Sicht des Einsatzzeitpunkts: Liegt dieser Wert dann schon vor?

Der ML-Workflow in sechs Schritten



Der Ablauf ist iterativ und führt oft wieder zur Datenphase zurück.

Die sechs Schritte im Einzelnen

Schritt	Was passiert	Werkzeug im Kurs
1 Daten	beschaffen, explorieren, bereinigen, Qualität prüfen	pandas, seaborn
2 Feature Engineering	fachlich sinnvolle Merkmale bauen, skalieren, kodieren	pandas, <code>sklearn.preprocessing</code>
3 Modellauswahl	ein zum Problem passendes Verfahren wählen	scikit-learn
4 Training	Modell auf den Trainingsdaten anpassen	<code>fit</code>
5 Evaluation	Kennzahlen auf den Testdaten, Cross-Validation	<code>score</code> , <code>sklearn.metrics</code>
6 Einsatz	Modell bereitstellen, überwachen, bei Bedarf neu trainieren	Modell speichern und laden

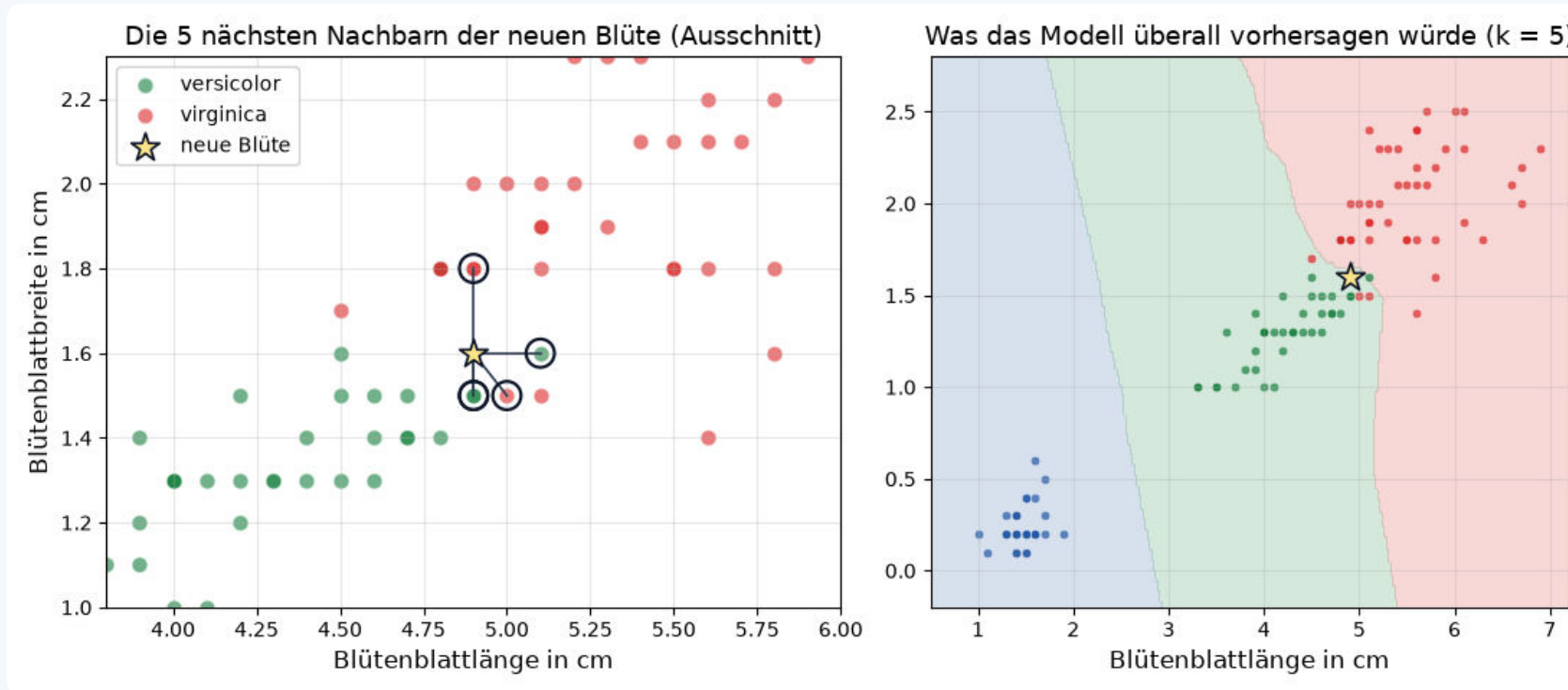
Gute Merkmale verbessern ein Modell meist mehr als ein Wechsel des Verfahrens.

Das erste Modell in zehn Zeilen

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier

X, y = load_iris(return_X_y=True)          # 150 Blüten, 4 Merkmale
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=1
)
model = KNeighborsClassifier(n_neighbors=5)
model.fit(X_train, y_train)                # lernen
print(model.predict(X_test[:5]))           # -> [0 1 1 0 2]
print(y_test[:5])                          # -> [0 1 1 0 2]
print(model.score(X_test, y_test))         # -> 1.0
```

Was k-nächste Nachbarn dabei tut



- Für eine neue Blüte sucht das Modell die $k = 5$ Trainingsblüten mit dem kleinsten Abstand
- Die Nachbarn stimmen ab: hier 3 mal versicolor, 2 mal virginica, also versicolor
- Rechts dieselbe Abstimmung für jeden Punkt der Fläche: so entstehen die Entscheidungsgebiete

Estimator-Schnittstelle: `fit`, `predict`, `score`

Die Modelle für überwachttes Lernen folgen demselben Muster:

Methode	Aufgabe
<code>model = Verfahren(...)</code>	Modell mit Einstellungen anlegen
<code>model.fit(X_train, y_train)</code>	aus den Trainingsdaten lernen
<code>model.predict(X_neu)</code>	Vorhersagen für neue Zeilen liefern
<code>model.score(X_test, y_test)</code>	Güte auf den Testdaten messen

```
from sklearn.linear_model import LogisticRegression

model = LogisticRegression(max_iter=1000) # nur diese Zeile ändert sich
model.fit(X_train, y_train)
print(model.score(X_test, y_test))      # -> 0.9666...
```

Typische Anwendungsfelder

Feld	Beispiele
Alltag	Sprachassistenten, automatische Übersetzung, Produktempfehlungen
Medizin und Gesundheitswesen	Unterstützung bei Diagnosen und Behandlungsentscheidungen
Marketing	personalisierte Werbung, Kundensegmentierung
Prognosen	Nachfrageprognosen, Ausfallwahrscheinlichkeiten, Risikobewertungen
Texte, Bilder, Audio	unstrukturierte Daten auswerten, z. B. mit Natural Language Processing oder Bildverarbeitung

Zusammenfassung

- KI ist der Oberbegriff, ML lernt aus Daten, Deep Learning nutzt neuronale Netze
- Überwachtes Lernen braucht gelabelte Beispiele, unüberwachtes Lernen sucht Strukturen ohne Lösung
- ML lohnt sich bei genug Daten, wiederkehrenden Entscheidungen und messbaren Fehlerkosten
- Merkmale bilden `x`, die Zielgröße ist `y`. Testdaten bleiben beim Training unter Verschluss
- Modelle für überwachtes Lernen folgen dem Muster `fit`, `predict`, `score`

Teil 5: Überwachtes Lernen: Regression und Klassifikation

Zahlen vorhersagen mit linearer Regression, Klassen vorhersagen mit logistischer Regression, Bäumen, Random Forest, SVM und k-nächsten Nachbarn.

Was Sie in diesem Teil lernen

- Eine lineare Regression trainieren und ihre Koeffizienten lesen
- Erklären, was Verlust und Gradientenabstieg bedeuten
- Fünf Klassifikationsverfahren in ihrer Grundidee unterscheiden
- Daten korrekt skalieren: erst teilen, dann `StandardScaler`
- Modelle mit `classification_report` nebeneinanderstellen

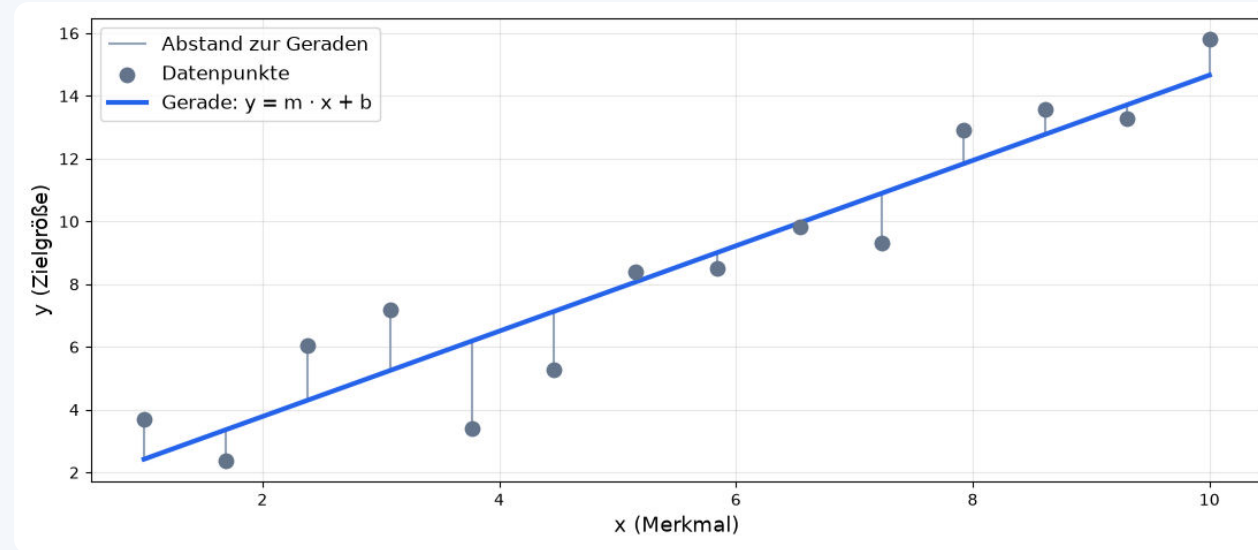
Regression und Klassifikation im Vergleich

	Regression	Klassifikation
Zielgröße	eine Zahl	eine Klasse
Beispiel im Kurs	mittlerer Hauswert eines Bezirks	Tumor gutartig oder bösartig
Einfachstes Verfahren	lineare Regression	logistische Regression
scikit-learn	<code>LinearRegression</code>	<code>LogisticRegression</code> , <code>DecisionTreeClassifier</code> , <code>RandomForestClassifier</code> , <code>SVC</code> , <code>KNeighborsClassifier</code>

Beides ist überwachtes Lernen: Für jedes Trainingsbeispiel ist die richtige Antwort bekannt.

Idee der linearen Regression

Eine Gerade durch die Datenpunkte, die den Zusammenhang möglichst gut beschreibt.



- einfach und gut interpretierbar
- schnell zu trainieren
- Grundlage für komplexere Modelle

Modellgleichung

Ein Merkmal:

$$y = m \cdot x + b$$

- x : Eingangsvariable, z. B. Werbebudget
- y : vorhergesagte Zielgröße, z. B. Umsatz
- m : Steigung der Geraden, zeigt den Einfluss von x auf y
- b : Achsenabschnitt, der Wert von y bei $x = 0$

Mehrere Merkmale (multiple lineare Regression):

$$y = b + m_1x_1 + m_2x_2 + \cdots + m_nx_n$$

Verlust (MSE) und Gradientenabstieg

Gesucht sind die Werte für m und b mit dem kleinsten mittleren quadratischen Fehler:

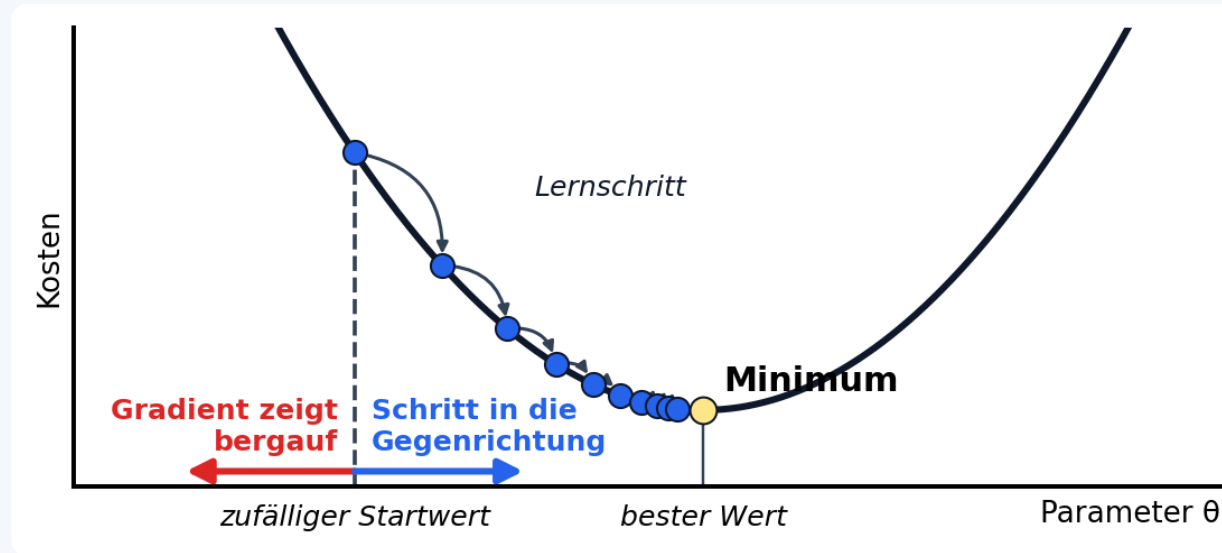
$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

- y_i : tatsächlicher Wert, $\hat{y}_i$: vorhergesagter Wert, n : Anzahl der Datenpunkte
- Große Abweichungen zählen durch das Quadrat besonders stark

Gradientenabstieg verbessert m und b schrittweise, α ist die Lernrate:

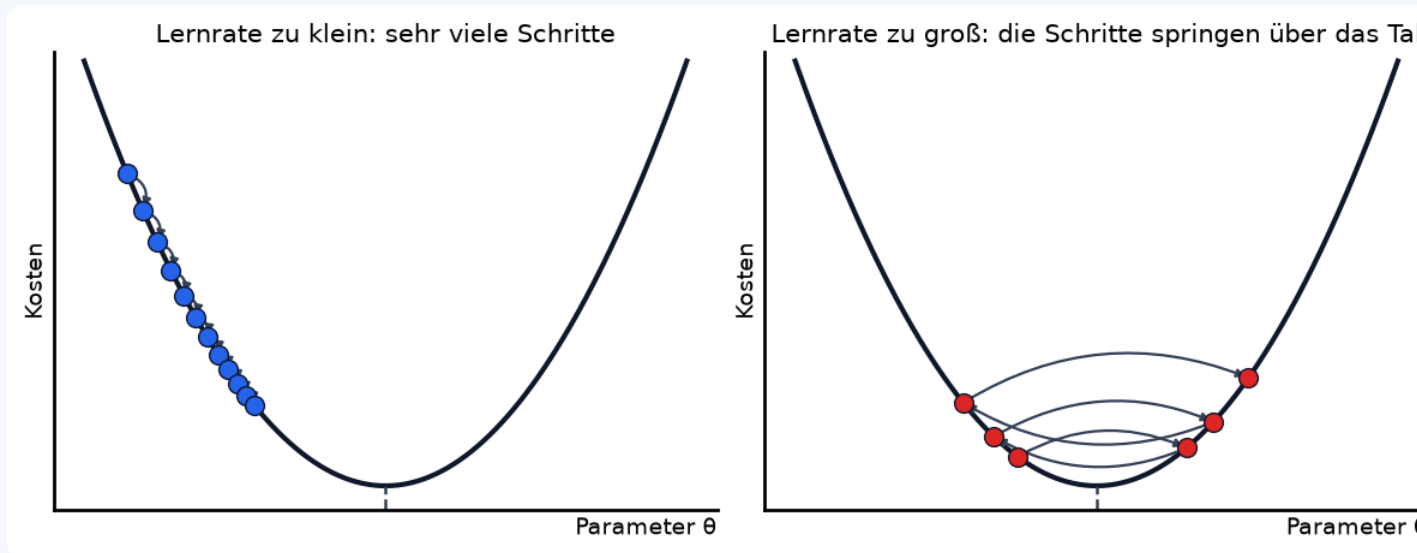
$$m := m - \alpha \cdot \frac{\partial MSE}{\partial m} \qquad b := b - \alpha \cdot \frac{\partial MSE}{\partial b}$$

Gradientenabstieg als Idee

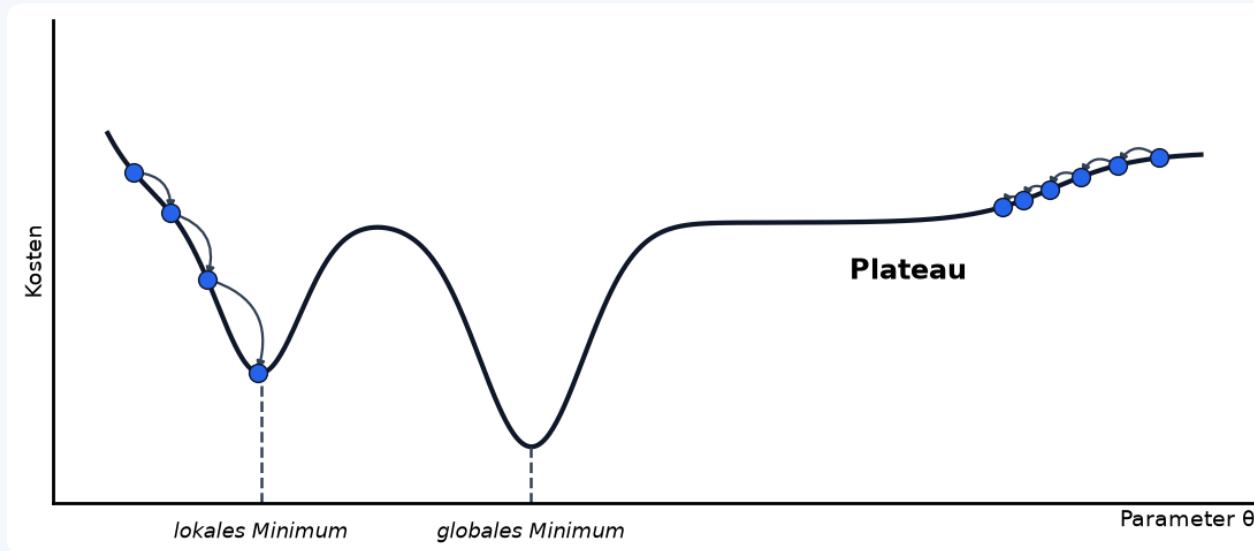


- Start mit zufälligen oder grob gewählten Parametern
- Der Gradient zeigt in die Richtung des stärksten Anstiegs, also gehen wir in die Gegenrichtung
- Lernrate zu groß: die Schritte springen über das Minimum. Zu klein: das Lernen dauert sehr lange
- Wiederholen, bis sich kaum noch etwas ändert (Konvergenz)

Lernrate: zu klein und zu groß



Lokales Minimum und Plateau



Gradientenabstieg in NumPy

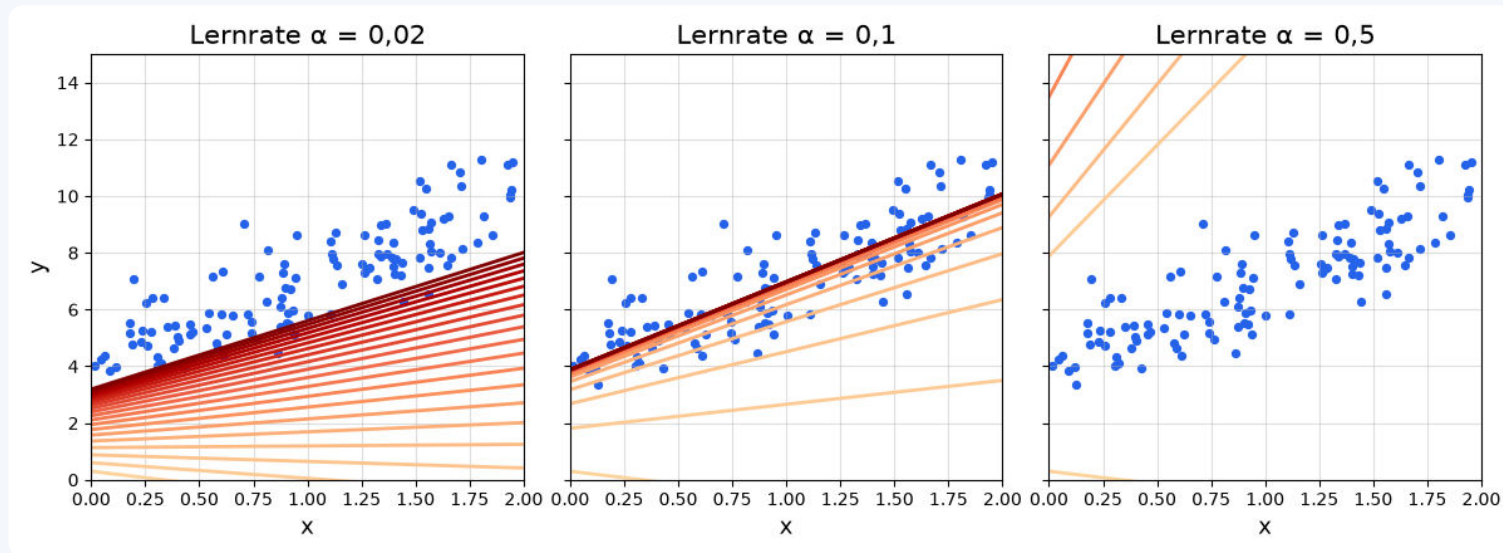
```
import numpy as np

rng = np.random.default_rng(42)
x = rng.uniform(0, 10, 100)
y = 2.5 * x + 1.0 + rng.normal(0, 1, 100)  # wahre Werte: m=2.5, b=1.0

m, b, alpha = 0.0, 0.0, 0.01
for schritt in range(1000):
    y_hat = m * x + b
    dm = -2 * np.mean(x * (y - y_hat))  # Ableitung nach m
    db = -2 * np.mean(y - y_hat)  # Ableitung nach b
    m = m - alpha * dm
    b = b - alpha * db
print(round(m, 3), round(b, 3))  # -> 2.508 0.944
```

Schritt	m	b	MSE
1	1.652	0.263	29.9
10	2.589	0.433	1.022
1000	2.508	0.944	0.959

Lernraten im Vergleich

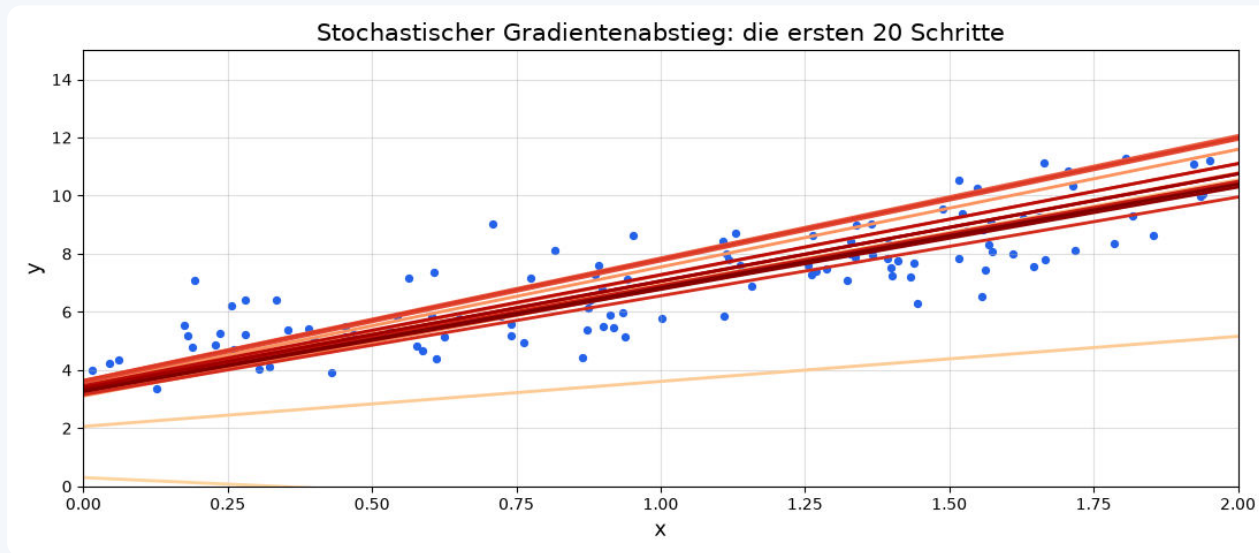


Exakte Lösung und Varianten des Gradientenabstiegs

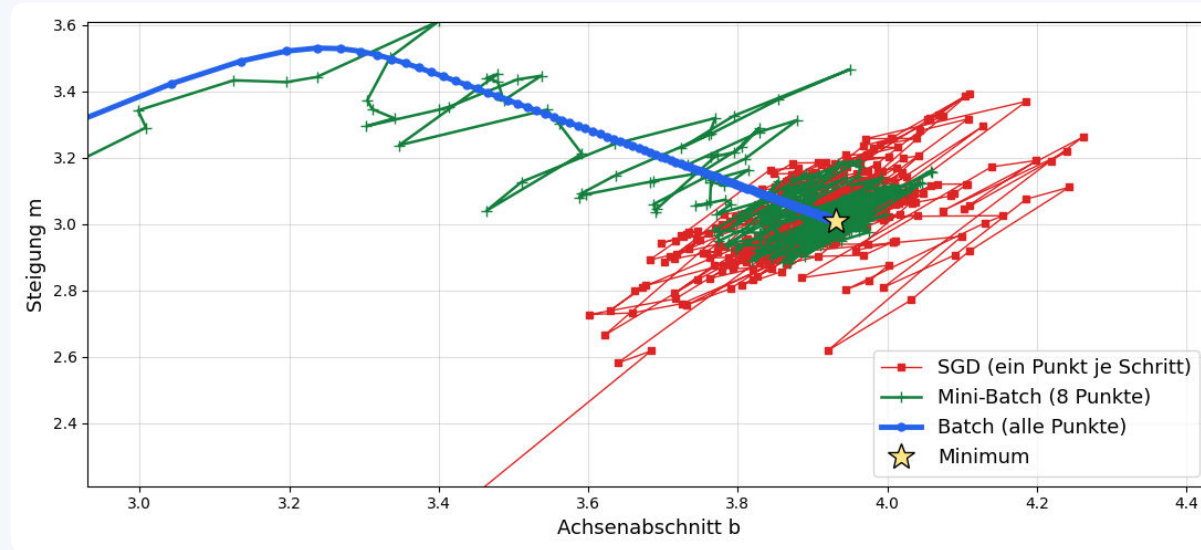
- Für die lineare Regression lässt sich das Minimum des MSE auch direkt berechnen: exakt, ohne Lernrate, schnell bei kleinen bis mittleren Datensätzen
- Bei sehr großen Datensätzen wird die direkte Berechnung teuer. Dann rechnet man schrittweise

Methode	Gradient aus	Vorteil	Nachteil
Batch Gradient Descent	allen Datenpunkten	stabile Konvergenz	rechenintensiv bei großen Datensätzen
Stochastic Gradient Descent (SGD)	einem zufälligen Datenpunkt pro Schritt	sehr effizient bei großen Datensätzen	schwankt stark
Mini-Batch Gradient Descent	kleiner Gruppe (z. B. 32 oder 64 Punkte)	Kompromiss: stabil und effizient	Batch-Größe muss passen

SGD: die ersten Schritte



Batch, Mini-Batch und SGD im Parameterraum



Datensatz California Housing

```
import pandas as pd

df = pd.read_csv("data/california_housing.csv")
print(df.shape)                # -> (20640, 9)

X = df.drop(columns="MedHouseVal")
y = df["MedHouseVal"]          # mittlerer Hauswert in 100.000 USD
print(X.columns.tolist())
```

```
['MedInc', 'HouseAge', 'AveRooms', 'AveBedrms',
 'Population', 'AveOccup', 'Latitude', 'Longitude']
```

Jede Zeile ist ein Bezirk in Kalifornien: mittleres Einkommen, Alter der Häuser, Zimmer und Schlafzimmer pro Haushalt, Einwohner, Belegung, Lage.

LinearRegression trainieren

```
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=1
)
model = LinearRegression()
model.fit(X_train, y_train)
y_pred = model.predict(X_test)

print(y_pred[:3].round(2))                # -> [2.13 0.94 2.71]
print(y_test[:3].values)                  # -> [3.55 0.707 2.294]
print(round(mean_squared_error(y_test, y_pred), 3)) # -> 0.529
print(round(r2_score(y_test, y_pred), 3))    # -> 0.597
```

Koeffizienten lesen

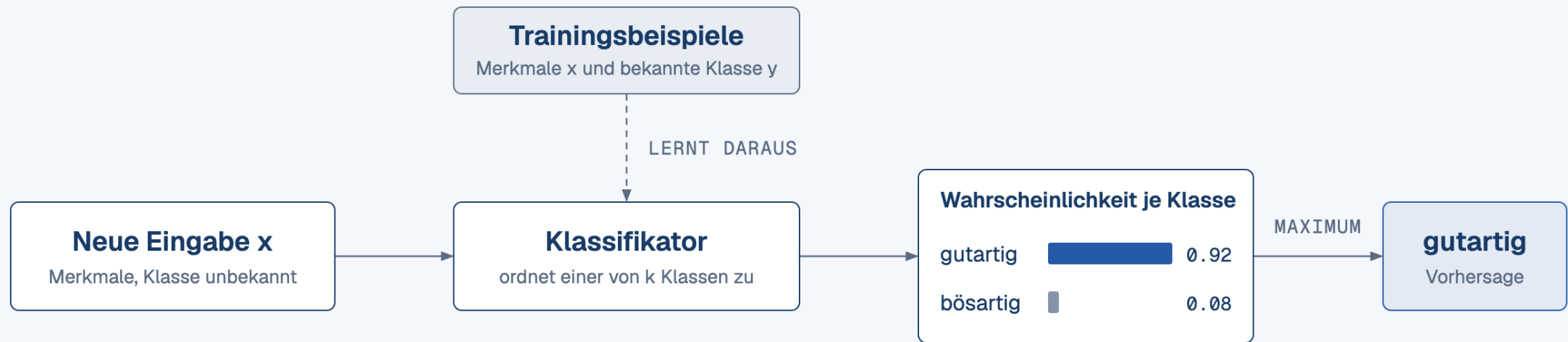
```
koef = pd.Series(model.coef_, index=X.columns).round(3)
print(koef)
print(round(model.intercept_, 2))      # -> -37.52
```

MedInc	0.439
HouseAge	0.010
AveRooms	-0.105
AveBedrms	0.632
Population	-0.000
AveOccup	-0.003
Latitude	-0.426
Longitude	-0.441

- **MedInc** = 0.439: Steigt das mittlere Einkommen um eine Einheit (10.000 USD), steigt der vorhergesagte Hauswert um 0.439 Einheiten (rund 44.000 USD), wenn alle anderen Merkmale gleich bleiben
- Das Vorzeichen zeigt die Richtung, der Betrag die Stärke pro Einheit des Merkmals

Koeffizienten verschiedener Merkmale sind nur vergleichbar, wenn die Merkmale dieselbe Skala haben. Population (Tausende) und AveBedrms (um 1) haben sie nicht.

Klassifikation: Problemstellung



- Gegeben: Trainingsbeispiele mit Merkmalsvektor $\mathbf{x}$ und bekannter Klasse y
- Gesucht: eine Funktion, die neue Eingaben einer von k vordefinierten Klassen zuordnet
- Viele Modelle schätzen dazu eine Wahrscheinlichkeit je Klasse, $P(y = C_k \mid \mathbf{x})$, und geben die Klasse mit der höchsten Wahrscheinlichkeit aus
- Zwei Klassen: **binäre** Klassifikation (Spam oder nicht, gutartig oder böartig)

Datensatz Brustkrebs

```
from sklearn.datasets import load_breast_cancer

data = load_breast_cancer()
X = pd.DataFrame(data.data, columns=data.feature_names)
y = data.target

print(X.shape)           # -> (569, 30)
print(data.target_names) # -> ['malignant' 'benign']
print(np.bincount(y))    # -> [212 357]
```

- 569 Gewebeproben, 30 Messwerte je Probe (Radius, Textur, Fläche, Konkavität der Zellkerne)
- Zielgröße: 0 = bösartig (malignant), 1 = gutartig (benign)

Die Kodierung ist andersherum, als man erwartet: Die 1 steht hier für gutartig. Prüfen Sie immer target_names.

Vorbereitung: teilen, dann skalieren

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=1
)
scaler = StandardScaler()
X_train_s = scaler.fit_transform(X_train)    # lernt Mittelwert und Streuung
X_test_s = scaler.transform(X_test)         # wendet sie nur an
```

- `StandardScaler` bringt jedes Merkmal auf Mittelwert 0 und Standardabweichung 1
- Logistische Regression, SVM und k-nächste Nachbarn reagieren empfindlich auf unterschiedliche Skalen

`fit_transform` nur auf den Trainingsdaten. Wer vor dem Split skaliert, lässt Information aus den Testdaten ins Training: ein Datenleck.

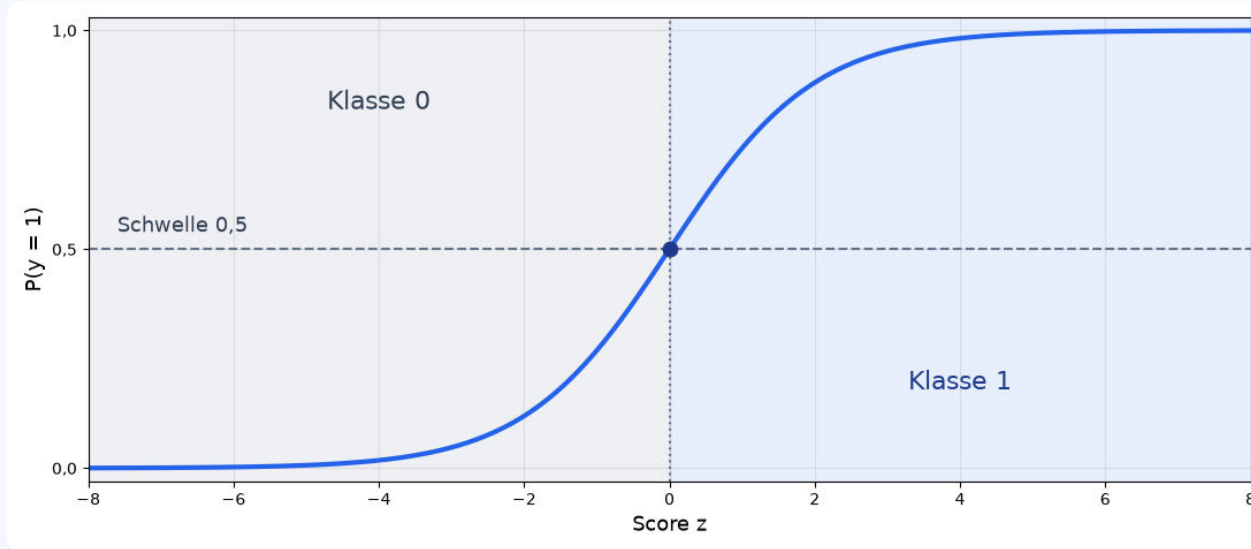
Logistische Regression: Idee

- Trotz des Namens ein Verfahren für **Klassifikation**
- Schritt 1: ein linearer Score wie bei der Regression, $z = \mathbf{w}^\top \mathbf{x} + b$
- Schritt 2: die Sigmoid-Funktion macht aus dem Score eine Wahrscheinlichkeit zwischen 0 und 1
- Schritt 3: eine Schwelle (Standard 0,5) macht aus der Wahrscheinlichkeit eine Klasse



Sigmoid und Schwelle

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$



- Stark negativer Score: Wahrscheinlichkeit nahe 0. Stark positiver Score: nahe 1
- Bei $z = 0$ ist die Wahrscheinlichkeit genau 0,5

Logistische Regression in scikit-learn

```
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report

logreg = LogisticRegression(max_iter=1000)
logreg.fit(X_train_s, y_train)
y_pred = logreg.predict(X_test_s)
print(classification_report(y_test, y_pred,
                           target_names=data.target_names))
```

	precision	recall	f1-score	support
malignant	0.98	0.95	0.96	42
benign	0.97	0.99	0.98	72
accuracy			0.97	114

Wahrscheinlichkeiten und Schwelle verschieben

```
from sklearn.metrics import confusion_matrix

proba = logreg.predict_proba(X_test_s)[: , 1]    # P(gutartig)
print(proba[:4].round(3))                        # -> [0.862 0.01  0.99  0.006]

y_streng = (proba >= 0.7).astype(int)            # gutartig erst ab 70 %
print(confusion_matrix(y_test, y_streng))
# -> [ [40  2]
#      [ 1 71] ]
```

Schwelle für „gutartig“	bösartig übersehen	gutartig fälschlich als bösartig
0,3	3	0
0,5 (Standard)	2	1
0,7	2	1
0,9	1	7

predict rechnet immer mit 0,5. Für eine andere Schwelle nehmen Sie predict_proba und vergleichen selbst.

Entscheidungsbaum: Idee



- **Wurzelknoten:** Startpunkt mit allen Daten
- **Entscheidungsknoten:** stellt eine Bedingung, z. B. „Merkmal > 5?“
- **Blatt:** Endpunkt, gibt eine Klasse aus
- **Training:** am besten Split-Punkt teilen, dann für jede Teilmenge wiederholen

Entscheidungsbaum in scikit-learn

```
from sklearn.tree import DecisionTreeClassifier, export_text

baum = DecisionTreeClassifier(max_depth=2, random_state=1)
baum.fit(X_train, y_train)          # Bäume brauchen keine Skalierung
print(export_text(baum, feature_names=list(X.columns)))
print(round(baum.score(X_test, y_test), 3))  # -> 0.886
```

- `export_text` gibt den Baum als eingerückten Text aus, die nächste Folie zeigt ihn als Diagramm
- Drei Fragen reichen für 89 % richtige Vorhersagen auf den Testdaten

Der Baum als Diagramm



- Jeder Knoten prüft ein Merkmal gegen eine Schwelle: bei „ja“ nach links, bei „nein“ nach rechts
- Ein Fall wandert von der Wurzel bis zu einem Blatt. Das Blatt gibt die Klasse aus, die dort im Training die Mehrheit hatte
- Die Blätter sind nicht rein: Im Blatt ganz rechts liegen 9 gutartige Fälle unter 165

Baumtiefe

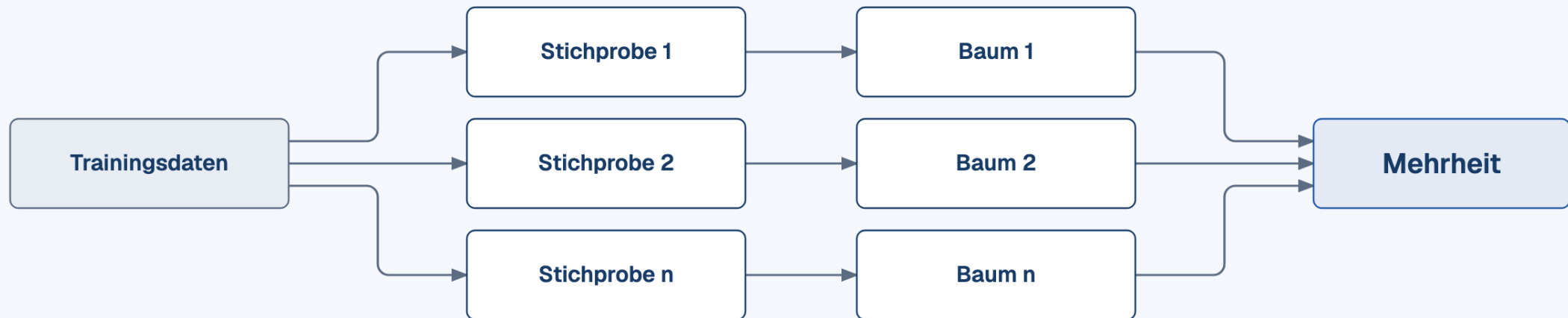
max_depth	Accuracy Training	Accuracy Test
1	0.930	0.877
2	0.958	0.886
3	0.969	0.912
5	1.000	0.947
ohne Grenze (Tiefe 5)	1.000	0.947

- Ein tiefer Baum lernt die Trainingsdaten auswendig: 100 % im Training
- Auf den Testdaten bleibt derselbe Baum bei 0.947 stehen
- `max_depth` ist der wichtigste Hyperparameter eines Baums

Einzelne Entscheidungsbäume neigen zu Overfitting. Begrenzen Sie die Tiefe oder kombinieren Sie viele Bäume.

Random Forest: Idee

- **Ensemble-Verfahren:** kombiniert viele Entscheidungsbäume
- **Bootstrapping:** jeder Baum bekommt eine eigene Zufallsstichprobe der Trainingsdaten, gezogen mit Zurücklegen
- **Bagging** (Bootstrap Aggregation): viele Bäume auf unterschiedlichen Stichproben trainieren, Ergebnisse zusammenführen
- Klassifikation: **Mehrheitsentscheid** der Bäume. Regression: Durchschnitt



Random Forest in scikit-learn

```
from sklearn.ensemble import RandomForestClassifier

rf = RandomForestClassifier(n_estimators=100, random_state=1)
rf.fit(X_train_s, y_train)
print(round(rf.score(X_test_s, y_test), 3))          # -> 0.956

wichtig = pd.Series(rf.feature_importances_, index=X.columns)
print(wichtig.sort_values(ascending=False).head(3).round(3))
# -> worst perimeter 0.131, worst concave points 0.130,
#      worst area 0.118
```

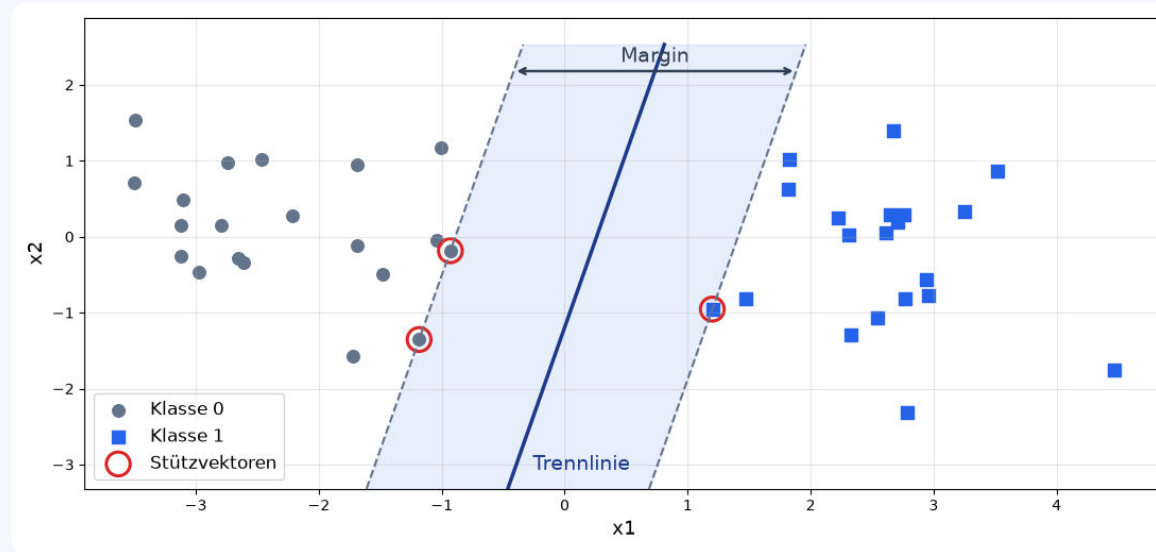
Vorteile

- meist genauer als ein Einzelbaum
- weniger anfällig für Overfitting
- bewertet die Wichtigkeit der Merkmale

Nachteile

- langsamer als ein einzelner Baum
- speicher- und rechenintensiv bei großen Datenmengen
- nicht mehr als Ganzes vorlesbar

SVM: Trennlinie und Margin



- Eine Support Vector Machine sucht die Trennlinie mit dem **größten Abstand** (Margin) zu beiden Klassen
- Nur die Punkte am Rand bestimmen die Linie: die **Stützvektoren** (Support Vectors)
- Bei mehr als zwei Merkmalen wird aus der Linie eine Hyperebene
- Eignet sich auch für kleine Datensätze mit vielen Merkmalen

SVM: Soft Margin und Kernel

- **Hard Margin:** perfekte Trennung ohne Fehler. Funktioniert nur bei sauber trennbaren Daten und neigt zu Overfitting
- **Soft Margin:** lässt einzelne falsch liegende Punkte zu und generalisiert besser
- Der Parameter `C` steuert den Kompromiss: großes `C` duldet wenige Fehler (schmale Margin), kleines `C` duldet mehr (breite Margin)
- **Kernel:** Sind die Klassen nicht mit einer Geraden trennbar, vergleicht die SVM die Punkte über eine Ähnlichkeitsfunktion, als lägen sie in einem höherdimensionalen Raum

Kernel	Geeignet für
<code>linear</code>	linear trennbare Daten
<code>poly</code>	gekrümmte Entscheidungsgrenzen
<code>rbf</code> (Standard)	beliebig geformte Grenzen, misst die Ähnlichkeit zweier Punkte über ihren Abstand

SVM und k-nächste Nachbarn in scikit-learn

SVM

```
from sklearn.svm import SVC

svm = SVC(kernel="rbf", C=1.0, gamma="scale")
svm.fit(X_train_s, y_train)
print(round(svm.score(X_test_s, y_test), 3))
# -> 0.974
```

k-nächste Nachbarn (kNN)

```
from sklearn.neighbors import KNeighborsClassifier

knn = KNeighborsClassifier(n_neighbors=5)
knn.fit(X_train_s, y_train)
print(round(knn.score(X_test_s, y_test), 3))
# -> 0.956
```

- kNN sucht zu einem neuen Punkt die **k** ähnlichsten Trainingspunkte und nimmt deren häufigste Klasse
- kNN lernt beim **fit** nichts: Es merkt sich die Trainingsdaten und rechnet erst bei **predict**
- Beide Verfahren rechnen mit Abständen. Ohne Skalierung dominiert das Merkmal mit den größten Zahlen

Modellvergleich in einer Schleife

```
from sklearn.metrics import accuracy_score

modelle = {
    "Logistische Regression": LogisticRegression(max_iter=1000),
    "Entscheidungsbaum": DecisionTreeClassifier(max_depth=3,
                                                random_state=1),
    "Random Forest": RandomForestClassifier(n_estimators=100,
                                           random_state=1),
    "SVM (RBF)": SVC(kernel="rbf", C=1.0, gamma="scale"),
    "kNN (k=5)": KNeighborsClassifier(n_neighbors=5),
}

for name, modell in modelle.items():
    modell.fit(X_train_s, y_train)
    acc_train = modell.score(X_train_s, y_train)
    acc_test = accuracy_score(y_test, modell.predict(X_test_s))
    print(f"{name:24s} {acc_train:.3f} {acc_test:.3f}")
```

Modellvergleich als Tabelle

Verfahren	Accuracy Training	Accuracy Test	Skalierung nötig	Nachvollziehbar	Wichtigste Stellschraube
Logistische Regression	0.991	0.974	ja	gut (Koeffizienten)	C
Entscheidungsbaum (Tiefe 3)	0.969	0.912	nein	sehr gut (vorlesbar)	max_depth
Random Forest (100 Bäume)	1.000	0.956	nein	mittel (Merkmalswichtigkeit)	n_estimators , max_depth
SVM (RBF)	0.987	0.974	ja	gering	C , gamma , kernel
kNN (k = 5)	0.982	0.956	ja	mittel (Nachbarn zeigen)	n_neighbors

114 Testfälle: Ein einziger Fall macht knapp einen Prozentpunkt aus. Aus dieser Tabelle folgt keine Rangliste.

Zusammenfassung

- Lineare Regression legt eine Gerade (oder Ebene) durch die Daten. Training heißt: den MSE minimieren
- Gradientenabstieg geht schrittweise bergab, die Lernrate bestimmt die Schrittweite
- Logistische Regression liefert Wahrscheinlichkeiten. Die Schwelle ist eine fachliche Entscheidung
- Erst teilen, dann skalieren. Bäume und Random Forest kommen ohne Skalierung aus, SVM und kNN nicht

Teil 6: Modelltraining, Feature Engineering, Hyperparameter und Evaluation

Wie Sie Merkmale aufbereiten, Modelle ehrlich prüfen, Einstellungen systematisch suchen und Ergebnisse mit den passenden Kennzahlen bewerten.

Was Sie in diesem Teil lernen

- Merkmale skalieren, kodieren, transformieren und neue Merkmale aus vorhandenen bilden
- Overfitting und Underfitting erkennen und mit Cross-Validation ehrlich messen
- Hyperparameter mit `GridSearchCV` systematisch suchen
- Regressionsmodelle mit MAE, MSE, RMSE und R^2 bewerten
- Klassifikationsmodelle mit Konfusionsmatrix, Precision, Recall, F1 und ROC bewerten, auch bei ungleichen Klassen

Feature Engineering im Überblick

Feature Engineering heißt: aus Rohdaten Merkmale machen, mit denen ein Modell gut lernen kann.

Schritt	Was passiert	Beispiel
Skalieren	Zahlen auf vergleichbare Größenordnung bringen	Age und Fare standardisieren
Kodieren	Kategorien in Zahlen umwandeln	Sex , Embarked als 0/1-Spalten
Transformieren	schiefe Verteilungen glätten	$\log(1 + \text{Fare})$
Neu bilden	Merkmale kombinieren oder gruppieren	Familiengröße, Altersgruppe
Auswählen	überflüssige Merkmale weglassen	Korrelation prüfen

Ein gut gebautes Merkmal bringt oft mehr als der Wechsel auf ein komplizierteres Modell.

Skalierung: wann sie nötig ist

Age liegt zwischen 0 und 80, **Fare** zwischen 0 und 512. Ohne Skalierung dominiert bei vielen Verfahren das Merkmal mit den größeren Zahlen.

Verfahren	Skalierung nötig?	Grund
Logistische Regression	ja	Gradientenabstieg hängt von der Größenordnung ab
k-NN, SVM, k-Means	ja	rechnen mit Abständen
PCA	ja	sucht Richtungen mit großer Varianz
Neuronale Netze	ja	Training läuft stabiler und schneller
Entscheidungsbaum, Random Forest	nein	vergleichen nur Schwellen je Merkmal

StandardScaler und MinMaxScaler

Scaler	Formel	Ergebnis	Einsatz
StandardScaler	$z = \frac{x - \mu}{\sigma}$	Mittelwert 0, Standardabweichung 1	Standardwahl
MinMaxScaler	$x' = \frac{x - x_{\min}}{x_{\max} - x_{\min}}$	Werte zwischen 0 und 1	begrenzte Wertebereiche, neuronale Netze

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=1)

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)  # lernt Mittelwert, Streuung
X_test_scaled = scaler.transform(X_test)        # wendet sie nur an
```

Erst teilen, dann skalieren. `fit_transform` nur auf den Trainingsdaten, auf den Testdaten nur `transform`. Sonst fließt Wissen aus den Testdaten ins Training.

One-Hot- gegenüber Label-Encoding

One-Hot-Encoding: eine 0/1-Spalte je Kategorie

ID	Farbe	Farbe_Rot	Farbe_Blau	Farbe_Grün
1	Rot	1	0	0
2	Blau	0	1	0
3	Grün	0	0	1
4	Rot	1	0	0

Label-Encoding: eine Zahl je Kategorie

ID	Farbe	Farbe_Label
1	Rot	2
2	Blau	0
3	Grün	1
4	Rot	2

- Label-Encoding behauptet eine Reihenfolge: Blau (0) kleiner als Grün (1) kleiner als Rot (2). Bei Farben ist das falsch.
- Für Kategorien ohne Rangfolge: One-Hot. Für echte Rangfolgen (Schulnote, Pflegegrad): Zahlen in der richtigen Reihenfolge.

Encoding in pandas und scikit-learn

```
import pandas as pd
from sklearn.preprocessing import LabelEncoder, OneHotEncoder

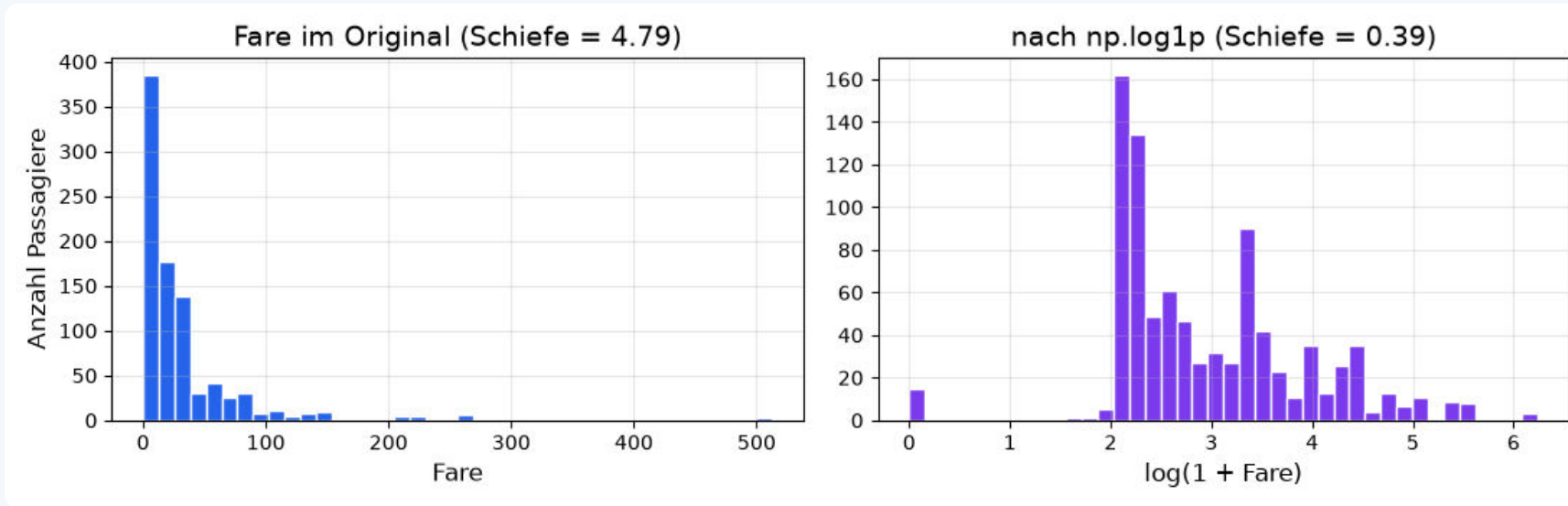
df = pd.read_csv("data/titanic.csv")
# One-Hot mit pandas: schnell für die Analyse
encoded = pd.get_dummies(df, columns=["Sex", "Embarked"], drop_first=True)

# One-Hot mit scikit-learn: merkt sich die Kategorien für neue Daten
ohe = OneHotEncoder(handle_unknown="ignore", sparse_output=False)
X_cat = ohe.fit_transform(df[ ["Sex", "Pclass"] ])
print(ohe.get_feature_names_out())
# -> ['Sex_female' 'Sex_male' 'Pclass_1' 'Pclass_2' 'Pclass_3']

# Label-Encoding: eine Zahl je Kategorie
df["Sex_label"] = LabelEncoder().fit_transform(df["Sex"])
print(df[ ["Sex", "Sex_label"] ].head(3))    # female -> 0, male -> 1
```

- `drop_first=True` lässt eine Spalte weg: aus `Sex_female = 0` folgt `Sex_male = 1`
- `handle_unknown="ignore"` : eine unbekannte Kategorie in neuen Daten führt nicht zum Abbruch

Log-Transformation schiefer Größen



```
import numpy as np
df["Fare_log"] = np.log1p(df["Fare"])  #  $\log(1 + x)$ , verträgt auch  $\text{Fare} = 0$ 
```

- Wenige sehr teure Tickets ziehen die Verteilung nach rechts. Der Logarithmus staucht große Werte stärker als kleine.
- Typische Kandidaten: Preise, Einkommen, Kosten, Wartezeiten

Neue Merkmale aus vorhandenen

```
# Familiengröße: Geschwister/Partner + Eltern/Kinder + die Person selbst
df["Familiengroesse"] = df["SibSp"] + df["Parch"] + 1
df["Allein"] = (df["Familiengroesse"] == 1).astype(int)

# Altersgruppe: aus einer Zahl wird eine Kategorie (Binning)
df["Altersgruppe"] = pd.cut(
    df["Age"], bins=[0, 12, 18, 30, 50, 80],
    labels=["Kind", "Jugendlich", "Jung", "Erwachsen", "Senior"])

print(df.groupby("Altersgruppe", observed=True)["Survived"].mean().round(2))
print(df.groupby("Familiengroesse")["Survived"].mean().round(2))
```

- Fachwissen steckt im Merkmal: nicht `SibSp` und `Parch` einzeln zählen, sondern „reist allein oder mit Familie“
- Weitere Muster: Wochentag oder Monat aus einem Datum, Verhältnisse (Kosten je Arztbesuch), Summen je Gruppe

Ein neues Merkmal darf nur Informationen nutzen, die zum Zeitpunkt der Vorhersage bekannt sind.

Train, Validierung und Test

Teil	Wofür	Wie oft ansehen
Training	Modell lernt seine Parameter	beliebig oft
Validierung	Modelle und Einstellungen vergleichen	beliebig oft
Test	einmalige Schlussprüfung	genau einmal, ganz am Ende

```
from sklearn.model_selection import train_test_split

# 1. Testteil abspalten (20 Prozent), 2. Rest in Training und Validierung
X_rest, X_test, y_rest, y_test = train_test_split(
    X, y, test_size=0.2, random_state=1, stratify=y)
X_train, X_val, y_train, y_val = train_test_split(
    X_rest, y_rest, test_size=0.25, random_state=1, stratify=y_rest)
# Ergebnis: 60 Prozent Training, 20 Prozent Validierung, 20 Prozent Test
```

Wer Einstellungen so lange ändert, bis der Testwert gut aussieht, hat den Testteil zum Training benutzt. Die Zahl ist dann zu optimistisch.

Overfitting und Underfitting

- **Underfitting:** Das Modell ist zu einfach und verpasst das Muster. Schlecht auf Trainings- und auf neuen Daten.
- **Overfitting:** Das Modell ist zu flexibel und lernt das Rauschen der Trainingsdaten auswendig. Sehr gut im Training, schlecht auf neuen Daten.
- **Generalisierung:** Ziel ist ein niedriger Fehler auf Daten, die das Modell nie gesehen hat.
- **Stellschrauben:** Modellkomplexität (Kapazität), mehr Daten, bessere Merkmale

Polynombeispiel: Code

```
import numpy as np
from numpy.polynomial import Polynomial

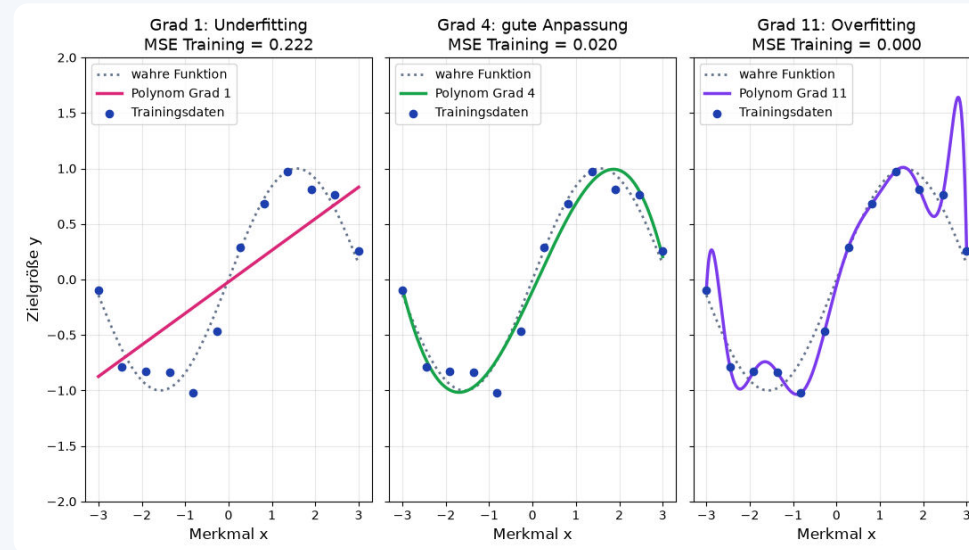
rng = np.random.default_rng(42)
X_train = np.linspace(-3, 3, 12)           # 12 Messpunkte
y_train = np.sin(X_train) + rng.normal(0, 0.15, 12) # Sinus plus Rauschen

X_neu = np.linspace(-2.9, 2.9, 50)        # neue Punkte
y_neu = np.sin(X_neu)

for grad in [1, 4, 11]:
    p = Polynomial.fit(X_train, y_train, grad)
    mse_train = np.mean((p(X_train) - y_train) ** 2)
    mse_neu = np.mean((p(X_neu) - y_neu) ** 2)
    print(f"Grad {grad:2d}: Training {mse_train:.3f}, neu {mse_neu:.3f}")
```

```
Grad  1: Training 0.222, neu 0.172
Grad  4: Training 0.020, neu 0.010
Grad 11: Training 0.000, neu 0.111
```

Polynombeispiel: Grad 1, 4 und 11



Bias und Varianz als Idee

	Underfitting	Overfitting
Fachbegriff	hoher Bias	hohe Varianz
Bedeutung	Modell liegt systematisch daneben	Modell reagiert auf jede Zufallsschwankung der Trainingsdaten
Trainingsfehler	hoch	sehr niedrig
Fehler auf neuen Daten	hoch	deutlich höher als im Training
Im Polynombeispiel	Grad 1	Grad 11
Abhilfe	flexibleres Modell, bessere Merkmale	einfacheres Modell, mehr Daten

- Mehr Komplexität senkt den Bias und erhöht die Varianz. Gesucht ist die Mitte.
- Diagnose: Trainings- und Validierungswert nebeneinander über die Modellkomplexität auftragen (zum Beispiel über `max_depth`)

Cross-Validation: die Idee

Ein einzelner Split hängt vom Zufall ab. Cross-Validation teilt die Daten in k Blöcke (Folds) und lässt jeden Block einmal Prüfdaten sein.

Durchlauf 1	Prüfen	Training	Training	Training	Training	Score 1
Durchlauf 2	Training	Prüfen	Training	Training	Training	Score 2
Durchlauf 3	Training	Training	Prüfen	Training	Training	Score 3
Durchlauf 4	Training	Training	Training	Prüfen	Training	Score 4
Durchlauf 5	Training	Training	Training	Training	Prüfen	Score 5

Ergebnis: Mittelwert und Streuung der 5 Scores

- Jede Datenzeile wird genau einmal zum Prüfen benutzt, alle Daten werden genutzt
- Die Streuung zeigt, wie stabil das Ergebnis ist
- Preis: k Trainingsläufe statt einem

cross_val_score und StratifiedKFold

```
from sklearn.datasets import load_breast_cancer
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import StratifiedKFold, cross_val_score

X, y = load_breast_cancer(return_X_y=True)
model = RandomForestClassifier(random_state=1)

cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=1)
scores = cross_val_score(model, X, y, cv=cv, scoring="roc_auc")

print(scores.round(3))
print(f"Mittel: {scores.mean():.3f}, Streuung: {scores.std():.3f}")
# -> [0.994 0.999 0.987 0.989 0.986]
# -> Mittel: 0.991, Streuung: 0.005
```

- `StratifiedKFold` hält in jedem Fold das Klassenverhältnis des Gesamtdatensatzes. Ohne das kann ein Fold zufällig kaum Fälle der seltenen Klasse enthalten.
- `scoring` wählt die Kennzahl: `"accuracy"`, `"f1"`, `"roc_auc"`, bei Regression `"r2"` oder `"neg_mean_absolute_error"`

Hyperparameter gegenüber Parametern

	Parameter	Hyperparameter
Wer legt sie fest?	das Training (<code>fit</code>)	Sie, vor dem Training
Woraus?	aus den Daten gelernt	ausprobiert und verglichen
Beispiele	Koeffizienten <code>coef_</code> und <code>intercept_</code> der Regression, Schwellen im Baum	<code>max_depth</code> beim Baum, <code>n_estimators</code> beim Random Forest, <code>k</code> bei k-NN
Wo im Code?	Attribute mit Unterstrich nach <code>fit</code>	Argumente im Konstruktor

- Schlecht gewählte Hyperparameter führen zu Underfitting oder Overfitting
- Hyperparameter werden auf Validierungsdaten oder per Cross-Validation verglichen, nie auf dem Testteil

GridSearchCV: beste Tiefe für einen Baum

```
import pandas as pd
from sklearn.model_selection import GridSearchCV, train_test_split
from sklearn.tree import DecisionTreeRegressor

df = pd.read_csv("data/california_housing.csv")
X, y = df.drop(columns="MedHouseVal"), df["MedHouseVal"]
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=1)

depths = [2, 4, 6, 8, 10, 12, 16, 20]
search = GridSearchCV(DecisionTreeRegressor(random_state=1),
                      {"max_depth": depths}, cv=5, scoring="r2")
search.fit(X_train, y_train)

print(search.best_params_, round(search.best_score_, 3))
# -> {'max_depth': 8} 0.682
print(f"R² auf dem Testteil: {search.score(X_test, y_test):.2f}") # -> 0.71
```

- `DecisionTreeRegressor` : derselbe Baum wie bei der Klassifikation, nur sagt jedes Blatt eine Zahl voraus
- Tiefe 2 ist zu grob (R^2 0.44), Tiefe 20 lernt auswendig (R^2 0.60). Die Cross-Validation findet die Mitte bei Tiefe 8
- 8 Kandidaten mal 5 Folds: 40 Trainingsläufe. Danach trainiert `GridSearchCV` das beste Modell auf allen Trainingsdaten neu.

GridSearchCV: Tiefe eines Random Forest

```
from sklearn.metrics import f1_score
X, y = load_breast_cancer(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, stratify=y, random_state=1)

param_grid = {"max_depth": [2, 3, 4, 5, 10, 20, None], # None: unbegrenzt
              "n_estimators": [50, 100, 200]}
search = GridSearchCV(RandomForestClassifier(random_state=1),
                      param_grid, cv=cv, scoring="f1", n_jobs=-1)
search.fit(X_train, y_train)

print(search.best_params_, round(search.best_score_, 3))
print(f"F1 auf dem Testteil: {f1_score(y_test, search.predict(X_test)):.3f}")
# -> {'max_depth': 10, 'n_estimators': 100} 0.965
# -> F1 auf dem Testteil: 0.966
```

- 7 mal 3 Kombinationen mal 5 Folds: 105 Trainingsläufe. Jeder weitere Hyperparameter vervielfacht den Aufwand.

RandomizedSearchCV und die Grenzen der Gittersuche

Verfahren	Vorgehen	Geeignet, wenn
Grid Search	probiert jede Kombination des Gitters	wenige Hyperparameter, wenige Werte
Random Search	zieht zufällig <code>n_iter</code> Kombinationen	viele Hyperparameter, begrenzte Rechenzeit
Bayes-Optimierung	schlägt aus bisherigen Ergebnissen neue Kandidaten vor	teure Trainingsläufe (Zusatzpakete nötig)

```
from sklearn.model_selection import RandomizedSearchCV

search = RandomizedSearchCV(
    RandomForestClassifier(random_state=1), param_distributions=param_grid,
    n_iter=8, cv=cv, scoring="f1", random_state=1, n_jobs=-1)
search.fit(X_train, y_train)
print(search.best_params_, round(search.best_score_, 3))
# -> {'n_estimators': 100, 'max_depth': 20} 0.965
```

- 8 statt 21 Kombinationen, derselbe Wert: in diesem Beispiel reicht die Zufallssuche

Evaluation Regression: MAE

Mean Absolute Error: mittlere absolute Abweichung

$$MAE = \frac{1}{m} \sum_{i=1}^m |y_i - \hat{y}_i|$$

- Durchschnittlicher Abstand zwischen Vorhersage $\hat{y}$ und wahrem Wert y , in der Einheit der Zielgröße
- Je kleiner, desto besser. MAE = 1000 bei Kosten in Euro heißt: Die Vorhersage liegt im Mittel 1000 Euro daneben.
- Jeder Fehler zählt mit seinem Betrag, einzelne Ausreißer fallen wenig ins Gewicht

```
from sklearn.metrics import mean_absolute_error

y_true = [3, 5, 8]
y_pred = [2, 5, 11]
print(mean_absolute_error(y_true, y_pred))    # -> 1.33 = (1 + 0 + 3) / 3
```


Evaluation Regression: MSE und RMSE

Mean Squared Error: mittlerer quadratischer Fehler

$$MSE = \frac{1}{m} \sum_{i=1}^m (y_i - \hat{y}_i)^2$$

Root Mean Squared Error: Wurzel daraus

$$RMSE = \sqrt{MSE}$$

- Das Quadrat bestraft große Fehler stärker: ein Fehler von 3 zählt neunfach, nicht dreifach
- MSE hat die quadrierte Einheit (Euro²) und ist schwer zu deuten. RMSE hat wieder die Einheit der Zielgröße.
- RMSE deutlich größer als MAE: Es gibt einzelne große Ausreißer in den Fehlern.

```
import numpy as np
from sklearn.metrics import mean_squared_error

mse = mean_squared_error(y_true, y_pred)      # -> 3.33 = (1 + 0 + 9) / 3
rmse = np.sqrt(mse)                          # -> 1.83
```

Evaluation Regression: R^2

Bestimmtheitsmaß

$$R^2 = 1 - \frac{\sum (y_i - \hat{y}_i)^2}{\sum (y_i - \bar{y})^2}$$

Vergleicht das Modell mit der einfachsten Alternative: immer den Mittelwert $\bar{y}$ vorhersagen.

Wert	Bedeutung
$R^2 = 1$	perfekte Vorhersage
$R^2 = 0.75$	das Modell erklärt 75 Prozent der Streuung der Zielgröße
$R^2 \approx 0$	nicht besser als der Mittelwert
$R^2 < 0$	schlechter als der Mittelwert

```
from sklearn.metrics import r2_score
print(r2_score(y_true, y_pred))    # -> 0.21 = 1 - 10 / 12.67
```

Regressionskennzahlen im Zusammenhang

```
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

model = LinearRegression().fit(X_train, y_train)    # Hauspreisdaten
y_pred = model.predict(X_test)

mae = mean_absolute_error(y_test, y_pred)
mse = mean_squared_error(y_test, y_pred)
print(f"MAE: {mae:.2f}")           # -> 0.53
print(f"MSE: {mse:.2f}")           # -> 0.53
print(f"RMSE: {np.sqrt(mse):.2f}") # -> 0.73
print(f"R²: {r2_score(y_test, y_pred):.2f}")    # -> 0.60
```

- Zielgröße `MedHouseVal` zählt in 100.000 Dollar: MAE 0.53 heißt rund 53.000 Dollar mittlerer Fehler
- Lesen Sie mehrere Kennzahlen nebeneinander: MAE für den typischen Fehler, RMSE für Ausreißer, R^2 für den Vergleich mit dem Mittelwert
- Ein sehr hohes R^2 im Training und ein niedriges auf neuen Daten zeigt Overfitting

Konfusionsmatrix am Zahlenbeispiel

Zehn Fälle, Klasse 1 = „krank“, Klasse 0 = „gesund“:

```
y_true = [0, 1, 1, 0, 1, 0, 0, 1, 1, 0] # Wahrheit
y_pred = [0, 1, 1, 1, 0, 1, 0, 1, 1, 0] # Vorhersage des Modells
```

	vorhergesagt 0 (gesund)	vorhergesagt 1 (krank)
wirklich 0 (gesund)	TN = 3 richtig entwarnt	FP = 2 Fehllalarm
wirklich 1 (krank)	FN = 1 übersehen	TP = 4 richtig erkannt

- True/False: Lag das Modell richtig? Positive/Negative: Was hat es vorhergesagt?
- Die Diagonale (TN und TP) zählt die richtigen Vorhersagen: 7 von 10

Accuracy, Precision, Recall und F1 von Hand

Aus der Matrix: TP = 4, FP = 2, FN = 1, TN = 3

Kennzahl	Frage	Formel	Rechnung
Accuracy	Wie viele Vorhersagen stimmen?	$\frac{TP+TN}{TP+TN+FP+FN}$	$(4 + 3) / 10 = \mathbf{0.70}$
Precision	Wie viele Alarme sind echt?	$\frac{TP}{TP+FP}$	$4 / 6 = \mathbf{0.67}$
Recall	Wie viele Kranke werden gefunden?	$\frac{TP}{TP+FN}$	$4 / 5 = \mathbf{0.80}$
F1	Mittel aus Precision und Recall	$2 \cdot \frac{Precision \cdot Recall}{Precision + Recall}$	$2 \cdot 0.67 \cdot 0.80 / 1.47 = \mathbf{0.73}$

- Precision schaut auf die Spalte „vorhergesagt 1“, Recall auf die Zeile „wirklich 1“
- F1 ist das harmonische Mittel: Es wird nur hoch, wenn beide Werte hoch sind

Kennzahlen mit scikit-learn

```
from sklearn.metrics import (accuracy_score, confusion_matrix, f1_score,
                             precision_score, recall_score)

cm = confusion_matrix(y_true, y_pred)
print(cm)
# -> [ [3 2]
#      [1 4] ]
tn, fp, fn, tp = cm.ravel()          # Reihenfolge: TN, FP, FN, TP

print(accuracy_score(y_true, y_pred))      # -> 0.7
print(round(precision_score(y_true, y_pred), 2)) # -> 0.67
print(round(recall_score(y_true, y_pred), 2))  # -> 0.8
print(round(f1_score(y_true, y_pred), 2))     # -> 0.73
```

- Reihenfolge der Argumente: erst die Wahrheit, dann die Vorhersage
- Als Bild: `ConfusionMatrixDisplay.from_predictions(y_true, y_pred)`

Welcher Fehler ist teurer?

Anwendung	False Positive (Fehlalarm)	False Negative (übersehen)	Teurer
Krankheitserkennung	gesunde Person geht zur Nachuntersuchung	krankte Person bleibt unbehandelt	meist FN
Betrugserkennung	korrekte Abrechnung wird geprüft, Aufwand und Ärger	Betrug wird ausgezahlt	meist FN, aber zu viele FP legen die Prüfstelle lahm
Spamfilter	wichtige Mail landet im Spamordner	Spam landet im Posteingang	meist FP

- FN teuer: auf **Recall** achten. FP teuer: auf **Precision** achten.
- Beide Fehler lassen sich kaum gleichzeitig auf null bringen. Wer mehr findet, löst mehr Fehllalarme aus.
- Die Gewichtung ist eine fachliche Entscheidung, keine technische

Schwelle verschieben

`predict()` entscheidet bei Wahrscheinlichkeit ≥ 0.5 für Klasse 1. Diese Schwelle können Sie selbst setzen.

```
# model: ein trainierter Klassifikator, zum Beispiel LogisticRegression
proba = model.predict_proba(X_test)[: , 1] # Wahrscheinlichkeit Klasse 1

for schwelle in [0.5, 0.3]:
    y_pred = (proba >= schwelle).astype(int)
    p = precision_score(y_test, y_pred)
    r = recall_score(y_test, y_pred)
    print(f"Schwelle {schwelle}: Precision {p:.2f}, Recall {r:.2f}")
```

Schwelle	Wirkung	Precision	Recall
niedriger (0.3)	mehr Fälle gelten als positiv	sinkt meist	steigt
höher (0.7)	nur sehr sichere Fälle gelten als positiv	steigt meist	sinkt

Die Schwelle auf Validierungsdaten wählen, nicht auf dem Testteil.

ROC von Hand: Beispieldaten

Fünf Fälle mit wahrem Label und vorhergesagter Wahrscheinlichkeit für Klasse 1:

Fall	wahres Label y	Wahrscheinlichkeit p
A	1	0.95
B	0	0.85
C	1	0.60
D	0	0.40
E	1	0.20

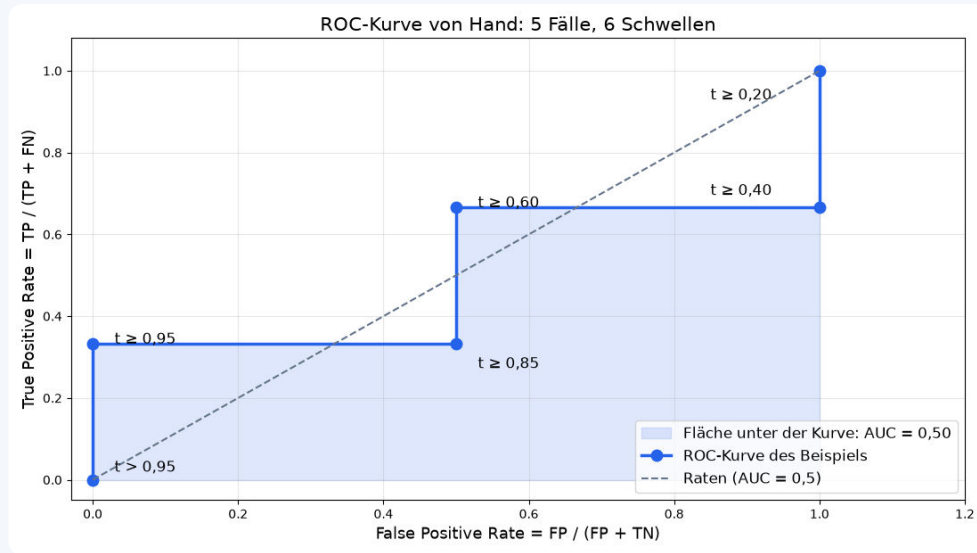
- 3 positive Fälle (A, C, E), 2 negative Fälle (B, D)
- Regel: Für eine Schwelle t gilt ein Fall als positiv, wenn $p \geq t$
- Für jede Schwelle zählen wir TP, FP, FN, TN und rechnen daraus zwei Raten

ROC von Hand: Schwellen durchprobieren

Schwelle t	als positiv vorhergesagt	TP	FP	FN	TN	$TPR = TP / (TP + FN)$	$FPR = FP / (FP + TN)$
> 0.95	keiner	0	0	3	2	0.00	0.00
≥ 0.95	A	1	0	2	2	0.33	0.00
≥ 0.85	A, B	1	1	2	1	0.33	0.50
≥ 0.60	A, B, C	2	1	1	1	0.67	0.50
≥ 0.40	A, B, C, D	2	2	1	0	0.67	1.00
≥ 0.20	A, B, C, D, E	3	2	0	0	1.00	1.00

- **TPR** (True Positive Rate) ist der Recall: Anteil der gefundenen Positiven
- **FPR** (False Positive Rate): Anteil der Negativen, die fälschlich Alarm auslösen
- Jede Zeile ergibt einen Punkt (FPR, TPR) der ROC-Kurve

ROC-Kurve und AUC



- x-Achse: FPR, y-Achse: TPR, ein Punkt je Schwelle
- Ideal: links oben (alle gefunden, kein Fehllalarm)
- Diagonale: Raten
- AUC = Fläche unter der Kurve, 1.0 perfekt, 0.5 Raten
- Von Hand: $0.5 \cdot 0.33 + 0.5 \cdot 0.67 = 0.50$

```
from sklearn.metrics import roc_auc_score
y = [1, 0, 1, 0, 1]
p = [0.95, 0.85, 0.60, 0.40, 0.20]
print(roc_auc_score(y, p))    # -> 0.5
```

Accuracy bei ungleichen Klassen

1000 Personen, 950 gesund, 50 krank. Ein „Modell“ sagt immer „gesund“:

	vorhergesagt gesund	vorhergesagt krank
wirklich gesund	TN = 950	FP = 0
wirklich krank	FN = 50	TP = 0

- $\text{Accuracy} = 950 / 1000 = \mathbf{95 \text{ Prozent}}$
- $\text{Recall} = 0 / 50 = \mathbf{0}$: Kein einziger Kranker wird gefunden
- $\text{F1} = 0$

Bei 95 Prozent Mehrheitsklasse ist eine Accuracy von 95 Prozent der Wert eines Modells, das nichts gelernt hat. Vergleichen Sie jede Accuracy zuerst mit dem Anteil der häufigsten Klasse.

Ungleiche Klassen ohne Zusatzpakete behandeln

```
from sklearn.datasets import make_classification
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report
X, y = make_classification(n_samples=2000, weights=[0.95, 0.05],
                          random_state=1)

# 1. stratifizieren
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, stratify=y, random_state=1)

# 2. gewichten
model = LogisticRegression(class_weight="balanced", max_iter=1000)
model.fit(X_train, y_train)

# 3. passend messen
print(classification_report(y_test, model.predict(X_test)))
```

Modell	Accuracy	Recall Klasse 1
ohne <code>class_weight</code>	0.98	0.73
mit <code>class_weight="balanced"</code>	0.92	0.95

- `class_weight="balanced"` gewichtet Fehler der seltenen Klasse stärker: Accuracy sinkt, Recall steigt von 0.73 auf 0.95
- Dazu: Schwelle anpassen und mit Recall, F1 oder ROC-AUC statt Accuracy vergleichen

classification_report lesen

```
print(classification_report(y_true, y_pred))  # Zehn-Fälle-Beispiel
```

	precision	recall	f1-score	support
0	0.75	0.60	0.67	5
1	0.67	0.80	0.73	5
accuracy			0.70	10
macro avg	0.71	0.70	0.70	10
weighted avg	0.71	0.70	0.70	10

- Eine Zeile je Klasse: Precision, Recall und F1 aus Sicht dieser Klasse. `support` ist die Anzahl der wahren Fälle.
- Zeile `1` enthält die von Hand gerechneten Werte: 0.67, 0.80, 0.73
- `macro avg` : einfacher Mittelwert über die Klassen, jede Klasse zählt gleich. `weighted avg` : nach `support` gewichtet.
- Bei ungleichen Klassen zuerst die Zeile der seltenen Klasse und `macro avg` lesen

Bewertungskriterien für Modelle

Eine gute Kennzahl reicht im Betrieb nicht. Ein Modell muss auch diese Fragen bestehen:

Kriterium	Frage	Kriterium	Frage
Genauigkeit	Stimmen die Vorhersagen?	Validität	Tut das Modell, wofür es gebaut wurde?
Aktualität	Sind die Daten rechtzeitig verfügbar?	Zuverlässigkeit	Läuft es in der Zielumgebung stabil?
Kosten	Was kostet der Dienst?	Sicherheit	Ist das System vor Bedrohungen und schweren Folgen geschützt?
Skalierbarkeit	Verkraftet es wachsende Datenmengen?	Machbarkeit	Lässt es sich praktisch umsetzen?
Zeit	Wie lange dauern Verarbeitung und Antwort?	Nachhaltigkeit	Hält es sein Niveau ohne ständige Updates?
Leistung und Durchsatz	Wie viel Arbeit und wie viele Daten je Zeiteinheit?	Ressourcen und Energie	Wie stark sind Rechner ausgelastet, wie viel Energie wird verbraucht?

Zusammenfassung

- Skalieren für abstands- und gradientenbasierte Verfahren, One-Hot für Kategorien ohne Rangfolge, `np.log1p` für schiefe Größen, neue Merkmale aus Fachwissen
- Erst teilen, dann `fit` nur auf den Trainingsdaten. Der Testteil wird genau einmal benutzt.
- Overfitting zeigt sich am Abstand zwischen Trainings- und Validierungswert. `cross_val_score` mit `StratifiedKFold` liefert Mittelwert und Streuung.
- `GridSearchCV` probiert jede Kombination per Cross-Validation, `RandomizedSearchCV` nur eine Stichprobe
- Regression: MAE, RMSE und R^2 zusammen lesen. Klassifikation: Konfusionsmatrix zuerst, dann Precision, Recall, F1 und AUC. Accuracy mit dem Anteil der häufigsten Klasse vergleichen.

Teil 7: Unüberwachtes Lernen: Clustering und Segmentierung

Gruppen in Daten finden, wenn es keine Zielgröße gibt, und diese Gruppen fachlich beschreiben.

Was Sie in diesem Teil lernen

- Wofür unüberwachtes Lernen da ist und worin es sich von Klassifikation unterscheidet
- Wie k-Means Schritt für Schritt arbeitet und wie Sie es mit scikit-learn einsetzen
- Wie Sie die Zahl der Cluster mit Ellbogen-Kurve und Silhouetten-Wert wählen
- Wie Sie Versichertendaten segmentieren, Profile je Segment bilden und Segmente benennen
- Wo k-Means an Grenzen stößt und wie PCA viele Merkmale in zwei Dimensionen darstellt

Unüberwachtes Lernen: wofür

- Es gibt keine Zielgröße y , nur Eingabedaten X . Das Verfahren sucht selbst nach Struktur.
- **Clustering:** ähnliche Datenpunkte zu Gruppen zusammenfassen
- **Dimensionsreduktion:** viele Merkmale auf wenige verdichten, zum Beispiel für ein Diagramm
- **Anomalieerkennung:** Punkte finden, die zu keiner Gruppe passen
- Typische Anwendungen: Kunden- oder Versichertensegmente, Gruppen ähnlicher Krankheitsverläufe, Produktempfehlungen, auffällige Abrechnungen

Unterschied zur Klassifikation

	Klassifikation	Clustering
Lernart	überwacht	unüberwacht
Zielgröße	bekannt (<code>y</code> mit Labels)	gibt es nicht
Gruppen	vorher festgelegt: „überlebt“, „nicht überlebt“	ergeben sich aus den Daten
Bedeutung der Gruppen	durch die Labels gegeben	müssen Sie nachträglich deuten
Bewertung	Accuracy, Precision, Recall gegen die Wahrheit	keine Wahrheit: Kompaktheit, Trennung, fachliche Prüfung
scikit-learn	<code>fit(X, y)</code> , dann <code>predict(X)</code>	<code>fit(X)</code> oder <code>fit_predict(X)</code>

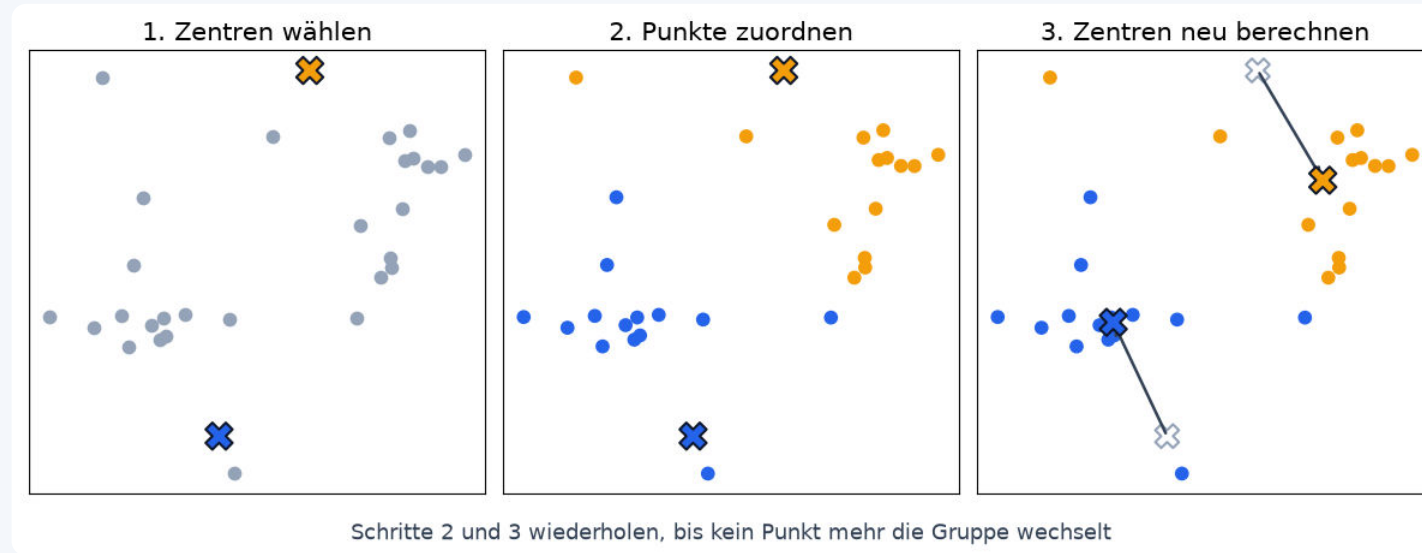
Die Clusternummern 0, 1, 2 sind beliebig. Bei einem neuen Lauf mit anderem Startwert kann dieselbe Gruppe eine andere Nummer tragen.

k-Means Schritt für Schritt

- **1. k festlegen:** Sie geben die Zahl der Cluster vor
- **2. Zentren wählen:** k Startpunkte als Clusterzentren setzen (zufällig)
- **3. Zuordnen:** jeder Punkt kommt zum nächstgelegenen Zentrum
- **4. Neu berechnen:** jedes Zentrum wandert in den Mittelwert seiner Punkte
- **5. Wiederholen:** Schritte 3 und 4, bis sich die Zentren nicht mehr ändern

k-Means verkleinert die Summe der quadrierten Abstände aller Punkte zu ihrem Zentrum.
scikit-learn nennt diesen Wert `inertia_`.

k-Means als Skizze



- Meist stehen die Zentren nach wenigen Runden still: Das Verfahren ist konvergiert
- Ein anderer Start kann zu einem anderen Ergebnis führen. `n_init=10` startet zehnmal und behält den Lauf mit der kleinsten `inertia_`.

KMeans in scikit-learn auf make_blobs

```
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
from sklearn.datasets import make_blobs

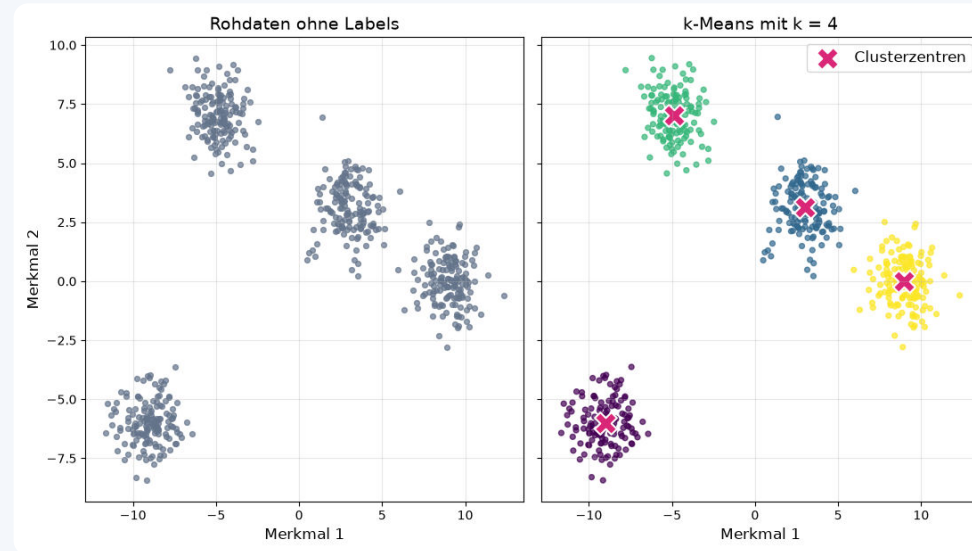
# zweiter Rückgabewert (die wahren Gruppen) wird verworfen
X, _ = make_blobs(n_samples=600, random_state=1,
                  centers=[(-5, 7), (3, 3), (9, 0), (-9, -6)])

kmeans = KMeans(n_clusters=4, n_init=10, random_state=1)
labels = kmeans.fit_predict(X)           # Clusternummer je Punkt
print(kmeans.cluster_centers_.round(1))  # Koordinaten der 4 Zentren
print(round(kmeans.inertia_, 1))        # -> 1161.0

plt.scatter(X[:, 0], X[:, 1], c=labels, s=15)
plt.scatter(kmeans.cluster_centers_[ :, 0], kmeans.cluster_centers_[ :, 1],
            c="red", s=300, marker="X")
plt.show()
```

- `make_blobs` erzeugt Punktwolken zum Ausprobieren, hier um vier fest vorgegebene Zentren
- `fit_predict(X)` : kein `y` , kein Train/Test-Split

Ergebnis: gefundene Cluster und Zentren

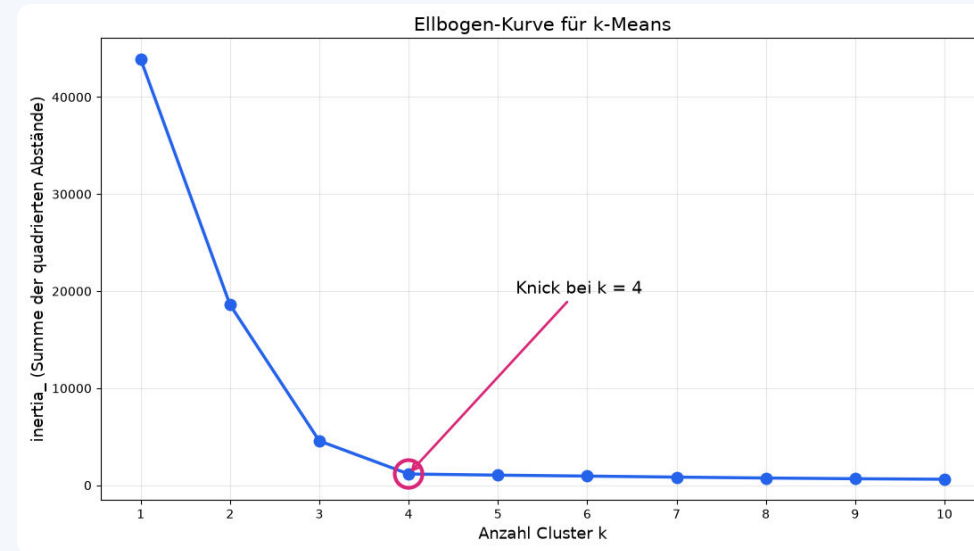


k wählen mit der Ellbogen-Kurve

```
ks = range(1, 11)
inertias = []
for k in ks:
    km = KMeans(n_clusters=k, n_init=10,
                random_state=1)
    km.fit(X)
    inertias.append(km.inertia_)

plt.plot(ks, inertias, marker="o")
plt.xlabel("Anzahl Cluster k")
plt.ylabel("inertia_")
plt.show()
```

- `inertia_` sinkt mit jedem weiteren Cluster, bei $k = \text{Anzahl der Punkte}$ wäre sie 0
- Gesucht ist der Knick: ab dort bringt ein weiteres Cluster nur noch wenig



Silhouetten-Wert

Für jeden Punkt: Wie nah liegt er an seinem eigenen Cluster (a), wie nah am nächsten fremden Cluster (b)?

$$s = \frac{b-a}{\max(a,b)}$$

Wert	Bedeutung
nahe 1	Punkt liegt klar in seinem Cluster
nahe 0	Punkt liegt auf der Grenze zwischen zwei Clustern
negativ	Punkt passt besser in ein anderes Cluster

```
from sklearn.metrics import silhouette_score

for k in range(2, 7):
    labels = KMeans(n_clusters=k, n_init=10, random_state=1).fit_predict(X)
    print(k, round(silhouette_score(X, labels), 2))
# -> 2 0.57 | 3 0.74 | 4 0.79 | 5 0.68 | 6 0.54
```

- `silhouette_score` mittelt über alle Punkte. Der höchste Wert spricht für $k = 4$, passend zum Ellbogen.

Skalieren vor dem Clustern

k-Means rechnet mit Abständen. Ohne Skalierung bestimmt das Merkmal mit den größten Zahlen fast allein das Ergebnis.

```
import pandas as pd
df = pd.read_csv("data/versicherte.csv")
merkmale = ["alter", "bmi", "arztbesuche_jahr", "leistungsausgaben_eur"]
df = df.drop_duplicates().dropna(subset=merkmale) # kein NaN erlaubt
print(df[merkmale].describe().loc[["min", "max", "std"]].round(1))
```

- Zwei Versicherte unterscheiden sich um 10 BMI-Punkte und um 1000 Euro Leistungsausgaben
- Quadrierter Abstand ohne Skalierung: $10^2 + 1000^2 = 100 + 1000000$. Der BMI trägt 0.01 Prozent bei.
- Nach `StandardScaler` hat jedes Merkmal Mittelwert 0 und Standardabweichung 1 und zählt gleich viel

```
from sklearn.preprocessing import StandardScaler
X_scaled = StandardScaler().fit_transform(df[merkmale])
```

Ohne Skalierung segmentiert k-Means die Versicherten praktisch nur nach `leistungsausgaben_eur`.

Segmentierung der Versichertendaten: Cluster bilden

```
from sklearn.cluster import KMeans

# 1. k prüfen: Ellbogen-Kurve auf den skalierten Daten
for k in range(2, 9):
    km = KMeans(n_clusters=k, n_init=10, random_state=1).fit(X_scaled)
    print(k, round(km.inertia_))

# 2. Mit dem gewählten k clustern und die Nummer in die Tabelle schreiben
kmeans = KMeans(n_clusters=4, n_init=10, random_state=1)
df["segment"] = kmeans.fit_predict(X_scaled)

# 3. Wie groß sind die Segmente?
print(df["segment"].value_counts().sort_index())
```

- Geclustert wird auf `X_scaled`, beschrieben wird später auf den Originalwerten in `df`
- Nur Merkmale verwenden, die für die Fragestellung zählen. Eine Versichertennummer oder Postleitzahl hat im Abstand nichts verloren.

Profil je Segment mit `groupby`

```
# Mittelwerte der Originalmerkmale je Segment
profil = df.groupby("segment")[merkmale].mean().round(1)
profil["anzahl"] = df["segment"].value_counts().sort_index()
print(profil)

# Zum Vergleich: der Durchschnitt aller Versicherten
print(df[merkmale].mean().round(1))

# Merkmale, die nicht im Clustering waren, helfen beim Deuten
print(pd.crosstab(df["segment"], df["raucher"], normalize="index").round(2))
print(pd.crosstab(df["segment"], df["zusatzversicherung"],
                  normalize="index").round(2))
```

- Lesen Sie die Profiltabelle zeilenweise: Worin weicht ein Segment am stärksten vom Gesamtdurchschnitt ab?
- Ein Segment mit sehr wenigen Mitgliedern besteht oft aus Ausreißern
- Auch `median()` ansehen: Einzelne sehr hohe Leistungsausgaben verzerren den Mittelwert

Segmenten sprechende Namen geben

```
# Namen aus der Profiltabelle ablesen. Die Nummern vergibt k-Means beliebig,  
# deshalb erst die Tabelle lesen, dann das Wörterbuch schreiben.  
namen = {  
    0: "jung, selten beim Arzt",  
    1: "mittleres Alter, durchschnittliche Ausgaben",  
    2: "älter, viele Arztbesuche",  
    3: "hohe Leistungsausgaben",  
}  
df["segment_name"] = df["segment"].map(namen)  
print(df["segment_name"].value_counts())
```

- Ein guter Name nennt die zwei oder drei Merkmale, in denen sich das Segment vom Durchschnitt abhebt
- Beschreibend bleiben, nicht wertend: „viele Arztbesuche“ statt „Problemfälle“
- Prüfen Sie den Namen an Einzelfällen: `df[df["segment"] == 2].sample(5, random_state=1)`

Die Namen im Code sind Platzhalter. Welche Nummer zu welchem Profil gehört, zeigt erst Ihre Profiltabelle.

Grenzen von k-Means

Grenze	Was passiert	Was Sie tun können
Runde Cluster	k-Means trennt nach Abstand zum Zentrum und findet nur kompakte, etwa kugelförmige Gruppen. Längliche oder ringförmige Gruppen werden zerschnitten.	anderes Verfahren (DBSCAN)
Ausreißer	Ein Extremwert zieht den Mittelwert und damit das Zentrum zu sich oder bekommt ein eigenes Cluster	Ausreißer vorher prüfen, schiefe Größen logarithmieren
k vorgeben	Das Verfahren liefert immer genau k Gruppen, auch wenn es keine gibt	Ellbogen, Silhouette, fachliche Prüfung
Startwerte	Ergebnis hängt vom Start ab	<code>n_init=10</code> , <code>random_state</code> setzen
Nur Zahlen	Mittelwerte und Abstände brauchen numerische, skalierte Merkmale	skalieren, Kategorien zur Beschreibung nutzen

DBSCAN als Alternative

DBSCAN bildet Cluster dort, wo Punkte dicht beieinander liegen. Punkte in dünn besetzten Gegenden gelten als Rauschen.

```
from sklearn.cluster import DBSCAN

X_blobs = StandardScaler().fit_transform(X)      # Blobs von vorhin, skaliert
labels = DBSCAN(eps=0.2, min_samples=5).fit_predict(X_blobs)
print(pd.Series(labels).value_counts().sort_index())
# -> -1: 2 | 0: 150 | 1: 149 | 2: 150 | 3: 149      (-1 = Rauschen)
```

	k-Means	DBSCAN
Zahl der Cluster	geben Sie vor	ergibt sich
Form der Cluster	rund	beliebig
Ausreißer	werden einem Cluster zugeschlagen	bekommen das Label <code>-1</code>
Stellschrauben	<code>n_clusters</code>	<code>eps</code> (Radius der Nachbarschaft), <code>min_samples</code>

- Empfindlich gegenüber `eps` : mit `eps=0.3` verschmelzen hier zwei Wolken zu einem Cluster

PCA zur zweidimensionalen Darstellung

Vier Merkmale lassen sich nicht in ein Streudiagramm zeichnen. PCA (Hauptkomponentenanalyse) bildet neue Achsen, die möglichst viel Streuung der Daten behalten.

```
from sklearn.decomposition import PCA

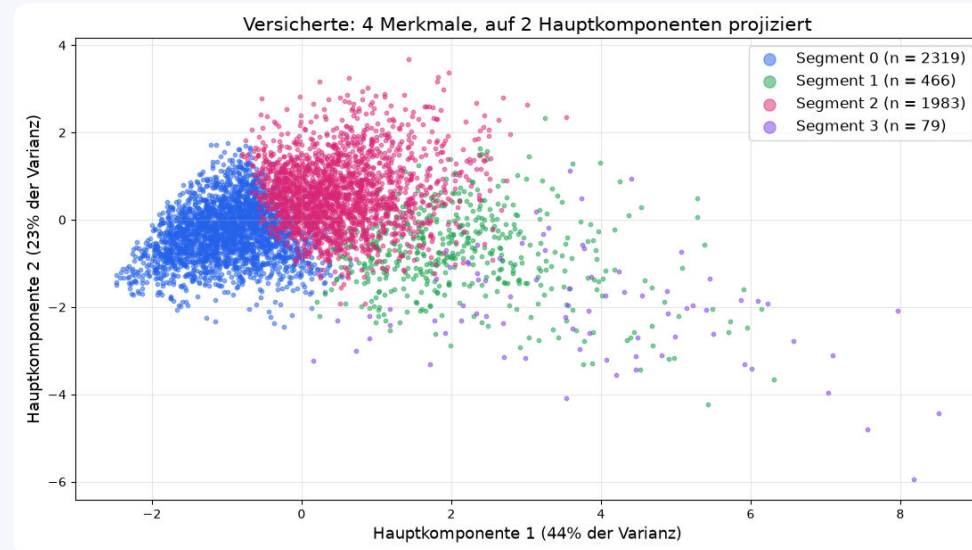
pca = PCA(n_components=2, random_state=1)
X_pca = pca.fit_transform(X_scaled)          # 4 Spalten -> 2 Spalten

anteil = pca.explained_variance_ratio_
print(anteil.round(2), "Summe:", anteil.sum().round(2))

plt.scatter(X_pca[:, 0], X_pca[:, 1], c=df["segment"], s=8, alpha=0.5)
plt.xlabel(f"Hauptkomponente 1 ({anteil[0]:.0%})")
plt.ylabel(f"Hauptkomponente 2 ({anteil[1]:.0%})")
plt.show()
```

- `explained_variance_ratio_` : Anteil der Gesamtstreuung je Hauptkomponente. Die Summe sagt, wie viel Information das 2D-Bild behält.
- Die Hauptkomponenten sind Mischungen der Originalmerkmale und unkorreliert. Vor der PCA die Merkmale standardisieren.

Segmente in der PCA-Ebene



t-SNE als Ausblick

t-SNE (t-distributed Stochastic Neighbor Embedding) ordnet Punkte in 2D so an, dass Nachbarn aus dem hochdimensionalen Raum Nachbarn bleiben.

```
from sklearn.manifold import TSNE

tsne = TSNE(n_components=2, random_state=1)
X_tsne = tsne.fit_transform(X_scaled)

plt.scatter(X_tsne[:, 0], X_tsne[:, 1], c=df["segment"], s=8, alpha=0.5)
plt.show()
```

	PCA	t-SNE
Art	linear	nichtlinear
Achsen	deutbar (Mischung der Merkmale)	ohne Bedeutung
Abstände zwischen Gruppen	aussagekräftig	nicht aussagekräftig
Neue Daten	<code>transform</code> möglich	kein <code>transform</code>
Rechenzeit	kurz	deutlich länger

- t-SNE ist ein Werkzeug zum Ansehen von Daten, nicht zur Vorverarbeitung für Modelle

Zusammenfassung

- Unüberwachtes Lernen arbeitet ohne Zielgröße. Clustering liefert Gruppennummern, die Bedeutung ergänzen Sie.
- k-Means: Zentren wählen, zuordnen, neu berechnen, wiederholen. Vorher skalieren, `n_init` und `random_state` setzen.
- k wählen: Knick in der Ellbogen-Kurve (`inertia_`), höchster `silhouette_score` , fachliche Brauchbarkeit
- Segmente beschreiben: `groupby("segment")` auf den Originalwerten, mit dem Gesamtdurchschnitt vergleichen, beschreibend benennen
- k-Means findet runde Cluster und leidet unter Ausreißern, DBSCAN ist eine Alternative. PCA zeigt viele Merkmale in 2D, `explained_variance_ratio_` sagt, wie viel dabei erhalten bleibt.

Teil 8: Einführung in Deep Learning

Vom einzelnen Neuron zum trainierten Netz in PyTorch, das Kleidungsstücke auf Bildern erkennt.

Was Sie in diesem Teil lernen

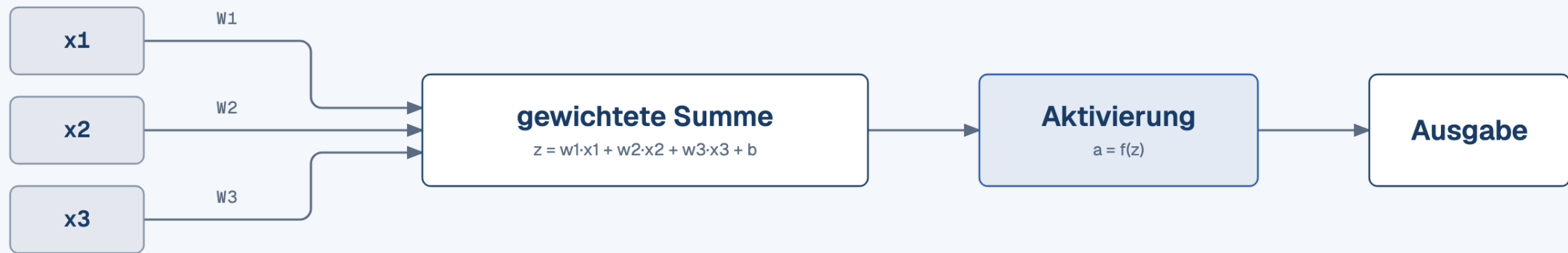
- Wie ein künstliches Neuron rechnet: Gewichte, Bias, Aktivierung
- Wie ein Netz trainiert wird: Vorwärtsslauf, Verlust, Rückwärtsslauf, Schritt des Optimierers
- Wie Sie in PyTorch Tensoren anlegen, ein `nn.Module` schreiben und es auf FashionMNIST trainieren
- Wie Sie die Genauigkeit (Accuracy) auf Testdaten messen und das Modell mit `state_dict` speichern
- Wann sich Deep Learning lohnt und wann ein klassisches Modell die bessere Wahl ist

Vom biologischen zum künstlichen Neuron

Künstliche neuronale Netze sind vom Gehirn inspiriert: Milliarden von Neuronen sind über Synapsen verbunden und tauschen elektrische und chemische Signale aus.

Biologisches Netz	Künstliches Netz
Neuron	Knoten im Netz
Synapse	Verbindung mit einem Gewicht
elektrischer Impuls	Aktivierung , berechnet aus den Eingabewerten
Lernen durch Erfahrung	Anpassen der Gewichte beim Training

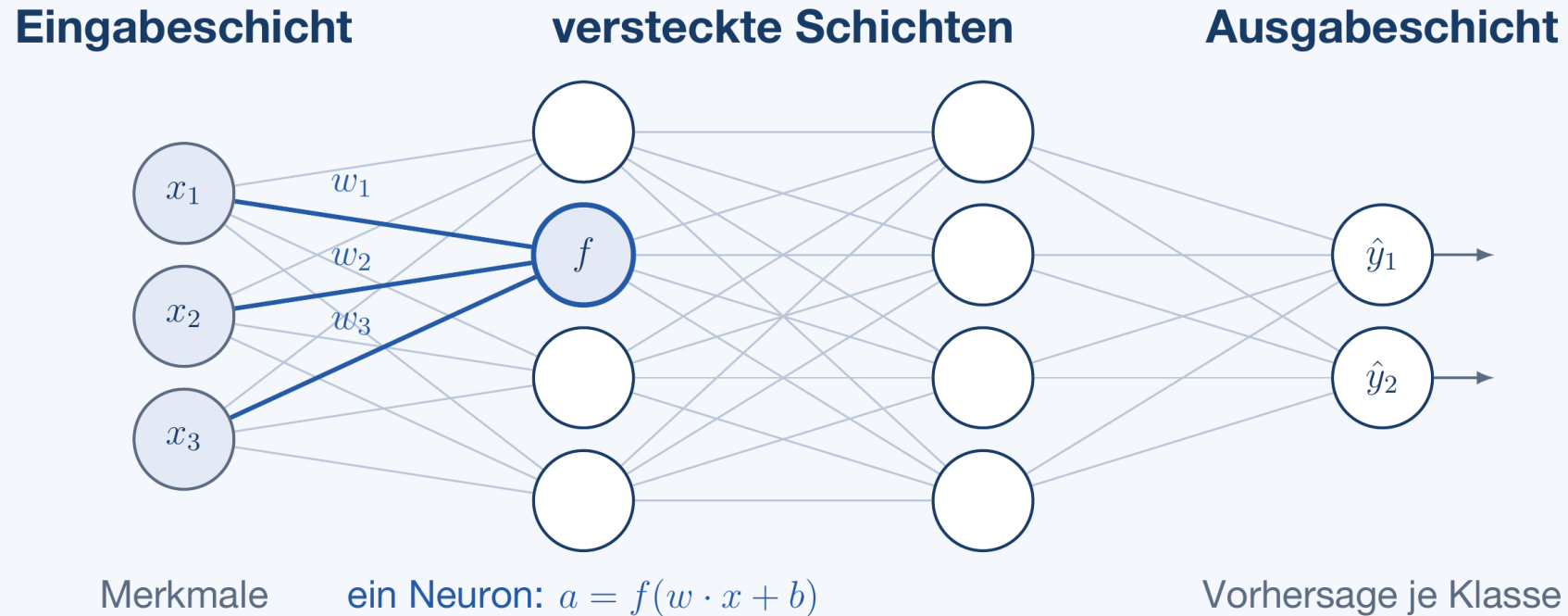
Das künstliche Neuron: Gewichte, Bias, Aktivierung



- **Eingaben** **x** : die Merkmale, zum Beispiel Pixelwerte oder Spalten einer Tabelle
- **Gewichte** **w** : wie stark eine Eingabe weitergegeben wird
- **Bias** **b** : verschiebt die Schwelle, ab der das Neuron anspricht
- **Summation:** **z** ist die gewichtete Summe plus Bias (Präaktivierung)
- **Aktivierung:** eine Funktion **f** macht aus **z** den Ausgabewert **a**

Gewichte und Bias sind die Parameter, die das Training anpasst. Die Eingaben kommen aus den Daten.

Vom Neuron zum Netz



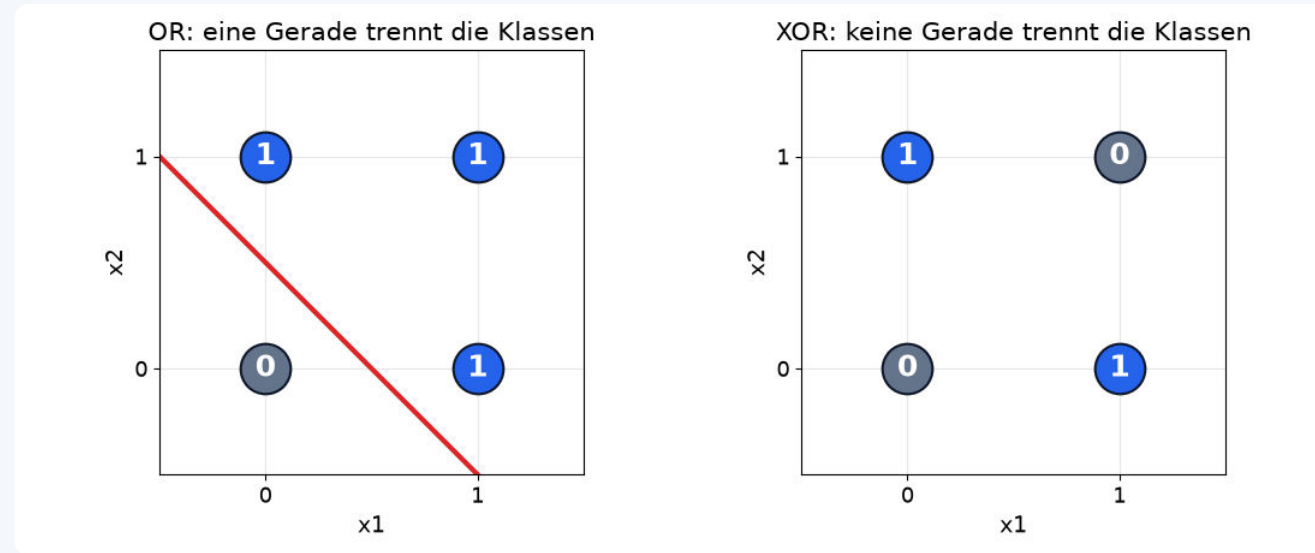
- Die klassische Darstellung: jeder Kreis ist ein Neuron, jede Linie ein Gewicht
- Das blau markierte Neuron ist das von der vorigen Folie: drei Eingaben, drei Gewichte, ein Bias, eine Aktivierung
- Schon dieses kleine Netz hat $3 \cdot 4 + 4 \cdot 4 + 4 \cdot 2 = 36$ Gewichte und 10 Bias-Werte

Schichten: das Feed-Forward-Netz



- Eine **Schicht** nimmt Eingaben entgegen, verarbeitet sie und gibt das Ergebnis weiter
- **Feed Forward**: von der Eingabe über versteckte Schichten zur Ausgabe, ohne Rückkopplung
- **Vollständig verbunden**: jedes Neuron einer Schicht hängt an jedem Neuron der nächsten
- Eine ganze Schicht ist eine Matrixrechnung: $Z = W X + b$
- In den versteckten Schichten entstehen **Merkmale**, die das Netz selbst aus den Daten bildet

Warum Nichtlinearität und Tiefe helfen: XOR



- OR lässt sich mit einer einzigen Geraden trennen, XOR nicht
- Ein Netz aus nur linearen Schichten bleibt insgesamt eine lineare Funktion, egal wie viele Schichten es hat
- Erst die Aktivierungsfunktion zwischen den Schichten bringt Nichtlinearität ins Netz
- Mehr versteckte Schichten bedeuten mehr Tiefe: einfache Muster werden Schicht für Schicht zu komplexen kombiniert

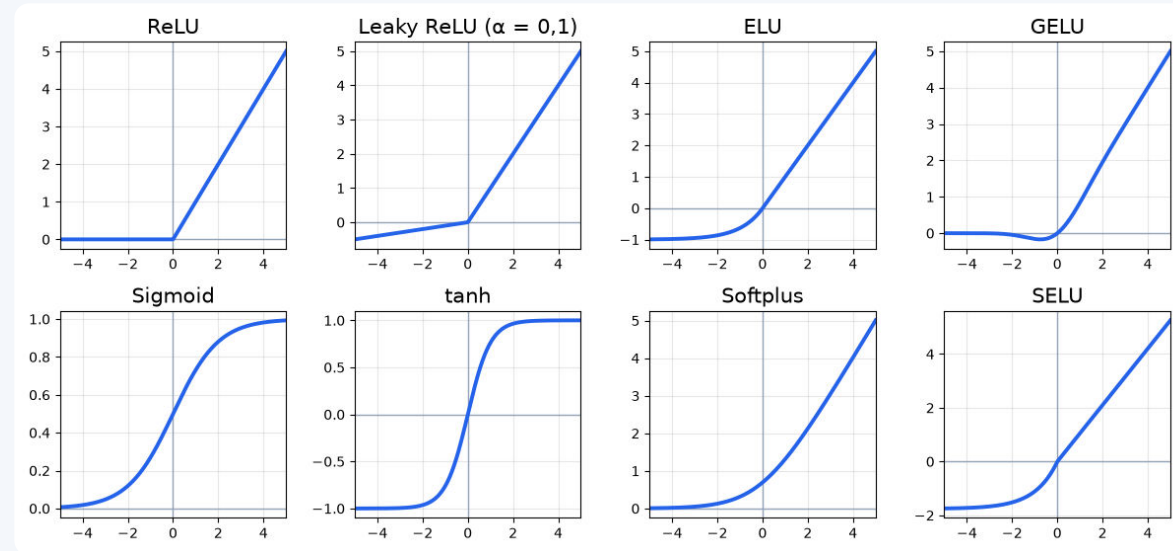
Aktivierungsfunktionen

Funktion	Formel	Wertebereich	Typischer Einsatz
ReLU	<code>max(0, z)</code>	0 bis unendlich	versteckte Schichten, Standardwahl in modernen Netzen
Sigmoid	<code>1 / (1 + exp(-z))</code>	0 bis 1	Ausgabe bei zwei Klassen, liest sich wie eine Wahrscheinlichkeit
Tanh	<code>tanh(z)</code>	-1 bis 1	ähnlich wie Sigmoid, gut für Werte um 0
Softmax	<code>exp(z_i) / Summe aller exp(z_j)</code>	0 bis 1, Summe 1	Ausgabe bei mehreren Klassen

- ReLU setzt negative Werte auf 0: schnell zu rechnen und gut zu trainieren
- Sigmoid in versteckten Schichten gilt als veraltet, weil die Funktion an den Rändern sättigt

`nn.CrossEntropyLoss` in PyTorch rechnet Softmax schon mit ein. Die letzte Schicht gibt deshalb rohe Werte (Logits) aus, ohne eigene Softmax-Schicht.

Aktivierungsfunktionen im Bild



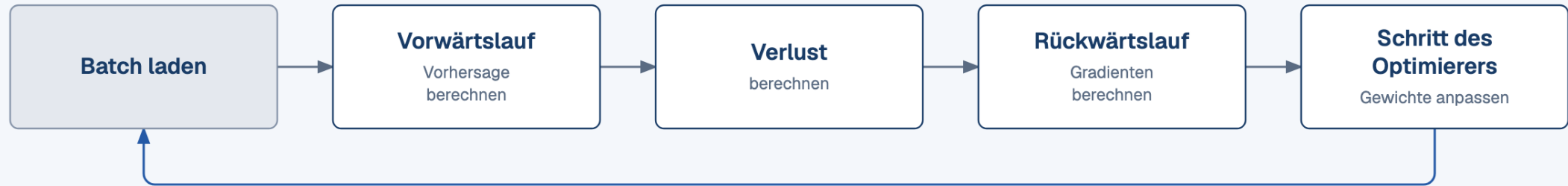
Verlustfunktionen

Die Verlustfunktion misst mit einer einzigen Zahl, wie schlecht die Vorhersagen sind. Das Training macht diese Zahl kleiner.

Verlust	In einem Satz	PyTorch
MSE	Mittelwert der quadrierten Abstände zwischen Vorhersage und wahrem Wert, für Regression	<code>nn.MSELoss()</code>
Cross-Entropy	Misst den Abstand zwischen vorhergesagten Klassenwahrscheinlichkeiten und wahren Labels, für mehrere Klassen	<code>nn.CrossEntropyLoss()</code>
Binary Cross-Entropy	Dasselbe für genau zwei Klassen, passt zu einer Sigmoid-Ausgabe	<code>nn.BCELoss()</code>

Cross-Entropy bestraft große Abweichungen stark: eine falsche Vorhersage mit hoher Sicherheit kostet viel mehr als eine unsichere.

Training als Schleife



- **Vorwärtsslauf:** Eingaben laufen durch das Netz, am Ende steht eine Vorhersage
- **Verlust:** Vergleich von Vorhersage und wahrem Wert
- **Rückwärtsslauf** (Backpropagation): für jedes Gewicht ausrechnen, in welche Richtung es den Verlust verändert
- **Schritt des Optimierers:** jedes Gewicht ein kleines Stück in die Richtung verschieben, die den Verlust senkt

Epoche, Batch, Lernrate und Optimierer

Begriff	Bedeutung	Im Beispiel
Batch	kleine Gruppe von Beispielen, auf der ein Gradient berechnet wird (üblich: 32 oder 64)	<code>batch_size=64</code>
Epoche	ein vollständiger Durchlauf durch alle Trainingsdaten	60000 Bilder = 938 Batches
Lernrate	Schrittweite, mit der die Gewichte angepasst werden	<code>lr=1e-3</code>
SGD	Gradientenabstieg auf Batches mit fester Lernrate	<code>optim.SGD(..., lr=1e-2)</code>
Adam	Weiterentwicklung von SGD, passt die Schrittweite je Gewicht selbst an	<code>optim.Adam(..., lr=1e-3)</code>

- Zu große Lernrate: der Verlust springt oder steigt. Zu kleine Lernrate: das Training kommt kaum voran
- Übliche Startwerte: `1e-2` für SGD, `1e-3` für Adam

Overfitting bei Netzen: Dropout und Early Stopping

Ein Netz mit hunderttausend Gewichten kann die Trainingsdaten auswendig lernen. Das Zeichen dafür kennen Sie schon: der Verlust auf den Trainingsdaten sinkt weiter, der auf den Validierungsdaten steigt wieder.

Dropout

```
self.drop = nn.Dropout(0.3)
```

- schaltet beim Training zufällig 30 % der Neuronen einer Schicht ab
- das Netz kann sich nicht auf einzelne Neuronen verlassen
- bei der Auswertung ist Dropout aus

Early Stopping

- nach jeder Epoche den Verlust auf den Validierungsdaten messen
- die bisher besten Gewichte merken
- abbrechen, wenn sich der Wert mehrere Epochen lang nicht verbessert

`model.train()` vor dem Training, `model.eval()` vor der Auswertung. Nur so ist Dropout zur richtigen Zeit an und aus.

PyTorch: Tensoren anlegen und rechnen

```
import torch

x = torch.tensor([1, 2, 3])          # Tensor = n-dimensionales Array
zufall = torch.rand(2, 3)            # 2 Zeilen, 3 Spalten
print(zufall.shape, zufall.dtype)    # -> torch.Size([2, 3]) torch.float32

ones = torch.ones(2, 3)
print(x + ones)                      # Broadcasting wie in NumPy
# -> tensor([ [2., 3., 4.], [2., 3., 4.] ])

a = torch.tensor([ [1., 2], [3, 4] ])
b = torch.tensor([ [5., 6], [7, 8] ])
print(a @ b)                         # Matrizenmultiplikation -> [ [19., 22.], [43., 50.] ]
print(a * b)                         # elementweise -> [ [ 5., 12.], [21., 32.] ]
print(a.sum(), b.mean())             # -> tensor(10.) tensor(6.5000)
```

Ein Tensor verhält sich wie ein NumPy-Array. Zusätzlich kann er Gradienten mitführen und auf einer Grafikkarte liegen.

Tensoren, NumPy und das Gerät

```
import numpy as np

arr = np.array([ [1.0, 2.0], [3.0, 4.0] ])
t = torch.from_numpy(arr)           # NumPy -> Tensor, teilt den Speicher
print(t.dtype)                     # -> torch.float64
t = t.float()                       # float32, wie es nn.Linear erwartet
zurueck = t.numpy()                 # Tensor -> NumPy

device = torch.device("cpu")        # in diesem Kurs immer die CPU
t = t.to(device)
```

Gerät	Bedeutung
cpu	Prozessor, läuft überall
cuda	NVIDIA-Grafikkarte
mps	Grafikeinheit in Apple-Silicon-Rechnern

NumPy rechnet mit float64, die Schichten in PyTorch mit float32. Ohne `.float()` bricht der Vorwärtsslauf mit einem Typfehler ab.

Autograd an einer kleinen Funktion

```
x = torch.tensor(2.0, requires_grad=True)    # Gradienten mitführen

y = x**2 + 3*x + 5                           # f(x)
y.backward()                                 # Ableitung berechnen

print(x.grad)                                # -> tensor(7.)
```

- Von Hand: $f'(x) = 2*x + 3$, also $f'(2.0) = 7.0$
- `requires_grad=True` : PyTorch merkt sich jede Rechnung, an der `x` beteiligt ist
- `backward()` läuft diese Rechnungen rückwärts ab und legt die Ableitung in `x.grad`
- Genau das passiert im Rückwärtslauf für jedes Gewicht des Netzes

Ein einzelner Trainingsschritt

```
import torch.nn as nn
import torch.optim as optim

torch.manual_seed(42)
netz = nn.Linear(10, 1)           # 10 Eingänge, 1 Ausgang:  $Wx + b$ 
loss_fn = nn.MSELoss()
optimizer = optim.SGD(netz.parameters(), lr=1e-1)

x = torch.rand(10)                # Eingabevektor
y = torch.ones(1)                 # Ziel: das Netz soll 1 ausgeben

loss = loss_fn(netz(x), y)         # Vorwärtsslauf und Verlust
print(f"Verlust vorher: {loss.item():.4f}")
loss.backward()                   # Rückwärtslauf
optimizer.step()                  # Gewichte anpassen
print(f"Verlust nachher: {loss_fn(netz(x), y).item():.4f}")
```

Nach einem einzigen Schritt ist der Verlust kleiner als vorher. Training heißt, diesen Schritt viele tausend Mal zu wiederholen.

FashionMNIST laden

```
from torch.utils.data import DataLoader
from torchvision import datasets
from torchvision.transforms import ToTensor

train_data = datasets.FashionMNIST(root="data", train=True,
                                   download=True, transform=ToTensor())
test_data = datasets.FashionMNIST(root="data", train=False,
                                   download=True, transform=ToTensor())

train_loader = DataLoader(train_data, batch_size=64, shuffle=True)
test_loader = DataLoader(test_data, batch_size=64, shuffle=False)

bilder, labels = next(iter(train_loader))
print(bilder.shape, labels.shape)
# -> torch.Size([64, 1, 28, 28]) torch.Size([64])
```

- Graustufenbilder mit 28 x 28 Pixeln, zehn Produktkategorien (T-Shirt, Hose, Pullover, ...)
- 60000 Bilder zum Training, 10000 zum Testen
- `ToTensor()` macht aus jedem Bild einen Tensor mit Werten zwischen 0 und 1
- Der `DataLoader` liefert die Daten in Batches und mischt die Trainingsdaten in jeder Epoche neu

Ein `nn.Module` mit zwei Linearschichten

```
class Netz(nn.Module):
    def __init__(self):
        super().__init__()
        self.flatten = nn.Flatten()           # 1x28x28 -> 784
        self.layer1 = nn.Linear(28 * 28, 128) # 784 -> 128
        self.layer2 = nn.Linear(128, 10)      # 128 -> 10 Klassen

    def forward(self, x):
        x = self.flatten(x)
        x = torch.relu(self.layer1(x))        #  $x = \text{ReLU}(Wx + b)$ 
        return self.layer2(x)                 # Logits, ohne Softmax

torch.manual_seed(42)
model = Netz()
print(sum(p.numel() for p in model.parameters())) # -> 101770
```

- `__init__` legt die Schichten an, `forward` beschreibt den Vorwärtsslauf
- Sie rufen `forward` nicht direkt auf, sondern `model(bilder)`
- 101770 Parameter: $784 \cdot 128 + 128$ in der ersten, $128 \cdot 10 + 10$ in der zweiten Schicht

Die vollständige Trainingsschleife

```
loss_fn = nn.CrossEntropyLoss()           # mehrere Klassen
optimizer = optim.Adam(model.parameters(), lr=1e-3)
n_epochs = 3

for epoch in range(n_epochs):
    model.train()                          # Trainingsmodus
    running_loss = 0.0
    for bilder, labels in train_loader:
        optimizer.zero_grad()             # alte Gradienten löschen
        outputs = model(bilder)           # Vorwärtsslauf
        loss = loss_fn(outputs, labels)    # Verlust
        loss.backward()                   # Rückwärtslauf
        optimizer.step()                  # Gewichte anpassen
        running_loss += loss.item()
    print(f"Epoche {epoch + 1}/{n_epochs}, "
          f"Verlust: {running_loss / len(train_loader):.4f}")
```

Ohne `optimizer.zero_grad()` addiert PyTorch die Gradienten aller Batches auf, und das Training läuft aus dem Ruder.

Genauigkeit auf den Testdaten

```
model.eval()                                # Auswertungsmodus
correct, total = 0, 0

with torch.no_grad():                        # keine Gradienten nötig
    for bilder, labels in test_loader:
        outputs = model(bilder)
        predicted = outputs.argmax(dim=1)    # Klasse mit dem höchsten Logit
        total += labels.size(0)
        correct += (predicted == labels).sum().item()

print(f"Genauigkeit: {correct / total * 100:.2f} %")
```

- `outputs` hat die Form `[64, 10]` : zehn Werte je Bild, der größte gewinnt
- Vergleichswert: ein untrainiertes Netz rät und liegt bei etwa 10 % (zehn Klassen)

Modell speichern und laden

```
torch.save(model.state_dict(), "model.pth")      # nur die Gewichte

model2 = Netz()                                  # gleiche Architektur
model2.load_state_dict(torch.load("model.pth"))
model2.eval()                                    # bereit für Vorhersagen
```

- `state_dict()` ist ein Wörterbuch: Name der Schicht, dazu der Tensor mit den Gewichten
- Gespeichert werden nur die Zahlen. Die Klasse `Netz` muss beim Laden als Code vorhanden sein
- `model2` liefert danach dieselben Vorhersagen wie `model`

CNN: die Faltung

Ein kleiner Filter (zum Beispiel 3 x 3) wandert über das Bild. An jeder Stelle multipliziert er die Pixelwerte mit den Filterwerten und summiert die Produkte. Das Ergebnis ist eine **Feature Map**.

$$\underbrace{\begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{pmatrix}}_{\text{Bildausschnitt}} \odot \underbrace{\begin{pmatrix} 0 & 1 & 0 \\ 1 & -1 & 1 \\ 0 & 1 & 0 \end{pmatrix}}_{\text{Filter}} \Rightarrow 2 + 4 - 5 + 6 + 8 = 15$$

- Der Filter erkennt ein lokales Muster, zum Beispiel eine Kante
- Derselbe Filter gilt für das ganze Bild: das Muster wird an jeder Position gefunden
- Die Filterwerte sind die Gewichte. Das Netz lernt sie im Training selbst

```
nn.Conv2d(1, 32, kernel_size=3, padding=1)    # 1x28x28 -> 32x28x28
```

CNN: Pooling und der Nutzen für Bilder

$$\underbrace{\begin{pmatrix} 1 & 3 & 2 & 1 \\ 4 & 6 & 5 & 7 \\ 8 & 6 & 9 & 4 \\ 3 & 2 & 4 & 8 \end{pmatrix}}_{\text{Feature Map } 4 \times 4} \xrightarrow{\text{Max-Pooling } 2 \times 2} \begin{pmatrix} 6 & 7 \\ 8 & 9 \end{pmatrix}$$

- **Max-Pooling** übernimmt je Bereich den größten Wert: die Feature Map schrumpft, die stärksten Signale bleiben
- Das spart Rechenaufwand und macht Merkmale robust gegen kleine Verschiebungen
- **Nachbarschaft bleibt erhalten:** eine Linearschicht sieht 784 einzelne Zahlen, eine Faltung sieht Pixel mit ihren Nachbarn
- **Wenige Parameter:** `Conv2d(1, 32, 3)` hat 320 Gewichte, die erste Linearschicht des Netzes von vorhin 100480
- Typischer Aufbau: mehrmals Faltung, ReLU, Pooling, am Ende Linearschichten für die Klassen

Hugging Face datasets

```
from datasets import load_dataset
from torch.utils.data import DataLoader

ds = load_dataset("zalando-datasets/fashion_mnist")
print(ds)                # DatasetDict mit den Teilen "train" und "test"

train = ds["train"].with_format("torch")    # Zugriff liefert Tensoren
beispiel = train[0]
print(beispiel["image"].shape, beispiel["label"])

loader = DataLoader(train, batch_size=64, shuffle=True)
```

- Der Hugging Face Hub ist ein öffentlicher Katalog mit Datensätzen für Bild, Text und Audio
- `load_dataset("<name>")` lädt einen Datensatz einmal herunter und legt ihn im lokalen Cache ab
- `with_format("torch")` sorgt dafür, dass jeder Zugriff Tensoren liefert, die direkt in einen `DataLoader` passen
- Aufteilen geht ohne scikit-learn: `ds["train"].train_test_split(test_size=0.2, seed=42)`

Typische Anwendungsfelder

Daten	Übliche Netze	Beispiele
Bilder	CNN	Bilderkennung, Erkennen von Defekten in der Fertigung
Text und Sprache	rekurrente Netze, heute Transformer	Spam-Erkennung, maschinelle Übersetzung, Spracherkennung, Chatbots
Zeitreihen	rekurrente Netze	Vorhersage von Kursen oder Wetter
Tabellendaten	Feed-Forward-Netze	Bewertung von Kreditrisiken

- Bei Bildern, Audio und Text haben tiefe Netze die großen Durchbrüche gebracht
- Bei Tabellendaten gewinnen Gradient Boosting und Random Forest auf mittelgroßen Datensätzen (rund 10000 Zeilen) meist gegen neuronale Netze, bei deutlich geringerem Rechenaufwand (Grinsztajn, Oyallon, Varoquaux, NeurIPS 2022)

Deep Learning oder klassisches ML

Frage	spricht für klassisches ML	spricht für Deep Learning
Welche Daten liegen vor?	Tabellen mit benannten Spalten	Bilder, Audio, freier Text
Wer bildet die Merkmale?	Sie selbst, aus Fachwissen	das Netz, in den versteckten Schichten
Wie viele Beispiele gibt es?	reicht schon bei eher wenigen Beispielen	zeigt seine Stärke erst bei sehr vielen Beispielen
Muss das Modell erklärbar sein?	Koeffizienten und Bäume lassen sich lesen	101770 Gewichte lassen sich nicht einzeln deuten
Wie viel Rechenzeit ist vorhanden?	Training auf dem Laptop	große Netze brauchen eine Grafikkarte

Beginnen Sie mit einem einfachen Modell aus scikit-learn als Vergleichswert. Ein Netz muss diesen Wert erst schlagen, bevor sich der Mehraufwand lohnt.

Ausblick: Transformer und Sprachmodelle

- Große Sprachmodelle (LLMs, zum Beispiel GPT und BERT) sind tiefe neuronale Netze
- Ihre Architektur heißt **Transformer**: der Attention-Mechanismus ersetzt die rekurrenten Schichten
- Trainiert werden sie mit derselben Schleife: Vorwärtsslauf, Verlust, Rückwärtsslauf, Schritt des Optimierers
- Anwendungen: Chatbots, Textassistenten, automatische Übersetzung

Literatur: Vaswani et al.: Attention Is All You Need (2017). Goodfellow, Bengio, Courville: Deep Learning (MIT Press, 2016). Zhang, Lipton, Li, Smola: Dive into Deep Learning (d2l.ai).

Zusammenfassung

- Ein Neuron rechnet $z = w*x + b$ und wendet darauf eine Aktivierungsfunktion an. Ohne Aktivierung bliebe jedes Netz linear
- Training ist eine Schleife aus vier Schritten: `model(x)`, `loss_fn(...)`, `loss.backward()`, `optimizer.step()`, davor `optimizer.zero_grad()`
- Epochenzahl, Batchgröße und Lernrate steuern das Training. Dropout und Early Stopping bremsen Overfitting
- In PyTorch beschreiben Sie das Netz als `nn.Module`, laden Daten mit dem `DataLoader` und speichern die Gewichte mit `state_dict`
- Deep Learning spielt seine Stärke bei Bildern, Text und Audio aus. Für Tabellen bleiben die klassischen Modelle der Startpunkt

Teil 9: Der ML-Workflow mit scikit-learn

Aufbereitung, Modell und Bewertung in einem einzigen Objekt zusammenbauen, prüfen, speichern.

Was Sie in diesem Teil lernen

- Sie erkennen in scikit-learn-Klassen dieselben drei Methoden: `fit` , `transform` , `predict`
- Sie wissen, bei welchen Schritten der Aufbereitung ein Datenleck entsteht
- Sie bauen aus Aufbereitung und Modell eine `Pipeline` und behandeln Zahlen- und Textspalten mit `ColumnTransformer` getrennt
- Sie prüfen die ganze Pipeline mit Cross-Validation und stellen sie mit `GridSearchCV` ein
- Sie speichern die fertige Pipeline und halten fest, womit sie entstanden ist

Die drei Rollen in scikit-learn

Rolle	Methoden	Was passiert	Beispiele
Estimator	<code>fit(X, y)</code>	lernt etwas aus Daten und merkt es sich	jede Klasse, die aus Daten lernt
Transformer	<code>fit</code> , <code>transform</code> , <code>fit_transform</code>	lernt Kennwerte und rechnet Daten damit um	<code>StandardScaler</code> , <code>OneHotEncoder</code> , <code>SimpleImputer</code> , <code>PCA</code>
Predictor	<code>fit</code> , <code>predict</code> , <code>score</code>	lernt ein Modell und sagt damit vorher	<code>LogisticRegression</code> , <code>RandomForestClassifier</code> , <code>KMeans</code>

- Gelernte Größen enden auf einen Unterstrich: `scaler.mean_`, `modell.coef_`, `kmeans.cluster_centers_`
- Einstellungen, die Sie selbst setzen, stehen im Konstruktor: `max_depth=5`, `strategy="median"`
- Weil diese Klassen dieselben Methoden haben, lassen sie sich hintereinanderstecken

Bekannt aus Teil 5: teilen, dann skalieren

```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression

X, y = load_breast_cancer(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=1, stratify=y)

scaler = StandardScaler()
X_train_s = scaler.fit_transform(X_train)  # fit NUR auf Training
X_test_s = scaler.transform(X_test)        # Testdaten nur umrechnen

modell = LogisticRegression(max_iter=1000)
modell.fit(X_train_s, y_train)
print(modell.score(X_test_s, y_test))
```

Merkregel: fit und fit_transform sehen nur Trainingsdaten. Testdaten bekommen ausschließlich transform und predict.

Was beim Leck durchsickert

```
import pandas as pd
df = pd.read_csv("data/titanic.csv")
X_num = df[ ["Age", "Fare"] ].fillna(df[ ["Age", "Fare"] ].median())

X_tr, X_te = train_test_split(X_num, test_size=0.2, random_state=1)
print(StandardScaler().fit(X_num).mean_) # -> [29.36 32.20] alle Zeilen
print(StandardScaler().fit(X_tr).mean_)   # -> [29.73 31.95] nur Training
```

Schritt	Was aus den Testdaten ins Training gelangt
Skalieren vor dem Split	Mittelwert und Standardabweichung
Fehlende Werte vor dem Split füllen	Median oder häufigster Wert
Encoding vor dem Split	Kategorien, die nur in den Testdaten vorkommen
Ausreißer vor dem Split entfernen	Quartile und Grenzen

Auch `fillna(df.median())` in Zeile 3 ist ein Leck: Der Median stammt aus allen Zeilen.

Wo die Handarbeit an Grenzen stößt

- Jeder weitere Schritt (fehlende Werte füllen, Encoding, Skalierung) braucht wieder ein eigenes `fit` auf Training und `transform` auf Test
- Bei Cross-Validation müssten Sie das in jedem der fünf Durchgänge neu tun
- Für neue Daten im Einsatz müssen alle Schritte in derselben Reihenfolge noch einmal laufen
- Beim Speichern brauchen Sie Scaler, Encoder und Modell als getrennte Dateien

Eine Pipeline fasst alle Schritte in einem Objekt mit einem `fit` und einem `predict` zusammen.

Pipeline mit zwei Schritten

```
from sklearn.pipeline import Pipeline

pipe = Pipeline([
    ("scaler", StandardScaler()),
    ("modell", LogisticRegression(max_iter=1000)),
])

pipe.fit(X_train, y_train)          # rohe Trainingsdaten hinein
print(pipe.score(X_test, y_test))  # rohe Testdaten hinein
```

- Jeder Schritt ist ein Paar aus frei gewähltem Namen und Objekt
- Alle Schritte außer dem letzten müssen Transformer sein
- Der letzte Schritt ist meist ein Predictor

Was `fit` und `predict` in der Pipeline auslösen

```
pipe.fit(X_train, y_train)
    scaler.fit_transform(X_train)    -> X_train_s
    modell.fit(X_train_s, y_train)

pipe.predict(X_test)
    scaler.transform(X_test)         -> X_test_s      (kein fit!)
    modell.predict(X_test_s)        -> y_pred
```

- Bei `fit` ruft die Pipeline für jeden Transformer `fit_transform` auf und reicht das Ergebnis weiter
- Bei `predict` und `score` ruft sie nur `transform` auf
- Die Merkgel von eben ist damit fest eingebaut

Auf einzelne Schritte zugreifen

```
pipe.named_steps["scaler"].mean_[:2]    # gelernte Größe des Scalers
pipe.named_steps["modell"].coef_        # Koeffizienten des Modells
pipe[:-1].transform(X_test)              # Daten nach der Aufbereitung

pipe.get_params()["modell__C"]            # -> 1.0
pipe.set_params(modell__C=0.1)           # Einstellung ändern
```

Parameter eines Schritts sprechen Sie mit `schrittname__parameter` an, mit doppeltem Unterstrich.

Titanic: Spalten wählen

```
import pandas as pd

df = pd.read_csv("data/titanic.csv")

num_cols = ["Age", "Fare", "SibSp", "Parch"]
cat_cols = ["Pclass", "Sex", "Embarked"]

X = df[num_cols + cat_cols]
y = df["Survived"]
print(X.isna().sum())    # -> Age 177, Embarked 2, sonst 0
```

Spalte	Entscheidung
PassengerId , Name , Ticket	weglassen: Kennungen ohne verallgemeinerbaren Inhalt
Cabin	weglassen: bei den meisten Zeilen leer
Pclass	als Kategorie behandeln: 1, 2, 3 sind Klassen, keine Messwerte
Age , Embarked	behalten, fehlende Werte füllt die Pipeline

Split mit `stratify`

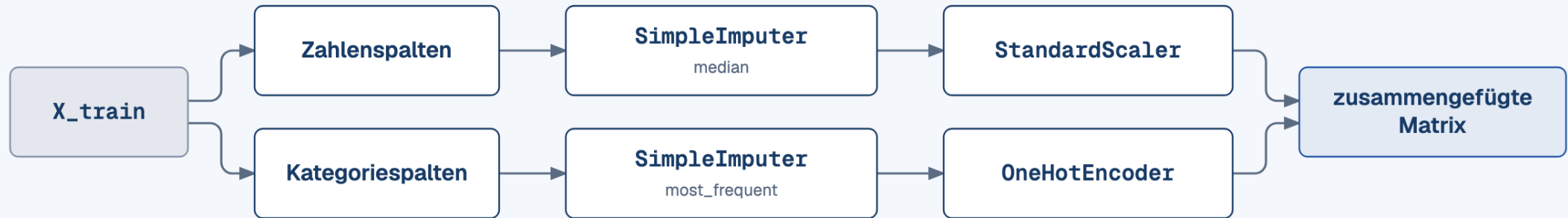
```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=1, stratify=y)

print(X_train.shape, X_test.shape)    # -> (712, 7) (179, 7)
print(y_train.mean(), y_test.mean()) # -> 0.383 0.385
```

- `stratify=y` hält den Anteil der Überlebenden in beiden Teilen gleich
- `random_state=1` liefert bei jedem Lauf dieselbe Aufteilung
- Die Testdaten legen Sie jetzt zur Seite, bis das Modell fertig eingestellt ist

Aufbereitung je Spaltengruppe mit ColumnTransformer



- Zahlen und Kategorien brauchen verschiedene Aufbereitung
- **ColumnTransformer** wendet auf jede Spaltengruppe einen eigenen Transformer an und klebt die Ergebnisse nebeneinander

Zwei kleine Pipelines für zwei Spaltengruppen

```
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import OneHotEncoder, StandardScaler

numeric_pipe = Pipeline([
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler()),
])

categorical_pipe = Pipeline([
    ("imputer", SimpleImputer(strategy="most_frequent")),
    ("encoder", OneHotEncoder(handle_unknown="ignore")),
])
```

- Zahlen: fehlende Werte mit dem Median füllen, danach standardisieren
- Kategorien: fehlende Werte mit dem häufigsten Wert füllen, danach One-Hot-Encoding

ColumnTransformer zusammensetzen

```
from sklearn.compose import ColumnTransformer

preprocess = ColumnTransformer([
    ("num", numeric_pipe, num_cols),
    ("cat", categorical_pipe, cat_cols),
])

Xt = preprocess.fit_transform(X_train)
print(Xt.shape)                # -> (712, 12)
print(preprocess.get_feature_names_out())
```

```
['num__Age' 'num__Fare' 'num__SibSp' 'num__Parch' 'cat__Pclass_1'
 'cat__Pclass_2' 'cat__Pclass_3' 'cat__Sex_female' 'cat__Sex_male'
 'cat__Embarked_C' 'cat__Embarked_Q' 'cat__Embarked_S']
```

handle_unknown="ignore" und unbekannte Kategorien

```
hafen = X_train[ ["Embarked"] ].dropna()
neu = pd.DataFrame({"Embarked": ["X"]})    # Hafen fehlt im Training

streng = OneHotEncoder()                  # Standard: "error"
streng.fit(hafen)
streng.transform(neu)                      # -> ValueError

tolerant = OneHotEncoder(handle_unknown="ignore")
tolerant.fit(hafen)
print(tolerant.transform(neu).toarray())  # -> [ [0. 0. 0.] ]
```

- Im Einsatz tauchen Kategorien auf, die das Training nie gesehen hat
- Mit `"ignore"` bekommt eine solche Zeile in allen Spalten dieser Kategorie eine 0
- Das Modell rechnet weiter, statt mit einer Fehlermeldung abzuberechnen

Vollständige Pipeline mit logistischer Regression

```
from sklearn.linear_model import LogisticRegression

clf = Pipeline([
    ("prep", preprocess),
    ("modell", LogisticRegression(max_iter=1000)),
])

clf.fit(X_train, y_train)
```

- `X_train` geht als DataFrame mit fehlenden Werten und Text hinein
- `prep` füllt, skaliert und kodiert, `modell` lernt auf dem Ergebnis
- Ein einziges Objekt `clf` enthält jetzt die gesamte Verarbeitung

Modell tauschen: Random Forest

```
from sklearn.ensemble import RandomForestClassifier

rf = Pipeline([
    ("prep", preprocess),
    ("modell", RandomForestClassifier(n_estimators=200, random_state=1)),
])

rf.fit(X_train, y_train)
```

- Die Aufbereitung bleibt unverändert, nur der letzte Schritt wechselt
- Bäume brauchen keine Skalierung, sie schadet ihnen aber auch nicht
- `random_state=1` im Modell, weil der Random Forest selbst Zufall benutzt

Cross-Validation der ganzen Pipeline

```
from sklearn.model_selection import cross_val_score

for name, pipe in [("LogReg", clf), ("RandomForest", rf)]:
    scores = cross_val_score(pipe, X_train, y_train,
                             cv=5, scoring="accuracy")
    print(f"{name}: {scores.mean():.3f} +/- {scores.std():.3f}")

# -> LogReg: 0.795 +/- 0.018
# -> RandomForest: 0.780 +/- 0.031
```

In jedem der fünf Durchgänge lernt die Pipeline Median, Skalierung und Kategorien neu, nur aus dem jeweiligen Trainingsteil.

GridSearchCV über Pipeline-Parameter

```
from sklearn.model_selection import GridSearchCV

param_grid = {
    "modell__max_depth": [3, 5, 8, None],
    "modell__min_samples_leaf": [1, 3, 5],
}

search = GridSearchCV(rf, param_grid, cv=5, scoring="accuracy", n_jobs=-1)
search.fit(X_train, y_train)

print(search.best_params_)
# -> {'modell__max_depth': 5, 'modell__min_samples_leaf': 1}
print(round(search.best_score_, 3))    # -> 0.81
```

- `modell__max_depth` heißt: Parameter `max_depth` im Schritt `modell`
- 4 mal 3 Kombinationen mal 5 Durchgänge ergeben 60 Trainingsläufe

Ergebnisse der Suche lesen

```
ergebnis = pd.DataFrame(search.cv_results_)
spalten = ["param_modell__max_depth", "param_modell__min_samples_leaf",
           "mean_test_score", "std_test_score", "rank_test_score"]
print(ergebnis[spalten].sort_values("rank_test_score").head())

best = search.best_estimator_      # fertige, neu trainierte Pipeline
```

- `mean_test_score` ist der Mittelwert über die fünf Validierungsteile, nicht das Testset
- Liegen die besten Zeilen enger beieinander als `std_test_score`, wählen Sie das einfachere Modell
- `best_estimator_` ist die Pipeline mit den besten Einstellungen, trainiert auf allen Trainingsdaten

Auswertung auf dem Testset

```
from sklearn.metrics import classification_report, confusion_matrix

y_pred = best.predict(X_test)
print(confusion_matrix(y_test, y_pred))
print(classification_report(y_test, y_pred, digits=2))
```

```
[ [101  9]
  [ 19 50] ]
```

	precision	recall	f1-score	support
0	0.84	0.92	0.88	110
1	0.85	0.72	0.78	69
accuracy			0.84	179

Das Testset benutzen Sie genau einmal, ganz am Ende. Wer danach weiter einstellt und erneut misst, hat kein Testset mehr.

Pipeline speichern und laden

```
import joblib

joblib.dump(best, "titanic_pipeline.joblib")

geladen = joblib.load("titanic_pipeline.joblib")
print(geladen.score(X_test, y_test))  # -> 0.84, wie vorher
```

- Eine Datei enthält Imputer, Scaler, Encoder und Modell mit allen gelernten Größen
- Aufbereitung und Modell können im Einsatz nicht mehr auseinanderlaufen
- Zum Laden brauchen Sie dieselbe scikit-learn-Version wie beim Speichern: Laden mit einer anderen Version wird nicht unterstützt

Laden Sie .joblib-Dateien nur aus vertrauenswürdiger Quelle: Beim Laden kann beliebiger Code ausgeführt werden.

Vorhersage für neue Daten

```
neu = pd.DataFrame([{\n    "Age": 30, "Fare": 50.0, "SibSp": 0, "Parch": 0,\n    "Pclass": 1, "Sex": "female", "Embarked": "S",\n}])\n\nprint(geladen.predict(neu))          # -> [1]\nprint(geladen.predict_proba(neu).round(2))  # -> [ [0.06 0.94] ]
```

- Neue Daten kommen als DataFrame mit denselben Spaltennamen wie im Training
- Fehlende Werte dürfen vorkommen, die Pipeline füllt sie mit den gelernten Werten
- Die Reihenfolge der Spalten ist egal, `ColumnTransformer` wählt nach Namen aus

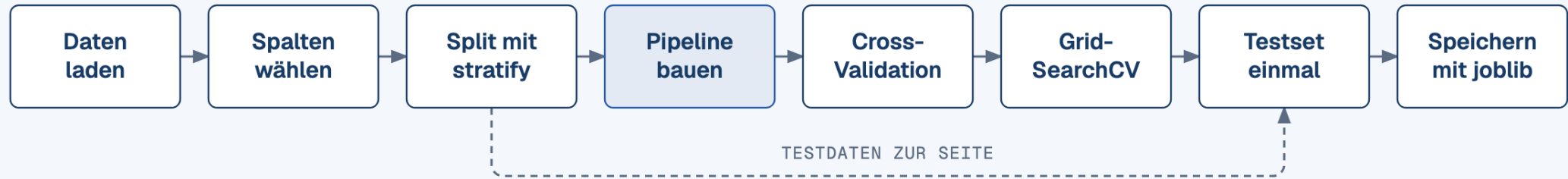
Reproduzierbarkeit

```
import sys, sklearn, pandas, numpy
print(sys.version.split()[0], sklearn.__version__,
      pandas.__version__, numpy.__version__)
```

- `random_state=1` an jeder Stelle mit Zufall: `train_test_split`, `RandomForestClassifier`, `KMeans`, `KFold(shuffle=True)`
- Versionen von Python, scikit-learn, pandas und numpy zusammen mit dem Modell notieren
- Rohdaten unverändert lassen und die Datei benennen, mit der trainiert wurde (Name, Datum, Zeilenzahl)
- Das ganze Notebook von oben nach unten durchlaufen lassen, bevor Sie Ergebnisse weitergeben

Test für Reproduzierbarkeit: Kernel neu starten, alles ausführen, dieselben Zahlen erhalten.

Der Workflow im Überblick



- Alles zwischen Split und Testset läuft nur auf den Trainingsdaten
- Die Pipeline ist das Objekt, das durch alle Schritte wandert

Checkliste: minimaler reproduzierbarer Workflow

- Zielgröße und Merkmalsspalten ausdrücklich benennen, Kennungen weglassen
- `train_test_split` mit `random_state` und bei Klassifikation mit `stratify=y`, vor jeder Aufbereitung
- Imputation, Skalierung und Encoding stehen in der Pipeline, nicht davor
- `OneHotEncoder(handle_unknown="ignore")` für Kategorien
- Modellvergleich und Einstellung nur über Cross-Validation auf den Trainingsdaten
- Kennzahl passend zur Fragestellung wählen, bei ungleichen Klassen nicht Accuracy
- Testset genau einmal auswerten
- Pipeline mit `joblib.dump` speichern, Versionen und Datenstand dazuschreiben

Häufige Fehler im Workflow

Fehler	Folge
Scaler oder Encoder auf allen Daten anpassen, dann teilen	Datenleck, zu gute Zahlen
Zeilen mit fehlender Zielgröße löschen, ohne nach dem Grund zu fragen	verzerrte Stichprobe
Automatisch erkannten Datentypen vertrauen (Datum, Ja/Nein)	falsche Aufbereitung
Accuracy bei ungleichen Klassen	das Modell übersieht die seltene Klasse
Kategorien mit sehr vielen Ausprägungen ungeprüft one-hot-kodieren	tausende fast leere Spalten
Aufbereitung nicht mit dem Modell versionieren	Modell lässt sich nicht nachvollziehen
Zufälliger Split bei zeitlich geordneten Daten	das Modell lernt aus der Zukunft

Zusammenfassung

- Transformer und Modelle teilen sich `fit`, `transform` und `predict`, deshalb lassen sie sich zu einer `Pipeline` verketteten
- Ein Datenleck entsteht, sobald `fit` Testdaten sieht. Die Pipeline verhindert das, auch in jedem Durchgang der Cross-Validation
- `ColumnTransformer` gibt Zahlen- und Kategoriespalten getrennte Aufbereitung, `handle_unknown="ignore"` fängt neue Kategorien ab
- `GridSearchCV` stellt Pipeline-Parameter über `schritt_parameter` ein, das Testset kommt genau einmal am Ende
- Eine `.joblib`-Datei mit notierten Versionen und `random_state` macht das Ergebnis wiederholbar

Teil 10: Praxisbeispiele und Interpretation von Modellen

Drei Fragestellungen mit demselben Workflow lösen und danach prüfen, was die Modelle gelernt haben.

Was Sie in diesem Teil lernen

- Sie übertragen den Pipeline-Workflow auf Klassifikation, Regression und Clustering
- Sie bewerten ein Modell bei stark ungleichen Klassen mit Recall und Precision statt Accuracy
- Sie lesen Koeffizienten, einen Entscheidungsbaum und Merkmalswichtigkeiten
- Sie kennen die Schwächen von `feature_importances_` und setzen `permutation_importance` ein
- Sie wissen, was eine Interpretation aussagt, was nicht, und was in einen Ergebnisbericht gehört

Drei Beispiele, ein Ablauf

	Klassifikation	Regression	Clustering
Frage	Fällt die Maschine aus?	Wie hoch ist der Hauswert?	Welche Gruppen von Versicherten gibt es?
Datei	<code>data/ai4i2020.csv</code>	<code>data/california_housing.csv</code>	<code>data/versicherte.csv</code>
Zielgröße	<code>Machine failure</code> (0/1)	<code>MedHouseVal</code>	keine
Besonderheit	nur 3,4 % Ausfälle	Deckelung der Zielgröße	Skalierung entscheidet über das Ergebnis
Kennzahl	Recall, Precision	RMSE, MAE, R^2	Segmentprofil, fachliche Lesbarkeit
Letzter Schritt der Pipeline	<code>RandomForestClassifier</code>	<code>LinearRegression</code>	<code>KMeans</code>

Klassifikation: Maschinenausfall vorhersagen

```
import pandas as pd

df = pd.read_csv("data/ai4i2020.csv")
print(df.shape)                # -> (10000, 14)
print(df["Machine failure"].value_counts())  # -> 0: 9661, 1: 339
```

Spalte	Bedeutung
Type	Produktvariante L, M oder H
Air temperature [K] , Process temperature [K]	Umgebungs- und Prozesstemperatur
Rotational speed [rpm] , Torque [Nm]	Drehzahl und Drehmoment
Tool wear [min]	aufgelaufener Werkzeugverschleiß
Machine failure	Zielgröße: Ausfall ja oder nein
TWF , HDF , PWF , OSF , RNF	Art des Ausfalls (Verschleiß, Wärme, Leistung, Überlast, Zufall)

Spalten wählen, ohne die Antwort mitzugeben

```
from sklearn.model_selection import train_test_split
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder, StandardScaler

num_cols = ["Air temperature [K]", "Process temperature [K]",
            "Rotational speed [rpm]", "Torque [Nm]", "Tool wear [min]"]
cat_cols = ["Type"]
X = df[num_cols + cat_cols]          # ohne UDI, Product ID, TWF ... RNF
y = df["Machine failure"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=1, stratify=y)

prep = ColumnTransformer([
    ("num", StandardScaler(), num_cols),
    ("cat", OneHotEncoder(handle_unknown="ignore"), cat_cols),
])
```

TWF bis RNF stehen erst fest, wenn der Ausfall schon passiert ist. Mit diesen Spalten erreicht ein Modell 99,9 % Accuracy und ist im Einsatz wertlos.

Baseline: hohe Accuracy, kaum Recall

```
from sklearn.pipeline import Pipeline
from sklearn.model_selection import cross_validate
from sklearn.dummy import DummyClassifier
from sklearn.linear_model import LogisticRegression

def bewerte(modell):
    pipe = Pipeline([("prep", prep), ("modell", modell)])
    cv = cross_validate(pipe, X_train, y_train, cv=5,
                        scoring=["accuracy", "recall", "precision"])
    return {k[5:]: round(float(v.mean()), 2)
            for k, v in cv.items() if k.startswith("test_")}

print(bewerte(DummyClassifier(strategy="most_frequent")))
# -> {'accuracy': 0.97, 'recall': 0.0, 'precision': 0.0}
print(bewerte(LogisticRegression(max_iter=1000)))
# -> {'accuracy': 0.97, 'recall': 0.18, 'precision': 0.74}
```

- Das Modell, das nie einen Ausfall meldet, erreicht 97 % Accuracy
- Die logistische Regression findet nur 18 von 100 Ausfällen

`class_weight="balanced"`: seltene Klasse stärker gewichten

```
from sklearn.ensemble import RandomForestClassifier

print(bewerte(LogisticRegression(max_iter=1000, class_weight="balanced")))
print(bewerte(RandomForestClassifier(n_estimators=200, random_state=1)))
print(bewerte(RandomForestClassifier(n_estimators=200, min_samples_leaf=5,
                                     class_weight="balanced",
                                     random_state=1)))
```

Modell	Accuracy	Recall	Precision
immer „kein Ausfall“	0,97	0,00	0,00
LogisticRegression	0,97	0,18	0,74
LogisticRegression, <code>balanced</code>	0,81	0,80	0,13
RandomForest	0,98	0,53	0,90
RandomForest, <code>balanced</code> , <code>min_samples_leaf=5</code>	0,96	0,85	0,43

- `balanced` gewichtet Fehler auf der seltenen Klasse umgekehrt zu ihrer Häufigkeit
- Recall steigt, Precision sinkt: mehr gefundene Ausfälle, mehr Fehllalarme

Schwellenwert festlegen und auf dem Testset prüfen

```
from sklearn.model_selection import cross_val_predict
from sklearn.metrics import classification_report, confusion_matrix

rf = Pipeline([("prep", prep), ("modell", RandomForestClassifier(
    n_estimators=200, min_samples_leaf=5, class_weight="balanced",
    random_state=1))])

proba = cross_val_predict(rf, X_train, y_train, cv=5,
                          method="predict_proba")[:, 1]

# Schwelle 0.5: Recall 0.85, Precision 0.43
# Schwelle 0.3: Recall 0.92, Precision 0.27
# Schwelle 0.2: Recall 0.93, Precision 0.22

rf.fit(X_train, y_train)
y_pred = (rf.predict_proba(X_test)[:, 1] >= 0.3).astype(int)
print(confusion_matrix(y_test, y_pred))    # -> [ [1792 140] [3 65] ]
```

- Testset mit Schwelle 0,3: 65 von 68 Ausfällen gefunden, dafür 140 Fehllarme auf 2 000 Zeilen
- Welche Schwelle richtig ist, entscheidet die Fachseite: Was kostet ein übersehener Ausfall, was eine unnötige Wartung?

Regression: Hauswerte mit Pipeline

```
from sklearn.linear_model import LinearRegression

df = pd.read_csv("data/california_housing.csv")
X = df.drop(columns="MedHouseVal")
y = df["MedHouseVal"]          # Median-Hauswert in 100.000 USD

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=1)

reg = Pipeline([
    ("scaler", StandardScaler()),
    ("modell", LinearRegression()),
])
reg.fit(X_train, y_train)
```

- Alle acht Merkmale sind Zahlen ohne Lücken: Ein `StandardScaler` genügt als Aufbereitung
- Kein `stratify`, weil die Zielgröße stetig ist

RMSE, MAE und R² auf dem Testset

```
import numpy as np
from sklearn.metrics import (mean_squared_error, mean_absolute_error,
                             r2_score)
from sklearn.model_selection import cross_val_score

rmse_cv = -cross_val_score(reg, X_train, y_train, cv=5,
                           scoring="neg_root_mean_squared_error")
print(rmse_cv.mean().round(3))          # -> 0.727

y_pred = reg.predict(X_test)
print(round(np.sqrt(mean_squared_error(y_test, y_pred)), 3)) # -> 0.727
print(round(mean_absolute_error(y_test, y_pred), 3))         # -> 0.533
print(round(r2_score(y_test, y_pred), 3))                    # -> 0.597
```

- RMSE 0,73 heißt: Die Vorhersage liegt typischerweise um rund 73.000 USD daneben
- scikit-learn gibt Fehlermaße negativ zurück, damit „größer ist besser“ überall gilt

Residuenplot zeichnen

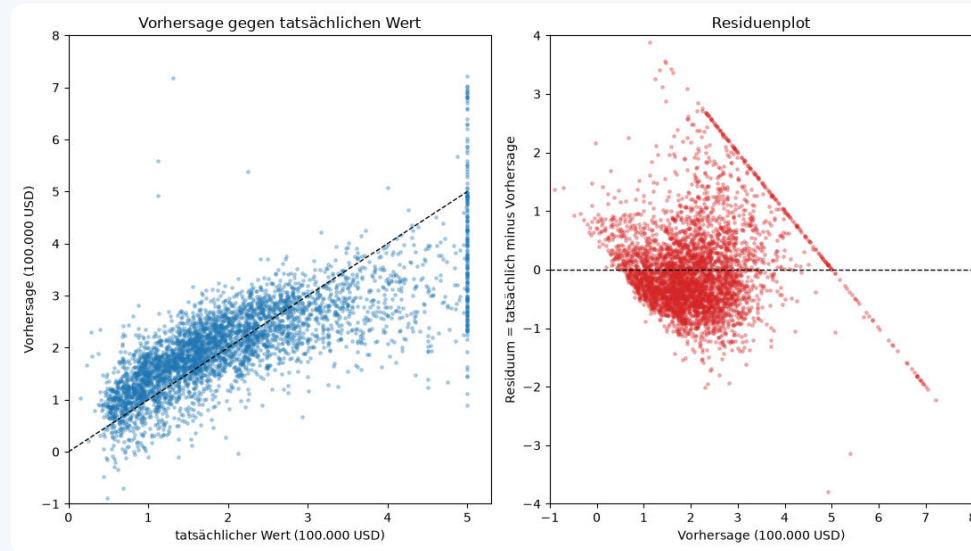
```
import matplotlib.pyplot as plt

residuen = y_test - y_pred

fig, ax = plt.subplots(figsize=(7, 4))
ax.scatter(y_pred, residuen, s=6, alpha=0.3)
ax.axhline(0, color="black", linestyle="--")
ax.set_xlabel("Vorhersage")
ax.set_ylabel("Residuum = tatsächlich minus Vorhersage")
plt.show()
```

- Ein gutes Modell zeigt eine strukturlose Punktwolke um die Nulllinie
- Muster im Residuenplot zeigen, was das Modell systematisch falsch macht

Residuenplot lesen



Clustering: Versicherte segmentieren

```
from sklearn.impute import SimpleImputer
from sklearn.cluster import KMeans

df = pd.read_csv("data/versicherte.csv").drop_duplicates()
merkmale = ["alter", "bmi", "arztbesuche_jahr", "leistungsausgaben_eur"]

segmentierung = Pipeline([
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler()),
    ("kmeans", KMeans(n_clusters=4, n_init=10, random_state=1)),
])

df = df.assign(segment=segmentierung.fit_predict(df[merkmale]))
print(df["segment"].value_counts().sort_index())
```

- `bmi` hat Lücken, `KMeans` bricht bei fehlenden Werten ab: Der Imputer gehört in die Pipeline
- Ohne Skalierung bestimmt allein `leistungsausgaben_eur` die Segmente, weil die Spalte die größten Zahlen hat

Segmentprofil berechnen

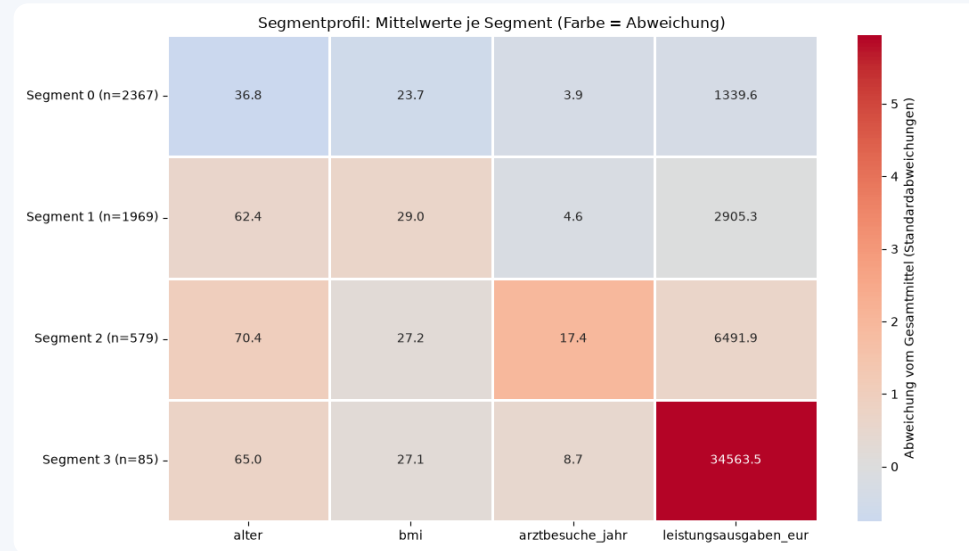
```
profil = (df.groupby("segment")[merkmale].mean().round(1)
          .assign(anzahl=df["segment"].value_counts().sort_index()))
print(profil)

print(df.groupby("segment")[ ["raucher", "zusatzversicherung"] ].mean()
      .round(2))
```

	alter	bmi	arztbesuche_jahr	leistungsausgaben_eur	anzahl
segment					
0	36.8	23.7	3.9	1339.6	2367
1	62.4	29.0	4.6	2905.3	1969
2	70.4	27.2	17.4	6491.9	579
3	65.0	27.1	8.7	34563.5	85

- Das Profil entsteht auf den Originalwerten, nicht auf den skalierten
- Merkmale, die nicht im Clustering waren (`raucher` , `zusatzversicherung`), helfen beim Beschreiben

Segmente beschreiben und benennen



Koeffizienten eines linearen Modells lesen

```
roh = LinearRegression().fit(X_train, y_train)
tabelle = pd.DataFrame({
    "roh": roh.coef_,
    "skaliert": reg.named_steps["modell"].coef_,
}, index=X_train.columns).round(3)
print(tabelle.loc[ ["MedInc", "AveBedrms", "Latitude", "Population"] ])
```

	roh	skaliert
MedInc	0.439	0.830
AveBedrms	0.632	0.321
Latitude	-0.426	-0.910
Population	-0.000	-0.004

- Roh: Änderung der Vorhersage je Einheit des Merkmals. **AveBedrms** wirkt am größten, weil die Spalte nur in einem engen Bereich um 1 schwankt
- Skaliert: Änderung je Standardabweichung. Erst jetzt lassen sich die Beträge vergleichen
- Das Vorzeichen gibt die Richtung an, jeweils bei festgehaltenen übrigen Merkmalen

Koeffizienten vergleichen Sie nur nach Skalierung.

plot_tree mit Merkmalsnamen

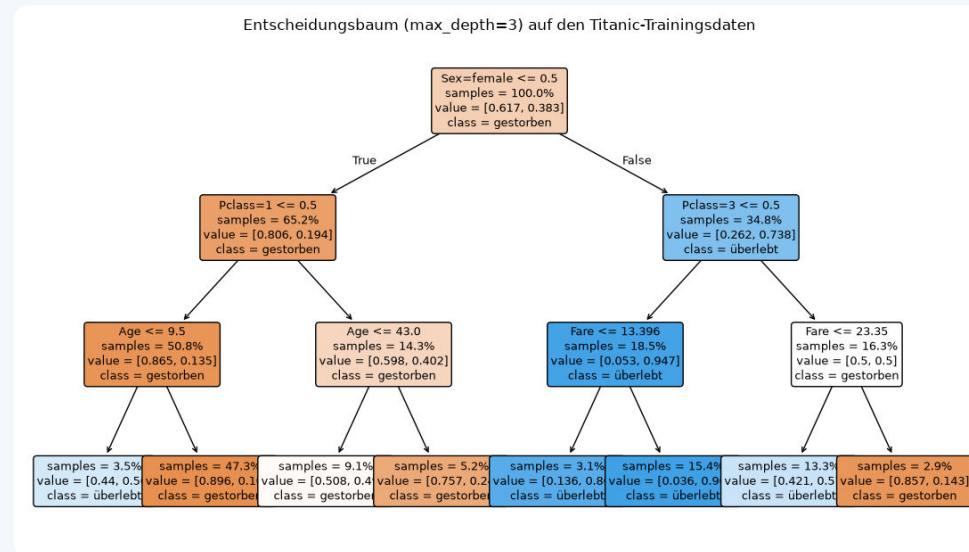
```
from sklearn.tree import DecisionTreeClassifier, plot_tree

baum = Pipeline([
    # preprocess, X_train, y_train: Titanic
    ("prep", preprocess),
    ("modell", DecisionTreeClassifier(max_depth=3, min_samples_leaf=20,
                                     random_state=1)),
]).fit(X_train, y_train)

namen = (pd.Index(baum.named_steps["prep"].get_feature_names_out())
         .str.replace(r"^(num|cat)_", "", regex=True))

fig, ax = plt.subplots(figsize=(11, 6))
plot_tree(baum.named_steps["modell"], feature_names=list(namen),
          class_names=["gestorben", "überlebt"], filled=True,
          rounded=True, impurity=False, proportion=True,
          fontsize=9, ax=ax)
plt.show()
```

Entscheidungsbaum lesen



feature_importances_ des Random Forest

```
rf = Pipeline([
    ("prep", preprocess),
    ("modell", RandomForestClassifier(n_estimators=200, random_state=1)),
])
rf.fit(X_train, y_train)

wichtigkeit = pd.Series(
    rf.named_steps["modell"].feature_importances_,
    index=rf.named_steps["prep"].get_feature_names_out(),
).sort_values(ascending=False)
print(wichtigkeit.head(5).round(3))
```

- Misst, wie stark ein Merkmal über alle Bäume hinweg die Unreinheit der Knoten verringert
- Die Werte summieren sich zu 1 und fallen beim Training ohne Mehraufwand an
- Sie sagen nichts über die Richtung: ob ein hoher Wert die Klasse wahrscheinlicher oder unwahrscheinlicher macht, bleibt offen

Schwächen von `feature_importances_`

- Sie werden aus den Trainingsdaten berechnet: Ein Merkmal, mit dem der Wald nur auswendig lernt, erscheint trotzdem wichtig
- Merkmale mit vielen verschiedenen Werten (stetige Zahlen, Kennungen) werden bevorzugt, weil sie mehr Möglichkeiten zum Teilen bieten
- Bei stark zusammenhängenden Merkmalen kann jedes einzelne unbedeutend wirken, obwohl die gemeinsame Information wichtig ist
- One-Hot-Spalten einer Kategorie werden einzeln bewertet

scikit-learn warnt in der eigenen Dokumentation: Wichtigkeiten aus der Unreinheit können bei Merkmalen mit vielen Ausprägungen irreführen.

permutation_importance als robustere Alternative

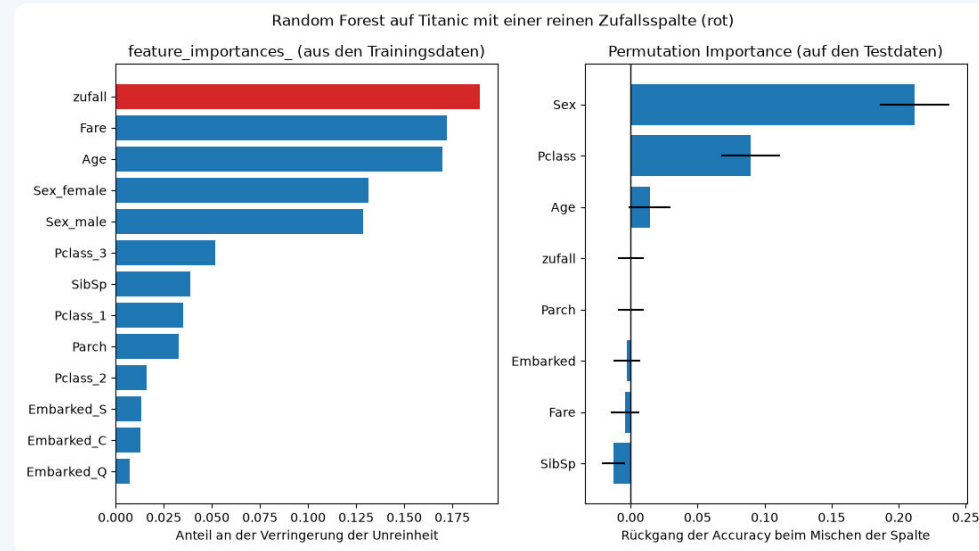
```
from sklearn.inspection import permutation_importance

result = permutation_importance(rf, X_test, y_test,
                               n_repeats=20, random_state=1)

perm = pd.DataFrame({
    "mittel": result.importances_mean,
    "std": result.importances_std,
}, index=X_test.columns).sort_values("mittel", ascending=False)
print(perm.round(3))
```

- Idee: die Werte einer Spalte zufällig durchmischen und messen, wie stark die Kennzahl auf den Testdaten fällt
- Arbeitet mit der ganzen Pipeline und den Originalspalten: `Sex` ist ein Merkmal, nicht zwei
- Funktioniert für jedes Modell, nicht nur für Bäume
- `n_repeats=20` wiederholt das Mischen und liefert eine Streuung dazu

Beide Verfahren im Vergleich

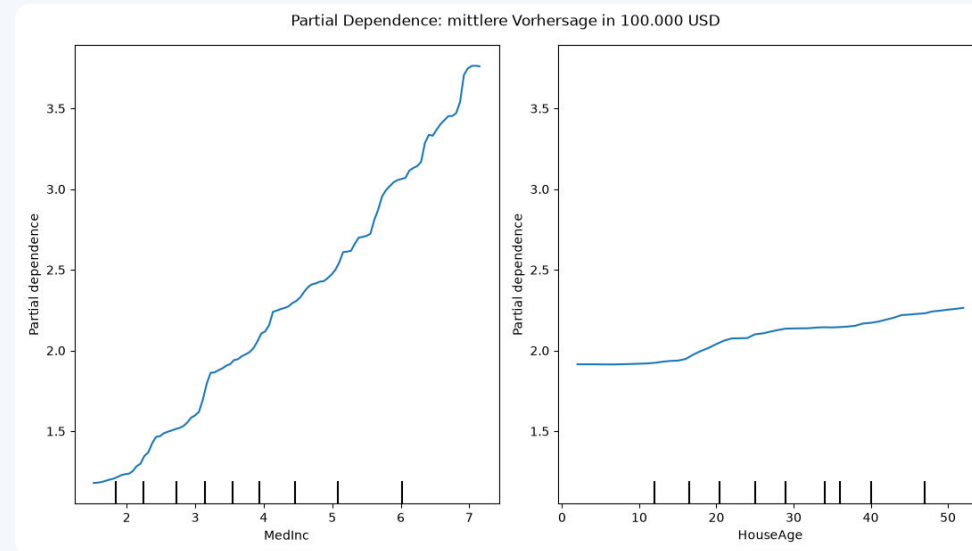


Partial Dependence

```
from sklearn.ensemble import (
    RandomForestRegressor)
from sklearn.inspection import (
    PartialDependenceDisplay)

# X_train, y_train: California Housing
wald = RandomForestRegressor(
    n_estimators=100,
    min_samples_leaf=5,
    random_state=1, n_jobs=-1)
wald.fit(X_train, y_train)

PartialDependenceDisplay.from_estimator(
    wald, X_test.sample(1000,
                        random_state=1),
    features=["MedInc", "HouseAge"])
plt.show()
```



- Zeigt die mittlere Vorhersage, wenn ein Merkmal variiert und alle anderen bleiben, wie sie sind
- Beantwortet die Frage nach der Richtung, die Wichtigkeiten offenlassen

Grenzen der Interpretation

- **Zusammenhang ist keine Ursache.** Das Modell nutzt, was mit der Zielgröße zusammen auftritt. Ob eine Änderung des Merkmals die Zielgröße ändert, beantwortet es nicht
- **Zusammenhängende Merkmale wirken einzeln unwichtig.** Zwei fast gleiche Spalten können beide unbedeutend erscheinen, obwohl die Information dahinter entscheidend ist
- **Die Interpretation gilt für das Modell, nicht für die Welt.** Ein anderes Modell mit gleicher Güte kann andere Merkmale bevorzugen
- **Ein schlechtes Modell hat nichts zu erklären.** Erst die Güte auf Testdaten prüfen, dann interpretieren

Formulieren Sie „das Modell stützt sich vor allem auf X“, nicht „X verursacht Y“.

Ausblick: SHAP und LIME

	SHAP	LIME
Frage	Welchen Beitrag hat jedes Merkmal zu dieser einen Vorhersage?	Welches einfache Modell beschreibt das Verhalten in der Nähe dieser einen Zeile?
Idee	verteilt die Abweichung vom Durchschnitt nach einem Verfahren aus der Spieltheorie (Shapley-Werte) auf die Merkmale	verändert die Zeile leicht, beobachtet die Vorhersagen und passt lokal ein lineares Modell an
Ergebnis	Beitrag je Merkmal und Zeile, über viele Zeilen auch eine globale Übersicht	Gewichte je Merkmal für eine einzelne Zeile
Aufwand	eigenes Paket, bei großen Daten rechenintensiv	eigenes Paket, Ergebnis schwankt mit der Zufallsstichprobe

- Beide erklären **einzelne** Vorhersagen, das können Permutation Importance und Partial Dependence nicht
- Die Grenzen der vorigen Folie gelten unverändert

Ergebnisse für die Fachabteilung darstellen

Abschnitt im Ergebnisbericht	Inhalt
Frage und Nutzen	Welche Entscheidung soll das Modell unterstützen?
Daten	Quelle, Zeitraum, Zeilenzahl, weggelassene Spalten und warum
Vorgehen	Aufteilung, Pipeline, Art der Prüfung, in zwei bis drei Sätzen
Güte	eine zur Fragestellung passende Kennzahl, daneben die einfache Baseline
Fehlerbild	Konfusionsmatrix oder Residuenplot in Worten: Was wird übersehen, was fälschlich gemeldet?
Wichtigste Merkmale	Permutation Importance als Balkendiagramm, Richtung aus Partial Dependence oder Koeffizienten
Grenzen	für welche Fälle das Modell nicht gilt, bekannte Schwächen der Daten
Wiederholbarkeit	Datenstand, Versionen, <code>random_state</code> , Ablageort der Pipeline

Übersetzen Sie Kennzahlen in Fälle: „Von 68 Ausfällen findet das Modell 65 und meldet 140 Mal falschen Alarm" statt „Recall 0,96".

Zusammenfassung

- Derselbe Ablauf aus Spaltenwahl, Split, Pipeline, Cross-Validation und einmaligem Testset trägt Klassifikation, Regression und Clustering
- Bei 3,4 % Ausfällen erreicht ein nutzloses Modell 97 % Accuracy: Recall, Precision, `class_weight="balanced"` und eine bewusst gewählte Schwelle sind die Werkzeuge
- Koeffizienten vergleichen Sie nur nach Skalierung, ein Baum der Tiefe 3 lässt sich als Regeln vorlesen
- `feature_importances_` bevorzugt Spalten mit vielen Werten, `permutation_importance` auf Testdaten ist die robustere Wahl
- Jede Interpretation beschreibt das Modell, nicht die Ursachen in der Welt, und gehört mit Baseline und Grenzen in den Bericht

Teil 11: Transfer in die Praxis

Vom Kursbeispiel zum ersten eigenen Modell mit eigenen Daten.

Was Sie in diesem Teil lernen

- Sie ordnen die Themen des Kurses auf einer Landkarte ein.
- Sie prüfen mit einer Checkliste, ob ein Vorhaben reif für ein erstes Modell ist.
- Sie erkennen die typischen Fehler beim Einstieg und wissen, wie Sie sie vermeiden.
- Sie machen aus einem Notebook ein Skript, das morgen dasselbe Ergebnis liefert wie heute.
- Sie wissen, womit Sie nach dem Kurs weiterlernen.

Rückblick: die Landkarte des Kurses



- Jeder Kasten baut auf dem vorigen auf: ohne saubere Tabelle kein brauchbares Modell.
- Die meiste Arbeitszeit steckt in der linken Hälfte (Daten lesen, prüfen, aufbereiten).
- Deep Learning folgt demselben Ablauf, nur mit einem anderen Modelltyp im Kasten „Modelle“.

Checkliste vor dem ersten eigenen Modell

Punkt	Frage, die Sie beantworten können müssen
Fragestellung	Welche Entscheidung soll das Ergebnis unterstützen? In einem Satz.
Zielgröße	Welche Spalte sagen Sie voraus, und liegt sie zum Zeitpunkt der Vorhersage schon vor?
Datenlage	Kennen Sie Verteilungen, Ausreißer und fehlende Werte? Sind IDs und Leckquellen aussortiert?
Baseline	Wie gut ist die einfachste Lösung (häufigste Klasse, Mittelwert, bestehende Regel)?
Kennzahl	Welche Fehler sind fachlich teuer, und welche Kennzahl bildet das ab?
Testdaten	Liegt ein Teil der Daten unberührt zurück, bis das Modell fertig ist?

Können Sie einen Punkt nicht beantworten, klären Sie ihn zuerst. Modellcode schreiben Sie danach.

Wann sich ML lohnt und wann nicht

ML lohnt sich, wenn

- ausreichend historische Daten vorliegen,
- eine wiederkehrende Vorhersage oder Einordnung einen messbaren Nutzen hat,
- sich die Zusammenhänge nicht mehr mit einfachen Regeln beschreiben lassen,
- Sie Fehlentscheidungen bewerten können (was kostet ein Fehlalarm, was ein übersehener Fall?).

ML lohnt sich eher nicht, wenn

- es kaum Daten, aber viele Ausnahmen und Sonderfälle gibt,
- die eigentliche Entscheidung fachlich noch nicht klar formuliert ist,
- eine bekannte Regel das Problem schon zuverlässig löst,
- sich weder Nutzen noch Fehlerkosten bewerten lassen.

Voraussetzungen für ein brauchbares Modell

- Ein gutes Modell ist Teil eines größeren Ablaufs: Daten beschaffen, prüfen, aufbereiten, bewerten, betreiben.
- Datenzugang, Datenqualität und eine saubere Zielgröße sind genauso wichtig wie der Algorithmus.
- Viele Vorhaben scheitern nicht am Modell, sondern an unreifen Daten oder einer unklaren Fragestellung.
- Ein Modell wird höchstens so gut wie seine Zielgröße: uneinheitlich oder verspätet erfasste Werte begrenzen jedes Ergebnis.

Bevor Sie das Modell wechseln, prüfen Sie die Daten: `df.info()`, `df.describe()`, `df.isna().sum()`, `df.duplicated().sum()`.

Typische Fehler beim Einstieg

Fehler	Woran Sie ihn erkennen	Gegenmittel
Datenleck	Testergebnis unrealistisch gut	Erst <code>train_test_split</code> , dann alles Weitere in einer <code>Pipeline</code>
Nur Accuracy	95 % Accuracy, aber die seltene Klasse wird nie erkannt	Konfusionsmatrix, Precision, Recall, F1
Kein Baseline-Vergleich	Niemand weiß, ob 0,80 gut ist	<code>DummyClassifier</code> oder <code>DummyRegressor</code> zuerst
Zellen in falscher Reihenfolge	Notebook läuft nach Neustart nicht mehr durch	„Restart Kernel and Run All“ vor dem Weitergeben
Keine festen Zufallswerte	Jeder Lauf liefert andere Zahlen	<code>random_state=1</code> bei Split, Modell und CV
Datentypen ungeprüft	Datum oder Zahl steht als Text (<code>object</code> oder <code>str</code>) in <code>df.dtypes</code>	<code>pd.to_datetime</code> , <code>pd.to_numeric</code> , <code>astype</code>
Zufälliger Split bei Zeitbezug	Modell lernt aus der Zukunft	Nach Datum trennen: ältere Daten trainieren, neuere testen

Personenbezogene und besonders schützenswerte Daten

- Sobald sich Daten auf eine identifizierbare Person beziehen, gilt die **DSGVO**, auch für Auswertungen und Modelle im Notebook.
- **Gesundheitsdaten** sind besondere Kategorien personenbezogener Daten nach **Art. 9 DSGVO**: Die Verarbeitung ist grundsätzlich untersagt, Ausnahmen regelt Art. 9 Abs. 2.
- Arbeiten Sie nur mit den Spalten, die Sie für die Frage brauchen (Datenminimierung), und nur für den festgelegten Zweck (Zweckbindung).
- **Ohne Namen ist nicht anonym**: Geschlecht, Geburtsdatum und Postleitzahl können einen Datensatz eindeutig machen. Auch pseudonymisierte Daten bleiben personenbezogen (Art. 4 Nr. 5 DSGVO).
- Zum Lernen und Ausprobieren: **synthetische oder anonymisierte Daten**, so wie die Versichertentabelle in diesem Kurs. Synthetische Daten sind aber nicht automatisch anonym.

Vom Notebook zum wiederholbaren Skript

```
import pandas as pd
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split

def lade_daten(pfad):
    df = pd.read_csv(pfad)
    X = df[ ["Pclass", "SibSp", "Parch", "Fare"] ]
    X = X.assign(weiblich=df["Sex"] == "female")
    return X, df["Survived"]

def trainiere(X, y):
    X_tr, X_te, y_tr, y_te = train_test_split(
        X, y, test_size=0.2, random_state=1, stratify=y)
    modell = RandomForestClassifier(random_state=1).fit(X_tr, y_tr)
    return modell, modell.score(X_te, y_te)

if __name__ == "__main__":
    modell, acc = trainiere(*lade_daten("data/titanic.csv"))
    print(f"Accuracy Test: {acc:.3f}")    # -> Accuracy Test: 0.810
```

- Jeder Schritt ist eine **Funktion** mit Eingabe und Rückgabe, nichts hängt von der Reihenfolge einzelner Zellen ab.
- Start aus dem Terminal ab Repo-Wurzel: `python train.py`

Umgebung und Stand festhalten

Pakete festschreiben

```
conda env export --from-history > environment.yml  
pip freeze > requirements.txt
```

- `environment.yml` beschreibt die conda-Umgebung, `requirements.txt` die pip-Pakete mit Version.
- Auf einem anderen Gerät:
`conda env create -f environment.yml`

Stand mit Git sichern

```
git status  
git add train.py environment.yml  
git commit -m "Erstes Modell: Baseline + Random Forest"
```

- Ein Commit pro abgeschlossenem Schritt, mit einer Nachricht, die das Ergebnis nennt.
- Daten mit Personenbezug gehören nicht ins Repository (`.gitignore`).

Wiederholbar heißt: gleicher Code, gleiche Paketversionen, gleiche Daten, gleicher random_state.

Ein erstes eigenes Projekt zuschneiden

- **Klein anfangen:** eine Tabelle, eine Frage, ein Notebook. Kein Datenbankanschluss, kein Dashboard.
- **Vorhandene Tabelle nehmen:** eine Auswertung, die Sie heute schon in Excel pflegen und deren Spalten Sie fachlich kennen.
- **Eine Frage formulieren:** „Welche Merkmale hängen mit X zusammen?“ oder „Lässt sich Y aus den übrigen Spalten vorhersagen?“
- **Erst beschreiben, dann modellieren:** `describe` , `groupby` , zwei bis drei Diagramme. Oft ist die Frage damit schon beantwortet.
- **Dann der Kursablauf:** Baseline, einfaches Modell, Kennzahl auf zurückgehaltenen Testdaten, Interpretation.

Legen Sie das erste eigene Notebook in den nächsten Tagen an. Training wirkt vor allem dann, wenn Anwendung und Follow-up folgen (Salas et al. 2012).

Literatur und Lernpfad

Schritt	Titel	Wofür
1. Python festigen	Allen B. Downey: <i>Think Python</i> , 2. Auflage (frei bei Green Tea Press)	Sprache von Grund auf, mit Aufgaben
2. Daten	Wes McKinney: <i>Python for Data Analysis</i> , 3. Auflage (frei: wesmckinney.com/book)	pandas vom Autor der Bibliothek
2. Daten	Jake VanderPlas: <i>Python Data Science Handbook</i> (frei: jakevdp.github.io/PythonDataScienceHandbook)	NumPy, pandas, matplotlib, scikit-learn in einem Band
3. Machine Learning	Aurélien Géron: <i>Hands-On Machine Learning with Scikit-Learn, Keras and TensorFlow</i> (O'Reilly, 2019)	Durchgehende Praxisprojekte, Pipelines
3. Machine Learning	scikit-learn User Guide (scikit-learn.org)	Nachschlagen: jedes Verfahren mit Beispiel
4. Deep Learning	Zhang, Lipton, Li, Smola: <i>Dive into Deep Learning</i> (frei: d2l.ai)	Neuronale Netze mit PyTorch-Code

Zusammenfassung

- Der Ablauf bleibt gleich: Daten prüfen, Baseline, einfaches Modell, passende Kennzahl, zurückgehaltene Testdaten, Interpretation.
- Datenqualität und eine klare Fragestellung entscheiden mehr als die Wahl des Algorithmus.
- Fünf Gewohnheiten schützen vor den häufigsten Fehlern: zuerst trennen, `Pipeline` nutzen, mehr als Accuracy ansehen, `random_state=1` setzen, Notebook komplett neu durchlaufen lassen.
- Gesundheitsdaten sind besondere Kategorien nach Art. 9 DSGVO: üben Sie mit synthetischen oder anonymisierten Daten und klären Sie echte Daten vorab.

Vielen Dank

Dr.-Ing. Grigory Devadze

Kursmaterial: <https://github.com/grigory-consulting/python-ml-grundlagen>